

ACM/ICPC

Rejected

2026 年 10 月 5 日

目录

1 不要忘记	1	3 图论	15
1.1 土制 cph(仅 linux)	1	3.1 最短路	15
1.2 土制 cph(全平台)	1	3.2 判奇环(无向图)	15
1.3 随机数	1	3.3 最小生成树	15
1.4 对拍	2	3.4 拓扑排序	15
1.5 解除 python 大数限制	2	3.5 LCA	15
1.6 常用函数	2	3.6 树的直径与中心	15
1.7 高精度	2	3.7 点分治与树的重心	16
1.8 vector 相关	2	3.8 基环树	16
1.9 整数域三分	2	3.9 Hierholzer 求欧拉路	16
1.10 运行时间	2	3.10 DSU on tree	16
1.11 记得快读快输	2	3.11 tarjan	17
1.12 !!! 数据范围	2	3.12 树哈希	17
1.13 __int128 的使用	2	3.13 Bellman-Ford / SPFA (判负环)	17
1.14 快读模板	3	3.14 Floyd 全源最短路	17
1.15 数组离散化	3	3.15 树上倍增	17
1.16 随机数生成器	3	3.16 树的直径	17
1.17 随机图	3	3.17 树的重心	17
1.18 对拍器	3	3.18 最大流	18
2 数据结构	3	3.19 最近公共祖先(LCA)	18
2.1 对顶 multiset	3	3.20 树链剖分	18
2.2 分块	4	3.21 最大路匹配	20
2.3 带权并查集	4	3.22 Dijkstra 算法	20
2.4 Trie	4	3.23 分层图最短路	20
2.5 ST 表	5	3.24 双向广搜	21
2.6 树状数组	5	3.25 tarjan(强连通分量)	21
2.7 线段树	5	3.26 边双连通分量	21
2.8 替罪羊树	6	3.27 割边	22
2.9 逆序对	8	3.28 点双连通分量	22
2.10 笛卡尔树	8	3.29 割点	22
2.11 可持久化	8	4 数学	23
2.12 线段树模板	9	4.1 线性筛	23
2.13 主席树(可持久化线段树)	9	4.2 因数	23
2.14 开点线段树	10	4.3 拉格朗日插值	23
2.15 权值线段树	10	4.4 快速幂与乘法逆元	24
2.16 线段树上最值操作	10	4.5 排列组合数	24
2.17 线段树上序列操作	11	4.6 Lucas	24
2.18 非经典线段树	12	4.7 卡特兰数	24
2.19 Kruskal 重构树	13	4.8 数论分块	24
2.20 左偏树	13	4.9 高斯消元	24
2.21 虚树的构建	14	4.10 矩阵快速幂	24
		4.11 自适应辛普森法	24
		4.12 多项式	25
		4.13 快速幂	25
		4.14 乘法逆元	25
		4.15 组合数求法	25

4.16 线性基	26
4.17 矩阵快速幂及高斯消元	26
4.18 线性筛求质因子个数 (Omega 函数)	27
4.19 线性筛 (O(n))	27
4.20 位运算的部分性质	27
4.21 builtin	28
5 字符串	28
5.1 前缀函数与 KMP	28
5.2 Manacher	28
5.3 字符串哈希	29
5.4 AC 自动机	29
5.5 后缀自动机 SAM	30
5.6 KMP 算法	30
5.7 Trie 树	31
5.8 string 常用函数	31
6 动态规划	32
6.1 数位 DP	32
6.2 单调队列优化 (滑动窗口)	32
6.3 无向图上回路计数	32
6.4 SOSDP	32
6.5 树形 DP	32
6.6 最长不下降子序列	33
7 计算几何	33
7.1 define	33
7.2 向量	33
7.3 点线操作	34
7.4 多边形与凸包	34
7.5 计算几何模板	34
8 其它	40
8.1 二维差分与前缀和	40
8.2 CDQ 分治	40
8.3 矩阵加速	40
8.4 cdq 斜率优化	41
8.5 并查集	42
8.6 FFT (快速傅里叶变换)	43
8.7 NTT (数论变换)	43
8.8 扫描线算法	44
8.9 最长上升子序列	44
8.10 全排列输出	44
8.11 位运算用法	44
8.12 Map 的使用	45
8.13 优先队列 (堆)	45
8.14 扫描线 + 线段树 (矩形面积并)	45
8.15 建树参考	46
8.16 Hash 树	46

1 不要忘记

1.1 土制 cph(仅 linux)

```

1 from os import system as e, listdir as l
2 from sys import argv
3 _, q, f = argv # q: 题号, f: 文件名
4
5 if "py" in f:
6     r = f"python3 {f}"
7 else:
8     e(f"g++ -std=c++23 -o2 -Wall {f}.cpp -o {f}")
9     r = "." + f
10
11 #
12 d = "samples-" + q.capitalize()
13 for i in l(d):
14     if not "in" in i: continue
15     print(i)
16     p = d + "/" + i[:-2]
17     if e(f"timeout 2 {r}<{p}in>{p}out"): print("TLE or RE")
18 else:
19     with open(f"{p}out") as o, open(f"{p}ans") as a:
20         print("AC" if o.read().split()==a.read().split() else "
WA")

```

1.2 土制 cph(全平台)

```

1 from os import listdir as l, system as e
2 from os.path import join as jp
3 from sys import argv as a
4 import subprocess as s
5 _,q,f = a
6 if "py" in f:
7     cmd = ["python3", f]
8 else:
9     e(f"g++ -std=gnu++20 -O2 -Wall {f}.cpp -o {f}")
10    cmd = jp(".", f"{f}.exe")
11
12 d='samples-'+q.capitalize()
13 g,r,p,n='\033[32m','\033[31m','\033[35m','\033[0m'
14 for i in l(d):
15     if not 'in' in i: continue
16     t=jp(d,i[:-2])
17     print(i,'测评结果 ',end='',flush=1)
18     try:
19         k = s.run(cmd, timeout=2, stdin=open(f"{t}in"),
20                   stdout=open(f"{t}out", 'w'), stderr=s.PIPE,
21                   text=1)
22         c = lambda o,a : o.read().split() == a.read().split()
23         # c = lambda o,a : all(abs(float(x)-float(y))/max(1,
24                     abs(float(y)))<=1e-4
25                     for x,y in zip(o.read().split(),a.read
26                     ().split()))
27         print(f'{g}AC{n}' if c(open(f"{t}out"),open(f"{t}ans"))
28               ) else f'{r}WA{n}')
29         if k.stderr:print(f'{r}{k.stderr}{n}')
30     except s.TimeoutExpired as k:
31         print(f'{p}TLE or RE{n}')
32         if k.stderr: print(f'{r}{k.stderr}{n}')

```

1.3 随机数

```

1 mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().
count());
2 int Rand(int x, int y) {
3     return uniform_int_distribution<int>(x, y)(rng);
4 }
5 shuffle(a.begin(), a.end(), rng);

```

1.4 对拍

```

1 // checker.cpp
2 while (1) {
3     system("A_gen > _data.in"); // Linux 下换成 ./xxx
4     system("A_bf < _data.in > A.ans");
5     system("A < _data.in > A.ans");
6     if (system("fc A.out A.ans")) { // Linux 下换成 diff
7         cout << "WA\n"; return 0;
8     } else {
9         cout << "AC\n";

```

```
10 }
11 }
```

1.5 解除 python 大数限制

```
1 import sys
2 sys.set_int_max_str_digits(100005)
```

1.6 常用函数

```
1 // 快读
2 inline int read()
3 {
4     int x=0,f=1;char ch=getchar();
5     while (ch<'0' || ch>'9'){if (ch=='-') f=-1;ch=getchar();}
6     while (ch>='0' && ch<='9'){x=x*10+ch-48;ch=getchar();}
7     return x*f;
8 }
9
10 // 取模
11 const int MOD=998244353;
12 ll add(ll x, ll y) { return (x+y)%MOD; }
13 ll del(ll x, ll y) { return add(x, MOD-y); }
14 ll mul(ll x, ll y) { return (x*y)%MOD; }
15 ll qpow(ll a, ll b = MOD - 2) {
16     ll res = 1;
17     for (; b; b >>= 1) {
18         if (b & 1) { res = mul(res, a); }
19         a = mul(a, a);
20     }
21     return res;
22 }
23
24 // 计算一个整数的二进制表示中有多少个 1
25 __builtin_popcountll(i);
26 std::popcount((unsigned long long)i); // c++20
27
28 // 计算一个整数的二进制表示中最高位的 1
29 __lg(i); // i 最多 32 位
```

1.7 高精度

```
1 cout << fixed << setprecision(15); // 输出精度
2
3 istream& operator>> (istream& is, __int128 &x)
4 {
5     string s; is>>s;
6     bool f=false;
7     __int128 ans = 0;
8     for (auto c: s) {
9         if (c == '-') { f = true; }
10        if (isdigit(c)) { ans = ans * 10 + (c-'0'); }
11    }
12    x = f? -ans: ans;
13    return is;
14 }
15 ostream& operator<< (ostream& os, __int128 &x)
16 {
17     string ans;
18     function<void(__int128)> write = [&](__int128 x){
19         if (x < 0) { ans += '-'; x=-x; }
20         if (x > 9) { write(x/10); }
21         ans += '0'+x%10;
22     };
23     write(x); return os << ans;
24 }
```

1.8 vector 相关

```
1 // 多维 vector, 需要 c++17, c++14 只能老老实实写 vector<vector
2 <int>>
3 vector a(n+1, vector (n+1, vector<int>(n+1)));
4
5 // 最值
6 int mx = *min_element(vec.begin(), vec.end(), cmp);
7 int mn = *min_element(vec.begin(), vec.end(), cmp);
8
9 // 去重与离散化
10 for (int i = 1; i <= n; ++i) { vec.push_back(a[i]); }
11 sort(vec.begin(), vec.end());
```

```
11 vec.erase(unique(vec.begin(), vec.end()), vec.end());
12 for (int i = 1; i <= n; ++i) {
13     idx[i]=lower_bound(vec.begin(), vec.end(), a[i])-vec.begin()
14         +1; // 下标从1开始
15 }
16 // 前缀和
17 vector <ll> a(n), s(n);
18 partial_sum(a.begin(), a.end(), s.begin());
19 ll sum = accumulate(a.begin(), a.end(), 0LL);
20
21 // 填充 1~n, 令 a[i]=i
22 iota(a.begin(), a.end(), 0);
```

1.9 整数域三分

```
1 // 整数域三分求单峰函数最大值
2 int l, r;
3 while (l + 2 <= r) {
4     int w = (r - l) / 3;
5     int m1 = l + w; int v1 = f(m1);
6     int m2 = r - w; int v2 = f(m2);
7     if (v1 <= v2) {
8         l = m1 + 1;
9     } else {
10        r = m2 - 1;
11    }
12 }
13 cout << max(f(l), f(r)) << "\n";
```

1.10 运行时间

```
1 auto start = chrono::high_resolution_clock::now();
2 auto stop = chrono::high_resolution_clock::now();
3 auto duration = std::chrono::duration_cast<std::chrono::
4     milliseconds>(stop - start).count();
5 if (duration > 998) { cout << ans << "\n"; exit(0); }
```

1.11 记得快读快输

```
1 ios::sync_with_stdio(0);
2 cin.tie(0), cout.tie(0);
```

1.12 !!! 数据范围

注意爆int

```
1 long long: 8 字节, -9,223,372,036,854,775,808到9
2             ,223,372,036,854,775,807
3 unsigned long long: 8 字节, 0到18,446,744,073,709,551,615
4 如果爆long long 就用__int128,但是注意输入输出
5
6 尽量不要使用double输入, 容易超时
```

1.13 __int128 的使用

```
1 #define int __int128
2 inline void read(int &n){
3     int x=0,f=1;
4     char ch=getchar();
5     while(ch<'0' || ch>'9'){
6         if(ch=='-') f=-1;
7         ch=getchar();
8     }
9     while(ch>='0' && ch<='9'){
10        x=(x<<1)+(x<<3)+(ch^48);
11        ch=getchar();
12    }
13    n=x*f;
14 }
15 inline void print(int n){
16     if(n<0){
17         putchar('-');
18         n*=-1;
19     }
20     if(n>9) print(n/10);
21     putchar(n % 10 + '0');
22 }
23 #undef int
```

1.14 快读模板

```
1 #define re register
2 #define il inline
3 il int read()
4 {
5     re int x=0,f=1;char c=getchar();
6     while(c<'0' || c>'9'){if(c=='-') f=-1;c=getchar();}
7     while(c>='0' && c<='9') x=(x<<3)+(x<<1)+(c^48),c=getchar();
8     return x*f;
9 }
```

1.15 数组离散化

```
1 sort(vals.begin(), vals.end());
2 vals.erase(unique(vals.begin(), vals.end()), vals.end());
```

1.16 随机数生成器

```
1 mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().
2 count());
3 int Rand(int x, int y) {
4     return uniform_int_distribution<int>(x, y)(rng);
5 }
6 shuffle(a.begin(), a.end(), rng);
```

1.17 随机图

```
1 // n 为节点个数
2 // m 为边的个数, 默认为 -1, 表示建树
3 // root 为根节点编号, 默认为 -1,
4 vector<array<int,2>> Graph(int n, int m=-1, int root=-1)
5 {
6     assert(m== -1 || (n-1<=m && m<=1LL*n*(n-1)/2));
7     assert(root== -1 || (1<=root && root<=n));
8     vector<array<int,2>> E;
9     set<array<int,2>> diff;
10    // 先建一棵以 0 为根的树, 然后选根偏移
11    for (int i = 1; i < n; ++i) {
12        E.push_back( { i, Rand(0, i-1) } );
13    }
14    if (root == -1) { root = Rand(0, n-1); }
15    for (auto& [x, y]: E) {
16        x = (x+root) % n + 1;
17        y = (y+root) % n + 1;
18        diff.insert( { x, y } );
19        diff.insert( { y, x } ); // 无向图中需要添加这条反向边
20    }
21    // 从树建树
22    if (m != -1) {
23        for (int i = n-1; i < m; ++i) {
24            while (1) {
25                int x = Rand(1, n);
26                int y = Rand(1, n);
27                if (x == y || diff.count({x,y})) { continue; }
28                E.push_back( { x, y } );
29                diff.insert( { x, y } );
30                diff.insert( { y, x } ); // 无向图中需要添加这条反向边
31                break;
32            }
33        }
34    }
35    shuffle(E.begin(), E.end(), rng);
36    return E;
37 }
```

1.18 对拍器

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 int main(){
4     int t=0;
5     while(true){
6         cout<<"test: "<<t<<endl;
7         system("data.exe>data.in");//data.exe 随机数生成器
8         system("std.exe<data.in>data.out");//std.exe 标准程序
9         system("solve.exe<data.in>solve.out");//solve.exe 暴力求解
10        算法
11        if(system("fc std.out solve.out>diff.log")){
```

```
11         cout<<"WA"<<endl;
12         break;
13     }
14 }
15 cout<<"AC\n";
16 }
17 /*
18 // checker.cpp
19 while (1) {
20     system("A_gen > _data.in"); // Linux 下换成 ./xxx
21     system("A_bf < _data.in > A.ans");
22     system("A < _data.in > A.ans");
23     if (system("fc A.out A.ans")) { // Linux 下换成 diff
24         cout << "WA\n"; return 0;
25     } else {
26         cout << "AC\n";
27     }
28 }*/
```

2 数据结构

2.1 对顶 multiset

```
1 //对顶 multiset 数据结构主要用于动态维护一组数的中位数（或第 k
2 小/大等顺序统计量）
3 struct PairingMultiset {
4     int sz;
5     multiset<int> minH, maxH;
6     ll minS, maxS;
7     void clear()
8     {
9         minH.clear(); maxH.clear();
10        sz = minS = maxS = 0;
11    }
12    void move()
13    {
14        while ((int)minH.size() < (sz+1)/2) {
15            minS += *maxH.rbegin();
16            minH.insert(*maxH.rbegin());
17            maxS -= *maxH.rbegin();
18            maxH.erase(maxH.find(*maxH.rbegin()));
19        }
20        while ((int)minH.size() > (sz+1)/2) {
21            maxS += *minH.begin();
22            maxH.insert(*minH.begin());
23            minS -= *minH.begin();
24            minH.erase(minH.find(*minH.begin()));
25        }
26    }
27    void add(int x)
28    { // 向对顶堆中插入一个元素
29        ++sz;
30        if (minH.empty() || x >= *minH.begin()) {
31            minS += x;
32            minH.insert(x);
33        } else {
34            maxS += x;
35            maxH.insert(x);
36        }
37        move();
38    }
39    void del(int x)
40    { // 从对顶堆中删除一个x
41        --sz;
42        if (x >= *minH.begin()) {
43            minS -= x;
44            minH.erase(minH.find(x));
45        } else {
46            maxS -= x;
47            maxH.erase(maxH.find(x));
48        }
49        move();
50    }
51    ll med()
52    { // 求中位数
53        return *minH.begin();
54    }
55    ll dis_to_med()
56    { // 求对顶堆中的数字到中位数的距离和
57        ll minSZ=(sz+1)/2, maxSZ=sz-minSZ, m=med();
58        return (minS-minSZ*m)+(maxSZ*m-maxS);
59    } p;
```

2.2 分块

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 const int maxn=5e5+10;
4 const int maxb=1e3+10;
5 int arr[maxn],sv[maxn],n,m;
6 int blen,bnum;
7 int bi[maxn],bl[maxn],br[maxn];
8 void build(){
9     blen=sqrt(n);
10    bnum=(n+blen-1)/blen;
11    for(int i=1;i<=n;i++){
12        bi[i]=(i-1)/blen+1;
13    }
14    for(int i=1;i<=bnum;i++){
15        bl[i]=(i-1)*blen+1;
16        br[i]=min(i*blen,n);
17    }
18    for(int i=1;i<=n;i++) sv[i]=arr[i];
19    for(int i=1;i<=bnum;i++){
20        sort(sv+bl[i],sv+br[i]+1);
21    }
22 }
23 int getcnt(int i,int v){
24     int l=bl[i],r=br[i],m,res=0;
25     while(l<=r){
26         m=(l+r)>>1;
27         if(sv[m]>=v){
28             res+=r-m+1;
29             r=m-1;
30         }else l=m+1;
31     }
32     return res;
33 }
34 void update(int ii,int v){
35     int l=bl[bi[ii]];
36     int r=br[bi[ii]];
37     arr[ii]=v;
38     for(int i=l;i<=r;i++) sv[i]=arr[i];
39     sort(sv+l,sv+r+1);
40 }
41 int query(int l,int r,int v){
42     int res=0;
43     if(bi[l]==bi[r]){
44         for(int i=l;i<=r;i++){
45             if(arr[i]>=v) res++;
46         }
47     }else{
48         for(int i=l;i<=br[bi[l]];i++){
49             if(arr[i]>=v) res++;
50         }
51         for(int i=bl[bi[r]];i<=r;i++){
52             if(arr[i]>=v) res++;
53         }
54         for(int i=bi[l];i<bi[r];i++){
55             res+=getcnt(i,v);
56         }
57     }
58     return res;
59 }
60 int main(){
61     cin>>n;
62     for(int i=1;i<=n;i++){
63         cin>>arr[i];
64     }
65     build();
66     cin>>m;
67     for(int i=1;i<=m;i++){
68         int op,a,b,c;
69         cin>>op>>a>>b;
70         if(op==0){
71             cin>>c;
72             cout<<query(a,b,c)<<endl;
73         }else update(a,b);
74     }
75 }
76 }
```

2.3 带权并查集

```
1 struct DSU {
2     int n;
```

```
3     vector<int> f, siz, pre;
4     DSU(int n): n(n),
5         f(vector<int>(n+1)),
6         siz(vector<int>(n+1,1)),
7         pre(vector<int>(n+1))
8     {
9         iota(f.begin(), f.end(), 0);
10    }
11    int find(int x) {
12        if (f[x] == x) { return x; }
13        int fa = find(f[x]);
14        pre[x] += pre[f[x]];
15        return f[x]=fa;
16    }
17    bool same(int x, int y) {
18        return find(x) == find(y);
19    }
20    void merge(int x, int y) {
21        x = find(x);
22        y = find(y);
23        f[x] = y;
24        pre[x] += siz[y];
25        siz[y] += siz[x];
26        siz[x] = 0;
27    }
28    int query(int x, int y) {
29        return abs(pre[x]-pre[y])-1;
30    }
31};
```

2.4 Trie

```
1 const string VAL="
2     abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789
3     ";
4 const int S=3e6+5, V=62;
5 struct Trie {
6     unordered_map<char,int> p;
7     int tot, ch[S][V], pass[S];
8     int newNode() {
9         ++tot;
10        memset(ch[tot], 0, sizeof(ch[tot]));
11        pass[tot] = 0;
12        return tot;
13    }
14    void init() {
15        tot = -1;
16        newNode();
17    }
18    Trie() {
19        for (int i = 0; i < V; ++i) { p[VAL[i]] = i; }
20        init();
21    }
22    void insert(string s) {
23        int now = 0;
24        for (auto c: s) {
25            if (!ch[now][p[c]]) { ch[now][p[c]] = newNode(); }
26            now = ch[now][p[c]];
27            ++pass[now];
28        }
29    }
30    int query(string s) {
31        int now = 0;
32        for (auto c: s) {
33            if (!ch[now][p[c]]) { return 0; }
34            now = ch[now][p[c]];
35        }
36        return pass[now];
37    }
38}; trie;
```

2.5 ST表

```
1 template <typename T>
2 struct ST {
3     int n=0, l=0;
4     vector<int> Log;
5     vector<vector<T>> st;
6     ST() { }
7     ST(const vector<T>& a): n(a.size()), Log(vector<int>(n+1))
8     {
```

```

8     for (int i = 2; i <= n; ++i) {
9         I=Log[i]=Log[i/2]+1;
10    }
11    st.assign(I+1, vector<T>(n));
12    copy(a.begin(), a.end(), st[0].begin());
13    for (int i = 1; i <= I; ++i) {
14        for (int j = 0; j+(1<<(i-1)) < n; ++j) {
15            st[i][j] = max( st[i-1][j], st[i-1][j+(1<<(i-1))] );
16        }
17    }
18 }
19 T query(int l, int r) {
20     assert(l <= r); // l > r 时的返回值需要特殊定义
21     int s = Log[r-l+1];
22     return max( st[s][l], st[s][r-(1<<s)+1] );
23 }
24 int find(int l, T x) { // 第一个区间 [l,r] 最值大于等于 x
25     的 r
26     if (l >= n) { return -1; }
27     int rl=l-1, rr=n;
28     while (rl != rr-1) {
29         int mid = (rl + rr) >> 1;
30         if (query(l, mid) >= x) { rr = mid; }
31         else { rl = mid; }
32     }
33     if (rr < n) { return rr; }
34     return -1;
35 }
36 int find(int l, int x) { // 倍增写法
37     assert(l < n);
38     if (st[0][l] < x) { return l; }
39     if (query(l, n-1) >= x) { return n; }
40     int ans = l, g = st[0][l];
41     for (int i = I; i >= 0; --i) {
42         if (ans+(1<<i) < n && gcd(g, st[i][ans]) >= x) {
43             g = max(g, st[i][ans]);
44             ans += 1 << i;
45         }
46     }
47     return ans;
48 };

```

2.6 树状数组

```

1 struct BIT {
2     int n;
3     vector<int> t;
4     BIT(int n): n(n), t(n+1) { } // 注意值域树状数组中 n=tot
5     int lowbit(int x) { return x&-x; }
6     void modify(int x, int d) {
7         for (; x <= n; x += lowbit(x)) { t[x] += d; }
8     }
9     // 前缀和
10    int query(int x) {
11        int res = 0;
12        for (; x; x -= lowbit(x)) { res += t[x]; }
13        return res;
14    }
15    // 求最小的前缀和等于x的位置
16    int select(int x) {
17        int ans = 0, sum = 0;
18        for (int i = __lg(n); i >= 0; --i) {
19            if (ans+(1<<i) <= n && sum+t[ans+(1<<i)] < x) {
20                ans += 1<<i;
21                sum += t[ans];
22            }
23        }
24        return ans + 1;
25    }
26 };
27 BIT T(n);

```

2.7 线段树

```

1 #define lson (p << 1)
2 #define rson (p << 1 | 1)
3 #define m ((l+r)>>1)
4 template<typename Info>
5 struct segmentTree {
6     int n;

```

```

7     vector<Info> a;
8
9     void push_up(int p) {
10         a[p] = a[lson] + a[rson];
11     }
12
13     template<typename F>
14     Info dfs(int p, int l, int r, int x, int y, F&& op) {
15         if (y <= l || r <= x) { return Info(); }
16         if (x <= l && r <= y) { op(p,l,r); return a[p]; }
17         Info res;
18         res = res + dfs(lson, l, m, x, y, op);
19         res = res + dfs(rson, m, r, x, y, op);
20         push_up(p);
21         return res;
22     }
23
24     template<typename F>
25     pair<int,Info> findFirst(int p, int l, int r, int x, int y,
26                             F&& pred) {
27         if (y <= l || r <= x) { return {-1,Info()}; }
28         if (x <= l && r <= y && !pred(a[p])) { return {-1,Info()}; }
29         if (l == r-1) { return {l,a[p]}; }
30         pair<int,Info> res = findFirst(lson, l, m, x, y, pred);
31         if (res.first == -1) {
32             res = findFirst(rson, m, r, x, y, pred);
33         }
34         return res;
35     }
36
37     template<typename F>
38     pair<int,Info> findLast(int p, int l, int r, int x, int y, F
39                             && pred) {
40         if (y <= l || r <= x) { return {-1,Info()}; }
41         if (x <= l && r <= y && !pred(a[p])) { return {-1,Info()}; }
42         if (l == r-1) { return {l,a[p]}; }
43         pair<int,Info> res = findLast(lson, m, r, x, y, pred);
44         if (res.first == -1) {
45             res = findLast(rson, l, m, x, y, pred);
46         }
47         return res;
48     }
49
50     segmentTree(int n = 0): n(n), a(4 << __lg(n)) { }
51     segmentTree(vector<Info>&& b): segmentTree(b.size()) {
52         function<void(int,int,int)> build = [&](int p, int l,
53             int r) -> void {
54             if (r - l == 1) { a[p] = Info(b[l]); return; }
55             build(lson, l, m);
56             build(rson, m, r);
57             push_up(p);
58         };
59         build(1, 0, n);
60     }
61
62     void modify(int pos, int x) {
63         dfs(1, 0, n, pos, pos+1, [&](int p, int l, int r){ a[p]
64             .mx = x; });
65     }
66
67     int query(int l, int r) {
68         return dfs(1, 0, n, l, r+1, [&](int,int,int){}).mx;
69     }
70
71     void Debug(int p, int l, int r) {
72         cerr << format("a[{}] = [ {}, {}, {} ]\n", p, l, r, a[
73             p].mx);
74         if (l == r - 1) { return; }
75         Debug(lson, l, m);
76         Debug(rson, m, r);
77     }
78     void Debug() { Debug(1, 0, n); }
79 };
80
81 #undef lson
82 #undef rson
83 #undef m
84 struct Info {
85     int mx;
86     Info(int mx = 0): mx(mx) { }
87     friend Info operator+ (Info u, Info v) {
88         return Info( max(u.mx, v.mx) );
89     }
90 };

```

84

};

2.8 替罪羊树

```

1  /*替罪羊树 是一种依靠重构操作维持平衡的重量平衡树。替罪羊树会
   在插入、删除操作后，检测树是否发生失衡；如果失衡，将有针对性地进行重构以恢复平衡。
2  一般地，替罪羊树不支持区间操作，且无法完全持久化；但它具有实现
   简单、常数较小的优点。
3  */
4  //单次查询，单次调整的代价均摊为  $\log(n)$ ，平衡因子一般为  $\theta$ 
   .7--0.8
5  //做词频压缩的
6  struct ScapegoatTree {
7      static const int MAXN = 200005;
8      static constexpr double ALPHA = 0.7;
9      static const int INF = 1e9 + 10;
10
11     int root, cnt;
12     int key[MAXN], num[MAXN];
13     int l[MAXN], r[MAXN];
14     int sz[MAXN], dif[MAXN];
15     int v[MAXN];
16
17     int vc;
18     int top, fa, side;
19
20     ScapegoatTree() {
21         clear();
22     }
23
24     int init(int x) {
25         ++cnt;
26         key[cnt] = x;
27         num[cnt] = 1;
28         l[cnt] = r[cnt] = 0;
29         sz[cnt] = dif[cnt] = 1;
30         return cnt;
31     }
32
33     void pushup(int p) {
34         sz[p] = sz[l[p]] + sz[r[p]] + num[p];
35         dif[p] = dif[l[p]] + dif[r[p]] + (num[p] > 0);
36     }
37
38     bool balance(int p) {
39         return ALPHA * dif[p] >= max(dif[l[p]], dif[r[p]]);
40     }
41
42     void inorder(int p) {
43         if (!p) return;
44
45         inorder(l[p]);
46
47         if (num[p] > 0)
48             v[++vc] = p;
49
50         inorder(r[p]);
51     }
52
53     int build(int L, int R) {
54         if (L > R) return 0;
55
56         int mid = (L + R) >> 1;
57         int p = v[mid];
58
59         l[p] = build(L, mid - 1);
60         r[p] = build(mid + 1, R);
61
62         pushup(p);
63
64         return p;
65     }
66
67     void rebuild() {
68         if (!top) return;
69
70         vc = 0;
71         inorder(top);
72
73         if (!vc) {
74             if (!fa) root = 0;
75             else if (side == 1) l[fa] = 0;

```

```

76         else r[fa] = 0;
77         return;
78     }
79
80     int p = build(1, vc);
81
82     if (!fa)
83         root = p;
84     else if (side == 1)
85         l[fa] = p;
86     else
87         r[fa] = p;
88 }
89
90 void insert(int p, int f, int s, int x) {
91     if (!p) {
92         if (!f)
93             root = init(x);
94         else if (s == 1)
95             l[f] = init(x);
96         else
97             r[f] = init(x);
98
99         return;
100     }
101
102     if (key[p] == x)
103         ++num[p];
104     else if (key[p] > x)
105         insert(l[p], p, 1, x);
106     else
107         insert(r[p], p, 2, x);
108
109     pushup(p);
110
111     if (!balance(p)) {
112         top = p;
113         fa = f;
114         side = s;
115     }
116 }
117
118 void insert(int x) {
119     top = fa = side = 0;
120     insert(root, 0, 0, x);
121     rebuild();
122 }
123
124 int countLess(int p, int x) {
125     if (!p) return 0;
126
127     if (key[p] >= x)
128         return countLess(l[p], x);
129
130     return sz[l[p]] + num[p] + countLess(r[p], x);
131 }
132
133 // x 的排名：第一个 >= x 的位置
134 int rank(int x) {
135     return countLess(root, x) + 1;
136 }
137
138 int kth(int p, int k) {
139     if (sz[l[p]] >= k)
140         return kth(l[p], k);
141
142     if (sz[l[p]] + num[p] < k)
143         return kth(r[p], k - sz[l[p]] - num[p]);
144
145     return key[p];
146 }
147
148 // 第 k 小
149 int kth(int k) {
150     if (k <= 0 || k > sz[root])
151         return INF;
152
153     return kth(root, k);
154 }
155
156 bool exist(int p, int x) {
157     if (!p) return false;
158
159     if (key[p] == x)

```

```

160         return num[p] > 0;
161
162         if (key[p] > x)
163             return exist(l[p], x);
164
165         return exist(r[p], x);
166     }
167
168     void erase(int p, int f, int s, int x) {
169         if (!p) return;
170
171         if (key[p] == x)
172             --num[p];
173         else if (key[p] > x)
174             erase(l[p], p, 1, x);
175         else
176             erase(r[p], p, 2, x);
177
178         pushup(p);
179
180         if (!balance(p)) {
181             top = p;
182             fa = f;
183             side = s;
184         }
185     }
186
187     void erase(int x) {
188         if (!exist(root, x)) return;
189
190         top = fa = side = 0;
191         erase(root, 0, 0, x);
192         rebuild();
193     }
194
195     // < x 的最大值
196     int pre(int x) {
197         int k = rank(x);
198
199         if (k == 1)
200             return -INF;
201
202         return kth(k - 1);
203     }
204
205     // > x 的最小值
206     int nxt(int x) {
207         int k = rank(x + 1);
208
209         if (k == sz[root] + 1)
210             return INF;
211
212         return kth(k);
213     }
214
215     // 当前元素总数
216     int size() {
217         return sz[root];
218     }
219
220     // 是否为空
221     bool empty() {
222         return sz[root] == 0;
223     }
224
225     // 清空
226     void clear() {
227         root = 0;
228         cnt = 0;
229         vc = 0;
230         top = fa = side = 0;
231     }
232 };
233
234 // 不做词频压缩版本
235 struct ScapegoatTree {
236     static const int MAXN = 200005;
237     static constexpr double ALPHA = 0.7;
238     int root, cnt;
239     int key[MAXN], l[MAXN], r[MAXN], sz[MAXN];
240     int v[MAXN];
241     int vc;
242     int top, fa, side;
243     ScapegoatTree() {

```

```

244         clear();
245     }
246     int init(int x) {
247         ++cnt;
248         key[cnt] = x;
249         l[cnt] = r[cnt] = 0;
250         sz[cnt] = 1;
251         return cnt;
252     }
253     void pushup(int p) {
254         sz[p] = sz[l[p]] + sz[r[p]] + 1;
255     }
256     bool balance(int p) {
257         return max(sz[l[p]], sz[r[p]]) <= ALPHA * sz[p];
258     }
259     void inorder(int p) {
260         if (!p) return;
261         inorder(l[p]);
262         v[++vc] = p;
263         inorder(r[p]);
264     }
265     int build(int L, int R) {
266         if (L > R) return 0;
267         int mid = (L + R) >> 1;
268         int p = v[mid];
269         l[p] = build(L, mid - 1);
270         r[p] = build(mid + 1, R);
271         pushup(p);
272         return p;
273     }
274     void rebuild() {
275         if (!top) return;
276         vc = 0;
277         inorder(top);
278         int p = build(1, vc);
279         if (!fa)
280             root = p;
281         else if (side == 1)
282             l[fa] = p;
283         else
284             r[fa] = p;
285     }
286     void insert(int p, int f, int s, int x) {
287         if (!p) {
288             if (!f)
289                 root = init(x);
290             else if (s == 1)
291                 l[f] = init(x);
292             else
293                 r[f] = init(x);
294
295             return;
296         }
297         if (x <= key[p])
298             insert(l[p], p, 1, x);
299         else
300             insert(r[p], p, 2, x);
301         pushup(p);
302         if (!balance(p)) {
303             top = p;
304             fa = f;
305             side = s;
306         }
307     }
308     void insert(int x) {
309         top = fa = side = 0;
310         insert(root, 0, 0, x);
311         rebuild();
312     }
313     // < x 的元素个数
314     int countLess(int p, int x) {
315         if (!p) return 0;
316         if (key[p] >= x)
317             return countLess(l[p], x);
318         return sz[l[p]] + 1 + countLess(r[p], x);
319     }
320     // x 的排名: 第一个 >= x 的位置
321     int rank(int x) {
322         return countLess(root, x) + 1;
323     }
324     int kth(int p, int k) {
325         if (sz[l[p]] >= k)
326             return kth(l[p], k);
327

```

```

328     if (sz[l[p]] + 1 < k)
329         return kth(r[p], k - sz[l[p]] - 1);
330
331     return key[p];
332 }
333 // 第 k 小
334 int kth(int k) {
335     return kth(root, k);
336 }
337 bool exist(int p, int x) {
338     if (!p) return false;
339     if (key[p] == x)
340         return true;
341     if (x < key[p])
342         return exist(l[p], x);
343     return exist(r[p], x);
344 }
345 void erase(int p, int f, int s, int x) {
346     if (key[p] == x) {
347         // 找到以后直接把整个节点删除
348         // 用左右子树合并的方法删除
349         if (!l[p] || !r[p]) {
350             int son = l[p] ? l[p] : r[p];
351
352             if (!f)
353                 root = son;
354             else if (s == 1)
355                 l[f] = son;
356             else
357                 r[f] = son;
358
359             return;
360         }
361         // 两个儿子都存在:
362         // 找右子树最小节点
363         int q = r[p];
364         while (l[q])
365             q = l[q];
366
367         key[p] = key[q];
368
369         erase(r[p], p, 2, key[q]);
370     }
371     else if (x < key[p]) {
372         erase(l[p], p, 1, x);
373     }
374     else {
375         erase(r[p], p, 2, x);
376     }
377     pushup(p);
378     if (!balance(p)) {
379         top = p;
380         fa = f;
381         side = s;
382     }
383 }
384 void erase(int x) {
385     if (!exist(root, x)) return;
386
387     top = fa = side = 0;
388     erase(root, 0, 0, x);
389     rebuild();
390 }
391 // < x 的最大值
392 int pre(int x) {
393     int k = rank(x);
394
395     if (k == 1)
396         return -INT_MAX;
397
398     return kth(k - 1);
399 }
400 // > x 的最小值
401 int nxt(int x) {
402     int k = rank(x + 1);
403
404     if (k == sz[root] + 1)
405         return INT_MAX;
406
407     return kth(k);
408 }
409 int size() {
410     return sz[root];
411 }

```

```

412 bool empty() {
413     return sz[root] == 0;
414 }
415 void clear() {
416     root = 0;
417     cnt = 0;
418     vc = 0;
419     top = fa = side = 0;
420 }
421 };

```

2.9 逆序对

```

1 ll msort(vector<int> &a, int l, int r)
2 {
3     if (l == r) { return 0; }
4     int mid = (l + r) >> 1;
5     ll ans=0;
6     ans += msort(a, l, mid);
7     ans += msort(a, mid+1, r);
8     int i=l, j=mid+1;
9     vector<int> b;
10    while (i <= mid && j <= r) {
11        if (a[i] <= a[j]) { b.push_back(a[i++]); }
12        else { b.push_back(a[j++]); ans+=mid-i+1; }
13    }
14    while (i <= mid) { b.push_back(a[i++]); }
15    while (j <= r) { b.push_back(a[j++]); }
16    copy(b.begin(), b.end(), a.begin()+1);
17    return ans;
18 }

```

2.10 笛卡尔树

```

1 vector<int> stk;
2 for (int i = 1; i <= n; ++i) {
3     int flag = 0;
4     while (!stk.empty()) {
5         if (a[stk.back()] > a[i]) { // 小根堆
6             flag = stk.back(); stk.pop_back();
7         } else {
8             break;
9         }
10    }
11    if (flag) { L[i] = flag; }
12    if (!stk.empty()) { R[stk.back()] = i; }
13    stk.push_back(i);
14 }
15 function<void(int)> dfs = [&](int x){
16     cerr << "now node " << x << "\n";
17     cerr << ":: L = " << L[x] << "\n";
18     cerr << ":: R = " << R[x] << "\n";
19     if (L[x]) dfs(L[x]);
20     if (R[x]) dfs(R[x]);
21 };
22 dfs(stk[0]);

```

2.11 可持久化

```

1 5 10
2 59 46 14 87 41
3 0 2 1
4 0 1 1 14
5 0 1 1 57
6 0 1 1 88
7 4 2 4
8 0 2 5
9 0 2 4
10 4 2 1
11 2 2 2
12 1 1 5 91
13 ---
14 59
15 87
16 41
17 87
18 88
19 46

```

2.12 线段树模板

```
1 //区间乘法
2 #include<bits/stdc++.h>
3 using namespace std;
4 #define maxn 1000010
5 #define ll long long
6 int p;
7 ll a[maxn], w[maxn*4];
8 ll lzy_add[maxn*4], lzy_mul[maxn*4];
9
10 void pushup(const int u) {
11     w[u] = (w[u<<1] + w[u<<1|1]) % p;
12 }
13
14 void build(const int u, int L, int R) {
15     lzy_mul[u] = 1;
16     lzy_add[u] = 0;
17     if (L == R) {
18         w[u] = a[L] % p;
19         return;
20     }
21     int M = (L+R)>>1;
22     build(u<<1, L, M);
23     build(u<<1|1, M+1, R);
24     pushup(u);
25 }
26
27 bool InRange(int L, int R, int l, int r) {
28     return (l <= L) && (R <= r);
29 }
30
31 bool OutofRange(int L, int R, int l, int r) {
32     return (R < l) || (L > r);
33 }
34
35 void maketag(int u, int L, int R, ll x, int type) {
36     if (type == 1) {
37         (lzy_mul[u] *= x) %= p;
38         (lzy_add[u] *= x) %= p;
39         (w[u] *= x) %= p;
40     } else {
41         (lzy_add[u] += x) %= p;
42         (w[u] += (R-L+1) * x) %= p;
43     }
44 }
45
46 void pushdown(int u, int L, int R) {
47     int M = (L+R)>>1;
48     // 左子节点
49     maketag(u<<1, L, M, lzy_mul[u], 1);
50     maketag(u<<1, L, M, lzy_add[u], 2);
51     // 右子节点
52     maketag(u<<1|1, M+1, R, lzy_mul[u], 1);
53     maketag(u<<1|1, M+1, R, lzy_add[u], 2);
54     lzy_mul[u] = 1;
55     lzy_add[u] = 0;
56 }
57
58 ll query(int u, int L, int R, int l, int r) {
59     if (InRange(L, R, l, r)) {
60         return w[u];
61     } else if (!OutofRange(L, R, l, r)) {
62         int M = (L+R)>>1;
63         pushdown(u, L, R);
64         return (query(u<<1, L, M, l, r) + query(u<<1|1, M+1, R, l, r)) % p;
65     } else {
66         return 0;
67     }
68 }
69
70 void update(int u, int L, int R, int l, int r, ll x, int type) {
71     if (InRange(L, R, l, r)) {
72         maketag(u, L, R, x, type);
73     } else if (!OutofRange(L, R, l, r)) {
74         int M = (L+R)>>1;
75         pushdown(u, L, R);
76         update(u<<1, L, M, l, r, x, type);
77         update(u<<1|1, M+1, R, l, r, x, type);
78         pushup(u);
79     }
80 }
```

```
81
82 int main() {
83     ios::sync_with_stdio(false);
84     cin.tie(nullptr);
85     ll n, m;
86     cin >> n >> m >> p;
87     for (int i = 1; i <= n; i++)
88         cin >> a[i];
89     build(1, 1, n);
90     while (m--) {
91         int op, x, y;
92         cin >> op >> x >> y;
93         if (op == 1 || op == 2) {
94             ll k;
95             cin >> k;
96             update(1, 1, n, x, y, k, op);
97         } else {
98             cout << query(1, 1, n, x, y) << endl;
99         }
100     }
101     return 0;
102 }
```

2.13 主席树 (可持久化线段树)

```
1 //对于单点历史修改与查询
2 #include<bits/stdc++.h>
3 using namespace std;
4 #define int long long
5 const int maxn=(1e6+1);
6 int top,root[maxn],arr[maxn],n,m,cnt=0;
7 struct kkk{
8     int l,r,val;
9 } a[maxn*30];
10 int build(int l,int r){
11     int rt=++cnt;
12     if(l==r)a[rt].val=arr[l];
13     else {
14         int mid=(l+r)>>1;
15         a[rt].l=build(l,mid);
16         a[rt].r=build(mid+1,r);
17     }
18     return rt;
19 }
20 int update(int ji,int jv,int l,int r,int i)//单点修改
21 {
22     int rt=++cnt;
23     a[rt].l=a[ji].l;
24     a[rt].r=a[ji].r;
25     if(l==r) a[rt].val=jv;
26     else {
27         int mid=(l+r)>>1;
28         if(ji<=mid)a[rt].l=update(ji,jv,l,mid,a[rt].l);
29         else a[rt].r=update(ji,jv,mid+1,r,a[rt].r);
30     }
31     return rt;
32 }
33 int query(int ji,int l,int r,int i){
34     if(l==r) return a[ji].val;
35     int mid=(l+r)>>1;
36     if(ji<=mid) return query(ji,l,mid,a[ji].l);
37     else return query(ji,mid+1,r,a[ji].r);
38 }
39 signed main(){
40     ios_base::sync_with_stdio(false);
41     cin.tie(nullptr);
42     cin>>n>>m;
43     for(int i=1;i<=n;i++){
44         cin>>arr[i];
45     }
46     root[0]=build(1,n);
47     for(int i=1,version,op,x,v;i<=m;i++){
48         cin>>version>>op>>x;
49         if(op==1){
50             cin>>v;
51             root[i]=update(x,v,1,n,root[version]);
52         } else {
53             root[i]=root[version];
54             cout<<query(x,1,n,root[i])<<endl;
55         }
56     }
57 }
```

2.14 开点线段树

```
1 //适用于范围很大,但是操作较少
2 //用于减小空间复杂度
3 //如果操作过多,则与没有优化相同,因此可能被卡
4 //在线段树的合并与分裂中不可替代,树链剖分
5 #include<bits/stdc++.h>
6 using namespace std;
7 typedef long long ll;
8 const int maxn=100001;
9 ll le[maxn],ri[maxn],sum[maxn],add[maxn],cnt;
10 void up(ll i, ll l, ll r){
11     sum[i]=sum[l]+sum[r];
12 }
13 void lazy(ll i,ll v, ll n){
14     sum[i]+=v*n;
15     add[i]+=v;
16 }
17 void down(ll i, ll ln, ll rn){
18     if(add[i]!=0){
19         if(le[i]==0){
20             le[i]++;cnt;
21         }
22         if(ri[i]==0){
23             ri[i]++;cnt;
24         }
25         lazy(le[i],add[i],ln);
26         lazy(ri[i],add[i],rn);
27         add[i]=0;
28     }
29 }
30 void Add(ll j1,ll jr,ll v,ll l,ll r,ll i){
31     if(j1<=l&&r<=jr){
32         lazy(i,v,r-l+1);
33         return;
34     }
35     ll mid=(l+r)>>1;
36     down(i,mid-l+1,r-mid);
37     if(j1<=mid){
38         if(le[i]==0){
39             le[i]++;cnt;
40         }
41         Add(j1,jr,v,l,mid,le[i]);
42     }
43     if(jr>mid){
44         if(ri[i]==0){
45             ri[i]++;cnt;
46         }
47         Add(j1,jr,v,mid+1,r,ri[i]);
48     }
49 }
50 up(i,le[i],ri[i]);
51 }
52 ll query(ll j1,ll jr,ll l, ll r,ll i){
53     if(j1<=l&&r<=jr){
54         return sum[i];
55     }
56     ll mid=(l+r)>>1;
57     down(i,mid-l+1,r-mid);
58     ll ans=0;
59     if(j1<=mid){
60         if(le[i]!=0){
61             ans+=query(j1,jr,l,mid,le[i]);
62         }
63     }
64     if(jr>mid){
65         if(ri[i]!=0){
66             ans+=query(j1,jr,mid+1,r,ri[i]);
67         }
68     }
69     return ans;
70 }
71 int main(){
72     int n,m;
73     cin>>n>>m;
74     cnt=1;
75     while(m--){
76         int opt;
77         cin>>opt;
78         if(opt==1){
79             int l,r,v;
80             cin>>l>>r>>v;
81             Add(l,r,v,1,n,1);
82 }
```

```
83     }
84     else {
85         int l,r;
86         cin>>l>>r;
87         cout<<query(l,r,1,n,1)<<endl;
88     }
89 }
90 }
```

2.15 权值线段树

```
1 //权值线段树模板
2 #define lson pos<<1
3 #define rson pos<<1|1
4 void build(int pos,int l,int r)
5 {
6     int mid=(l+r)>>1;
7     if(l==r)
8     {
9         tree[pos]=a[l];//a[l]表示数l有多少个
10        return;
11    }
12    build(lson,l,mid);
13    build(rson,mid+1,r);
14    tree[pos]=tree[lson]+tree[rson];
15 }
16 void update(int pos,int l,int r,int k,int cnt)//表示数k的个数
17     多cnt个
18 {
19     int mid=(l+r)>>1;
20     if(l==r)
21     {
22         tree[pos]+=cnt;
23         return;
24     }
25     if(k<=mid)
26         update(lson,l,mid,k,cnt);
27     else
28         update(rson,mid+1,r,k,cnt);
29     tree[pos]=tree[lson]+tree[rson];
30 }
31 int query(int pos,int l,int r,int k)//查询数k有多少个
32 {
33     int mid=(l+r)>>1;
34     if(l==r)
35         return tree[pos];
36     if(k<=mid)
37         return query(lson,l,mid,k);
38     else
39         return query(rson,mid+1,r,k);
40 }
41 int kth(int pos,int l,int r,int k)//查询第k大值是多少
42 {
43     int mid=(l+r)>>1;
44     if(l==r)
45         return l;
46     if(k<=mid)
47         return kth(lson,l,mid,k);
48     else
49         return kth(rson,mid+1,r,k-mid);
50 }
```

2.16 线段树上最值操作

```
1 //基本的实现
2 #include<bits/stdc++.h>
3 using namespace std;
4 const int maxn=1000001;
5 const int minest=-1000001;
6 long long arr[maxn<<2],sum[maxn<<2],mx[maxn<<2],sem[maxn<<2],
7     cnt[maxn<<2];
8 void up(int i){
9     int l=i<<1;
10    int r=i<<1|1;
11    sum[i]=sum[l]+sum[r];
12    mx[i]=max(mx[l],mx[r]);
13    if(mx[l]>mx[r]){
14        cnt[i]=cnt[l];
15        sem[i]=max(sem[l],mx[r]);
16    }else if(mx[r]>mx[l]){
17        cnt[i]=cnt[r];
18        sem[i]=max(sem[r],mx[l]);
19    }
```

```

18     }else {
19         sem[i]=max(sem[l],sem[r]);
20         cnt[i]=cnt[l]+cnt[r];
21     }
22 }
23 void Lazy(int i,int v){
24     if(v<mx[i]){
25         sum[i]-=(mx[i]-v)*cnt[i];
26         mx[i]=v;
27     }
28 }
29 void down(int i){
30     Lazy(i<<1,mx[i]);
31     Lazy(i<<1|1,mx[i]);
32 }
33 void setmin(int jl,int jr,int v,int l,int r,int i){
34     if(v>mx[i]) return;
35     if(jl<=l&&r<=jr&&v>sem[i]){
36         Lazy(i,v);
37         return;
38     }else {
39         down(i);
40         int mid=(l+r)>>1;
41         if(jl<=mid) setmin(jl,jr,v,l,mid,i<<1);
42         if(jr>mid) setmin(jl,jr,v,mid+1,r,i<<1|1);
43     }
44     up(i);
45 }
46 long long querymax(int jl,int jr,int l,int r,int i){
47     if(jl<=l&&r<=jr){
48         return mx[i];
49     }
50     down(i);
51     int mid=(l+r)>>1;
52     long long lm=minest,rm=minest;
53     if(jl<=mid) lm=querymax(jl,jr,l,mid,i<<1);
54     if(jr>mid) rm=querymax(jl,jr,mid+1,r,i<<1|1);
55     return max(lm,rm);
56 }
57 long long querysum(int jl,int jr,int l,int r,int i){
58     if(jl<=l&&r<=jr){
59         return sum[i];
60     }
61     down(i);
62     int mid=(l+r)>>1;
63     long long ans=0;
64     if(jl<=mid) ans+=querysum(jl,jr,l,mid,i<<1);
65     if(jr>mid) ans+=querysum(jl,jr,mid+1,r,i<<1|1);
66     return ans;
67 }
68 void build(int l,int r,int i){
69     if(l==r){
70         sum[i]=arr[l];
71         mx[i]=arr[l];
72         sem[i]=minest;
73         cnt[i]=1;
74         return;
75     }
76     int mid=(l+r)>>1;
77     build(l,mid,i<<1),build(mid+1,r,i<<1|1);
78     up(i);
79 }
80 int main(){
81     ios_base::sync_with_stdio(false);
82     cin.tie(nullptr);
83     int t;
84     cin>>t;
85     while(t--){
86         int n,m;
87         cin>>n>>m;
88         for(int i=1;i<=n;i++){
89             cin>>arr[i];
90         }
91         build(1,n,1);
92         while(m--){
93             int opt;
94             cin>>opt;
95             if(opt==0){
96                 int l,r;
97                 long long mi;
98                 cin>>l>>r>>mi;
99                 setmin(l,r,mi,1,n,1);
100             }else if(opt==1){
101                 int l,r;

```

```

102         cin>>l>>r;
103         cout<<querymax(l,r,1,n,1)<<endl;
104     }else {
105         int l,r;
106         cin>>l>>r;
107         cout<<querysum(l,r,1,n,1)<<endl;
108     }
109 }
110 }
111 }

```

2.17 线段树上序列操作

```

1 //线段树上的序列操作，稍难一点
2 //求连续1的最长子列，有点难度，容易打错
3
4 #include<bits/stdc++.h>
5 using namespace std;
6 const int maxn=100001;
7 int arr[maxn],sum[maxn<<2],pre0[maxn<<2],len0[maxn<<2],suf0[
8     maxn<<2],pre1[maxn<<2],len1[maxn<<2],suf1[maxn<<2];
9 int lazy[maxn<<2];
10 bool update[maxn<<2];
11 bool rever[maxn<<2];
12 void up(int i,int ln,int rn){
13     int l=i<<1;
14     int r=i<<1|1;
15     sum[i]=sum[l]+sum[r];
16     len0[i]=max(max(len0[l],len0[r]),suf0[l]+pre0[r]);
17     pre0[i]=len0[l]<ln?suf0[l]:pre0[l]+pre0[r];
18     suf0[i]=len0[r]<rn?suf0[r]:suf0[l]+suf0[r];
19     len1[i]=max(max(len1[l],len1[r]),suf1[l]+pre1[r]);
20     pre1[i]=len1[l]<ln?pre1[l]:pre1[l]+pre1[r];
21     suf1[i]=len1[r]<rn?suf1[r]:suf1[l]+suf1[r];
22 }
23 void updateLazy(int i,int v,int n){
24     sum[i]=v*n;
25     pre0[i]=suf0[i]=len0[i]=v==0?n:0;
26     pre1[i]=suf1[i]=len1[i]=v==1?n:0;
27     lazy[i]=v;
28     update[i]=true;
29     rever[i]=false;
30 }
31 void reverseLazy(int i,int n){
32     sum[i]=n-sum[i];
33     swap(pre0[i],pre1[i]);
34     swap(len0[i],len1[i]);
35     swap(suf0[i],suf1[i]);
36     rever[i]=!rever[i];
37 }
38 void down(int i,int ln,int rn){
39     if(update[i]){
40         updateLazy(i<<1,lazy[i],ln);
41         updateLazy(i<<1|1,lazy[i],rn);
42         update[i]=false;
43     }
44     if(rever[i]){
45         reverseLazy(i<<1,ln);
46         reverseLazy(i<<1|1,rn);
47         rever[i]=false;
48     }
49 }
50 void build(int l,int r,int i){
51     if(l==r){
52         sum[i]=arr[l];
53         pre0[i]=arr[l]?0:1;
54         suf0[i]=arr[l]?0:1;
55         len0[i]=arr[l]?0:1;
56         pre1[i]=arr[l]?1:0;
57         suf1[i]=arr[l]?1:0;
58         len1[i]=arr[l]?1:0;
59         return;
60     }
61     int mid=(l+r)>>1;
62     build(l,mid,i<<1),build(mid+1,r,i<<1|1);
63     up(i,mid-l+1,r-mid);
64     rever[i]=0;
65     update[i]=0;
66 }
67 void Update(int jl,int jr,int v,int l,int r,int i){
68     if(jl<=l&&r<=jr){
69         updateLazy(i,v,r-l+1);
70     }else {

```

```

70     int mid=(l+r)>>1;
71     down(i,mid-l+1,r-mid);
72     if(jl<=mid) Update(jl,jr,v,l,mid,i<<1);
73     if(jr>mid) Update(jl,jr,v,mid+1,r,i<<1|1);
74     up(i,mid-l+1,r-mid);
75 }
76 }
77 void Reverse(int jl,int jr,int l,int r,int i){
78     if(jl<=l&&r<=jr){
79         reverseLazy(i,r-l+1);
80     }else {
81         int mid=(l+r)>>1;
82         down(i,mid-l+1,r-mid);
83         if(jl<=mid) Reverse(jl,jr,l,mid,i<<1);
84         if(jr>mid) Reverse(jl,jr,mid+1,r,i<<1|1);
85         up(i,mid-l+1,r-mid);
86     }
87 }
88 tuple<int,int,int> querylength(int jl,int jr,int l,int r,int i
89 ){
90     if(jl<=l&&r<=jr){
91         return make_tuple(len1[i],pre1[i],suf1[i]);
92     }
93     int mid=(l+r)>>1;
94     down(i,mid-l+1,r-mid);
95     if(jr<=mid) return querylength(jl,jr,l,mid,i<<1);
96     if(jl>mid) return querylength(jl,jr,mid+1,r,i<<1|1);
97     tuple<int,int,int> r1=querylength(jl,jr,l,mid,i<<1);
98     tuple<int,int,int> r2=querylength(jl,jr,mid+1,r,i<<1|1);
99     int llen=get<0>(r1),lpre=get<1>(r1),lsuf=get<2>(r1);
100    int rlen=get<0>(r2),rpre=get<1>(r2),rsuf=get<2>(r2);
101    int len=max(max(llen,rlen),lsuf+rpre);
102    int pre=lpre<mid-max(l,jl)+1?lpre:rpre;
103    int suf=rsuf<min(r,jr)-mid?rsuf:rsuf+lsuf;
104    return make_tuple(len,pre,suf);
105 }
106 int query(int jl,int jr,int l,int r,int i){
107     if(jl<=l&&r<=jr){
108         return sum[i];
109     }
110     int mid=(l+r)>>1;
111     down(i,mid-l+1,r-mid);
112     int ans=0;
113     if(jl<=mid) ans+=query(jl,jr,l,mid,i<<1);
114     if(jr>mid) ans+=query(jl,jr,mid+1,r,i<<1|1);
115     return ans;
116 }
117 int main(){
118     int n,m;
119     cin>>n>>m;
120     for(int i=1;i<=n;i++) cin>>arr[i];
121     build(1,n,1);
122     while(m--){
123         int opt,l,r;
124         cin>>opt>>l>>r;
125         l+=1;
126         r+=1;
127         if(opt==0){
128             Update(l,r,0,1,n,1);
129         }else if(opt==1){
130             Update(l,r,1,1,n,1);
131         }else if(opt==2){
132             Reverse(l,r,1,n,1);
133         }else if(opt==3){
134             cout<<query(l,r,1,n,1)<<endl;
135         }else {
136             tuple<int,int,int>a=querylength(l,r,1,n,1);
137             cout<<get<0>(a)<<endl;
138         }
139     }

```

2.18 非经典线段树

```

1 // 这类题目一般是可以看出使用线段树，但是不能满足一般的懒更
2 // 新的条件
3 // 简单的一般需要进行势能分析，再减枝就可以解决
4 // 例如 洛谷 P4145，此题是开根号，换为取模同理
5 // P1471 求方差
6 #include<bits/stdc++.h>
7 using namespace std;
8 const int maxn=100001;

```

```

9 double a[maxn],sum[maxn<<2],pf[maxn<<2],lazy[maxn<<2];
10 void up(int i){
11     sum[i]=sum[i<<1]+sum[i<<1|1];
12     pf[i]=pf[i<<1]+pf[i<<1|1];
13 }
14
15 void Lazy(int i,double v,int n){
16     lazy[i]+=v;
17     pf[i]+=2*v*sum[i]+v*v*n;
18     sum[i]+=v*n;
19 }
20 void down(int i,int ln,int rn){
21     if(lazy[i]!=0){
22         Lazy(i<<1,lazy[i],ln);
23         Lazy(i<<1|1,lazy[i],rn);
24         lazy[i]=0;
25     }
26 }
27 void add(int jl,int jr,double v,int l,int r,int i){
28     if(jl<=l&&jr>=r){
29         Lazy(i,v,r-l+1);
30         return;
31     }
32     int mid=(l+r)>>1;
33     down(i,mid-l+1,r-mid);
34     if(jl<=mid){
35         add(jl,jr,v,l,mid,i<<1);
36     }
37     if(jr>mid) add(jl,jr,v,mid+1,r,i<<1|1);
38     up(i);
39 }
40 double query(int jl,int jr,int jt,int l,int r,int i){
41     if(jl<=l&&jr>=r){
42         if(jt){
43             return pf[i];
44         }else return sum[i];
45     }
46     int mid=(l+r)>>1;
47     down(i,mid-l+1,r-mid);
48     double ans=0;
49     if(jl<=mid) ans+=query(jl,jr,jt,l,mid,i<<1);
50     if(jr>mid) ans+=query(jl,jr,jt,mid+1,r,i<<1|1);
51     return ans;
52 }
53 void build(int l,int r,int i){
54     if(l==r){
55         sum[i]=a[l];
56         pf[i]=a[l]*a[l];
57     }
58     else {
59         int mid=(l+r)>>1;
60         build(l,mid,i<<1);
61         build(mid+1,r,i<<1|1);
62         up(i);
63     }
64     lazy[i]=0;
65 }
66
67 int main(){
68     ios_base::sync_with_stdio(false);
69     cin.tie(nullptr);
70     int n,m;
71     cin>>n>>m;
72     for(int i=1;i<=n;i++){
73         cin>>a[i];
74     }
75     build(1,n,1);
76     while(m--){
77         int opt;
78         cin>>opt;
79         if(opt==1){
80             int l,r;
81             double v;
82             cin>>l>>r>>v;
83             add(l,r,v,1,n,1);
84         }else if(opt==2){
85             int l,r;
86             cin>>l>>r;
87             printf("%.4lf\n",query(l,r,0,1,n,1)/(r-l+1));
88         }else {
89             int l,r;
90             cin>>l>>r;
91             printf("%.4lf\n",query(l,r,1,1,n,1)/(r-l+1)-query(
92                 l,r,0,1,n,1)/(r-l+1)*query(l,r,0,1,n,1)/(r-l

```

```

92         +1));
93     }
94 }

```

2.19 Kruskal 重构树

```

1 //Kruskal 重构树：最小生成树上任意两点路径最大值=新节点 LCA 的权
  值
2 const int maxn=1e5+10,maxm=2*maxn,maxk=3*maxn,maxh=20;
3 struct edge{ int u,v,w; }e[maxk];
4 int n,m,fa[maxk],head[maxk],tot=1;
5 struct graph{ int t,n; }g[maxk];
6 void add(int u,int v){ g[++tot].n=head[u]; g[tot].t=v; head[u]
  =tot; }
7 int nodekey[maxk],cntu,dep[maxk],st[maxk][maxh];
8 int find(int x){ return x==fa[x]?x:fa[x]=find(fa[x]); }
9 bool cmp(edge x,edge y){ return x.w<y.w; }
10 void kruskalbuild(){
11     for(int i=1;i<=n;i++) fa[i]=i;
12     sort(e+1,e+m+1,cmp); cntu=n;
13     for(int i=1,fx,fy;i<=m;i++){
14         fx=find(e[i].u); fy=find(e[i].v);
15         if(fx!=fy){
16             fa[fx]=fa[fy]=++cntu; fa[cntu]=cntu;
17             nodekey[cntu]=e[i].w;
18             add(cntu,fx); add(cntu,fy);
19         }
20     }
21 }
22 void dfs(int u,int f){
23     st[u][0]=f; dep[u]=dep[f]+1;
24     for(int p=1;p<maxh;p++) st[u][p]=st[st[u][p-1]][p-1];
25     for(int e=head[u];e;e=g[e].n) dfs(g[e].t,u);
26 }
27 int lca(int u,int v){
28     if(dep[u]<dep[v]) swap(u,v);
29     for(int p=maxh-1;p>=0;p--) if(dep[st[u][p]]>=dep[v]) u=st
      [u][p];
30     if(u==v) return u;
31     for(int p=maxh-1;p>=0;p--) if(st[u][p]!=st[v][p]){ u=st[u
      ][p]; v=st[v][p]; }
32     return st[u][0];
33 }

```

2.20 左偏树

```

1 //可并堆结构，两个堆的合并时间复杂度为 Log (n)
2 //查询操作需要并查集的优化
3 //P3377, 模板，删除一个堆中最小值
4 #include<bits/stdc++.h>
5 using namespace std;
6 const int maxn=100001;
7 int n,m;
8 int num[maxn],lef[maxn],righ[maxn],dis[maxn];
9 int father[maxn];
10 //father 并不是代表该节点的父亲节点，而是代表并查集所对应的路径
   信息
11 //需要保证，并查集最上方代表节点=左偏树的头，也就是堆顶
12 void prepare(){
13     dis[0]=-1;
14     for(int i=1;i<=n;i++){
15         lef[i]=0;
16         righ[i]=0;
17         dis[i]=0;
18         father[i]=i;
19     }
20 }
21 int find(int i){
22     father[i]=father[i]==i?i:find(father[i]);
23     return father[i];
24 }
25 int merge(int i,int j){
26     if(i==0||j==0) return i+j;
27     //维护小根堆，如果值一样，编号小的节点做头，大根堆同理
28     if(num[i]>num[j]||(num[i]==num[j]&&i>j)){
29         swap(i,j);
30     }
31     righ[i]=merge(righ[i],j);
32     if(dis[lef[i]]<dis[righ[i]]){
33         swap(lef[i],righ[i]);
34     }

```

```

35     dis[i]=dis[righ[i]]+1;
36     father[lef[i]]=father[righ[i]]=i;
37     return i;
38 }
39 //节点 i 一定是左偏树的头，在左偏树上删除节点 i，返回新树的头结点
   编号
40 int pop(int i){
41     father[lef[i]]=lef[i];
42     father[righ[i]]=righ[i];
43     father[i]=merge(lef[i],righ[i]); //保证删除节点后，能够正确
      再次查询
44     lef[i]=righ[i]=dis[i]=0;
45     return father[i];
46 }
47 int main(){
48     cin>>n>>m;
49     prepare();
50     for(int i=1;i<=n;i++) cin>>num[i];
51     for(int i=1;i<=m;i++){
52         int op;
53         cin>>op;
54         if(op==1){
55             int l,r;
56             cin>>l>>r;
57             if(num[l]==-1&&num[r]==-1){
58                 l=find(l),r=find(r);
59                 if(l!=r) merge(l,r);
60             }
61         }else {
62             int x;
63             cin>>x;
64             if(num[x]==-1) cout<<-1<<endl;
65             else {
66                 int ans=find(x);
67                 cout<<num[ans]<<endl;
68                 pop(ans);
69                 num[ans]=-1;
70             }
71         }
72     }
73 }
74 //进一步，删除任意一个节点
75 //P4971
76 #include<bits/stdc++.h>
77 using namespace std;
78 #define int long long
79 const int maxn=2000100;
80 int n,m;
81 int num[maxn],father[maxn],lef[maxn],righ[maxn],dis[maxn],up[
  maxn];
82 int t,w,k;
83 // up 数组存储的就是节点真正的父节点
84 void prepare(){
85     dis[0]=-1;
86     for(int i=1;i<=n;i++){
87         dis[i]=lef[i]=righ[i]=up[i]=0;
88         father[i]=i;
89     }
90 }
91 int find(int i){
92     father[i]=father[i]==i?i:find(father[i]);
93     return father[i];
94 }
95 int merge(int i,int j){
96     if(i==0||j==0){
97         return i+j;
98     }
99     //这段代码是维护的大根堆
100     if(num[i]<num[j]||(num[i]==num[j]&&i>j)){
101         swap(i,j);
102     }
103     righ[i]=merge(righ[i],j);
104     up[righ[i]]=i; //多了一句
105     if(dis[lef[i]]<dis[righ[i]]){
106         swap(lef[i],righ[i]);
107     }
108     dis[i]=dis[righ[i]]+1;
109     father[lef[i]]=father[righ[i]]=i;
110     return i;
111 }
112 //删除任意节点，返回整棵树的头节点
113 //这是主要的改动
114 int remove(int i){

```

```

116 int h=find(i);
117 father[lef[i]]=lef[i];
118 father[righ[i]]=righ[i];
119 int s=merge(lef[i],righ[i]);
120 int f=up[i];
121 father[i]=s;
122 up[s]=f;
123 if(h!=i){
124     father[s]=h;
125     if(lef[f]==i) lef[f]=s;
126     else righ[f]=s;
127     for(int d=dis[s];dis[f]>d+1;f=up[f],d++){// Log(n)
128         if(dis[lef[f]]<dis[righ[f]]){
129             swap(lef[f],righ[f]);
130         }
131         dis[f]=d+1;
132     }
133 }
134 up[i]=lef[i]=righ[i]=dis[i]=0;
135 return father[s];
136 }
137 void reduce(int i,int v){
138     int h=remove(i);
139     num[i]=max((int)0,num[i]-v);
140     father[h]=father[i]=merge(h,i);
141 }
142 int compute(){
143     int ans=0,mx=0;
144     for(int i=1;i<=n;i++){
145         if(father[i]==i){
146             ans+=num[i];
147             mx=max(num[i],mx);
148         }
149     }
150     if(w==2) ans-=mx;
151     else if(w==3) ans+=mx;
152     return ans;
153 }
154 void solve(){
155     cin>>n>>m;
156     prepare();
157     for(int i=1;i<=n;i++){
158         cin>>num[i];
159     }
160     for(int i=1;i<=m;i++){
161         int op;
162         cin>>op;
163         if(op==2){
164             int x;
165             cin>>x;
166             reduce(x,num[x]);
167         }else if(op==3){
168             int x,y;
169             cin>>x>>y;
170             reduce(find(x),y);
171         }else {
172             int x,y;
173             cin>>x>>y;
174             x=find(x),y=find(y);
175             if(x!=y) merge(x,y);
176         }
177     }
178     int ans=compute();
179     if(ans==0) cout<<"Gensokyo"<<' ';
180     else if(ans>k) cout<<"Hell"<<' ';
181     else cout<<"Heaven"<<' ';
182     cout<<ans<<endl;
183 }
184 signed main(){
185     ios::sync_with_stdio(0);
186     cin.tie(0), cout.tie(0);
187     cin>>t>>w>>k;
188     while(t--){
189         solve();
190     }
191 }

```

2.21 虚树的构建

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 const int maxn=1e5+10;
4 int h[maxn<<1],ne[maxn<<1],to[maxn<<1],tot1;//初始树

```

```

5 int n,q,k;
6 int hn[maxn<<1],nn[maxn<<1],tn[maxn<<1],tot2;//新树
7 int dep[maxn],st[maxn][20],dfn[maxn],dfncnt;
8 vector<int>arr,tmp;//关键点
9 bool iskey[maxn];
10 int cost[maxn],siz[maxn];
11 bool cmp(int x,int y){
12     return dfn[x]<dfn[y];
13 }
14 void add1(int u,int v){
15     ne[++tot1]=h[u];
16     to[tot1]=v;
17     h[u]=tot1;
18 }
19 void add2(int u,int v){
20     nn[++tot2]=hn[u];
21     tn[tot2]=v;
22     hn[u]=tot2;
23 }
24 void dfs(int u,int f){
25     dep[u]=dep[f]+1;
26     dfn[u]=++dfncnt;
27     st[u][0]=f;
28     for(int i=1;i<20;i++){
29         st[u][i]=st[st[u][i-1]][i-1];
30     }
31     for(int e=h[u];e;e=ne[e]){
32         int v=to[e];
33         if(v==f) continue;
34         dfs(v,u);
35     }
36 }
37 int getlca(int u,int v){
38     if(dep[u]!=dep[v]){
39         if(dep[u]<dep[v]) swap(u,v);
40         for(int p=19;p>=0;p--){
41             if(dep[st[u][p]]>=dep[v]){
42                 u=st[u][p];
43             }
44         }
45     }
46     if(u==v) return u;
47     for(int p=19;p>=0;p--){
48         if(st[u][p]!=st[v][p]){
49             u=st[u][p];
50             v=st[v][p];
51         }
52     }
53     return st[u][0];
54 }
55 int buildvtree(){
56     sort(arr.begin(),arr.end(),cmp);
57     tmp=arr;
58     for(int i=0;i<(int)arr.size()-1;i++){
59         tmp.push_back(getlca(arr[i],arr[i+1]));
60     }
61     sort(tmp.begin(),tmp.end(),cmp);
62     tmp.erase(unique(tmp.begin(),tmp.end()),tmp.end());
63     tot2=0;
64     for(int i=0;i<(int)tmp.size();i++){
65         hn[tmp[i]]=0;
66     }
67     for(int i=0;i<tmp.size()-1;i++){
68         add2(getlca(tmp[i],tmp[i+1]),tmp[i+1]);
69     }
70     return tmp[0];
71 }

```

3 图论

3.1 最短路

```

1 bitset<N> b[N];
2 for (int j = 1; j <= n; ++j) { // 注意中转点在最外层
3     for (int i = 1; i <= n; ++i) {
4         if (b[i][j]) { b[i] |= b[j]; }
5     }
6 }

```

3.2 判奇环 (无向图)

```

1 #include <iostream>

```

```

2 #include <vector>
3 using namespace std;
4 bool dfs(int u, int c, vector<vector<int>>& adj, vector<int>&
   color) {
5     color[u] = c;
6     for (int v : adj[u]) {
7         if (color[v] == -1) {           // 未染色, 染相反
           色并递归
8             if (!dfs(v, 1 - c, adj, color))
9                 return false;
10        } else if (color[v] == c) {     // 邻接点同色, 出
           现奇环
11            return false;
12        }
13    }
14    return true;
15 }
16
17 bool hasOddCycle(int n, vector<vector<int>>& adj) {
18     vector<int> color(n, -1);
19     for (int i = 0; i < n; ++i) {
20         if (color[i] == -1) {
21             if (!dfs(i, 0, adj, color))
22                 return true;
23         }
24     }
25     return false;
26 }
27
28 int main() {
29     int n, m;
30     cout << "请输入顶点数和边数: ";
31     cin >> n >> m;
32
33     vector<vector<int>> adj(n);
34     cout << "请输入每条边的两个端点 (编号从0开始): " << endl;
35     for (int i = 0; i < m; ++i) {
36         int u, v;
37         cin >> u >> v;
38         adj[u].push_back(v);
39         adj[v].push_back(u);
40     }
41
42     if (hasOddCycle(n, adj))
43         cout << "图中存在奇环" << endl;
44     else
45         cout << "图中不存在奇环 (是二分图)" << endl;
46
47     return 0;
48 }

```

3.3 最小生成树

```

1 int n, m;
2 vector<pii> G[N];
3 int kruskal()
4 { // 默认是一张连通图
5     vector<array<int,3>> e(m);
6     for (auto& [w, u, v]: e) { cin >> u >> v >> w; }
7     sort(e.begin(), e.end());
8     int ans = 0, cnt = 0;
9     d = dsu(n); // 见并查集模板
10    for (auto& [w, u, v]: e) {
11        if (d.find(u) != d.find(v)) {
12            d.merge(u, v);
13            ans += w;
14            ++cnt;
15            // 以下两行为建树, 完成后 G 中存放了最小生成树
16            G[u].push_back( { v, w } );
17            G[v].push_back( { u, w } );
18        }
19        if (cnt == n-1) { break; }
20    }
21    return ans;
22 }

```

3.4 拓扑排序

```

1 queue<int> q;
2 for (int i = 1; i <= n; ++i) {
3     if (deg[i] == 0) { q.push(i); }
4 }

```

```

5 vector<int> topo;
6 while (!q.empty()) {
7     int u = q.front(); q.pop();
8     topo.push_back(u);
9     for (int v: G[u]) {
10        if (--deg[v] == 0) { q.push(v); }
11    }
12 }

```

3.5 LCA

```

1 const int N=5e5+5, I=20;
2 int n, m, s, dep[N], f[I+5][N], mn[I+5][N];
3 vector<pii> G[N];
4
5 void dfs(int u, int fa)
6 { // 预处理父节点、深度、路径上的最小值
7     f[0][u] = fa;
8     dep[u] = dep[fa] + 1;
9     for (int i = 1; i <= I; ++i) {
10        f[i][u] = f[i-1][ f[i-1][u] ];
11        mn[i][u] = min( mn[i-1][u], mn[i-1][ f[i-1][u] ] );
12    }
13    for (auto [v, w]: G[u]) {
14        if (v != fa) {
15            mn[0][v] = w;
16            dfs(v, u);
17        }
18    }
19 }
20
21 int LCA(int u, int v)
22 { // 求 u, v 到其 LCA 路径上的最小值
23     int ans = numeric_limits<int>::max();
24     function<void(int&,int)> jump = [&](int &u, int i) {
25         ans = min(ans, mn[i][u]);
26         u = f[i][u];
27     };
28     if (dep[u] < dep[v]) { swap(u, v); }
29     for (int i = I; i >= 0; --i) {
30         if (dep[f[i][u]] >= dep[v]) { jump(u, i); }
31     }
32     if (u == v) { return ans; }
33     for (int i = I; i >= 0; --i) {
34         if (f[i][u] != f[i][v]) {
35             jump(u, i); jump(v, i);
36         }
37     }
38     jump(u, 0); jump(v, 0);
39     return ans;
40 }

```

3.6 树的直径与中心

```

1 int d[N], mxd;
2 void dfs(int u, int fa)
3 {
4     for (auto v: G[u]) {
5         if (v != fa) {
6             dfs(v, u);
7             mxd = max(mxd, d[u]+d[v]+1);
8             d[u] = max(d[u], d[v]+1);
9         }
10    }
11 }

```

3.7 点分治与树的重心

```

1 int n, siz[N], dis[N];
2 bool vis[N];
3 vector<pii> G[N];
4
5 void calcsiz(int u, int fa, int sum, int &core)
6 { // 求树的重心, sum 表述当前树的大小
7     siz[u] = 1;
8     int mxz = 0;
9     for (auto [v, w]: G[u]) {
10        if (v != fa && !vis[v]) {
11            calcsiz(v, u, sum, core);
12            siz[u] += siz[v];
13            mxz = max(mxz, siz[v]);

```

```

14     }
15 }
16 mxz = max(mxz, sum-siz[u]);
17 if (mxz * 2 <= sum) { core = u; }
18 }
19
20 void calcds(int u, int fa)
21 {
22     for (auto [v, w]: G[u]) {
23         if (v != fa && !vis[v]) {
24             dis[v] = dis[u] + w;
25             calcds(v, u);
26         }
27     }
28 }
29
30 void work(int u, int fa)
31 {
32     vis[u] = true;
33     for (auto [v, w]: G[u]) {
34         if (v != fa && !vis[v]) {
35             dis[v] = w;
36             calcds(v, u);
37         }
38     }
39     for (auto [v, w]: G[u]) {
40         if (v != fa && !vis[v]) {
41             int sum = siz[v];
42             int core = 0;
43             calcsiz(v, u, sum, core);
44             calcsiz(core, u, sum, core); // 第二次调用更新
45             siz[core] = sum;
46             work(core, u);
47         }
48     }
49 }
50
51 int main()
52 {
53     int core = 0;
54     calcsiz(1, 0, n, core);
55     calcsiz(core, 0, n, core);
56     work(core, 0);
57 }

```

3.8 基环树

```

1 vector<int> G[N], cycle;
2 void dfs(int u, int fa)
3 {
4     cycle.push_back(u);
5     if (u == t) {
6         sort(cycle.begin(), cycle.end());
7         for (int x: cycle) { cout << x << " "; }
8         exit(0);
9     }
10    for (int v: G[u]) {
11        if (v != fa) { dfs(v, u); }
12    }
13    cycle.pop_back();
14 }
15
16 int main()
17 {
18     dsu d(n); // 见并查集板子
19     for (int i = 1; i <= n; ++i) {
20         int u, v;
21         cin >> u >> v;
22         G[u].push_back(v);
23         G[v].push_back(u);
24         if (d.same(u, v)) { s=u; t=v; }
25         d.merge(u, v);
26     }
27     dfs(s, 0);
28 }

```

3.9 Hierholzer 求欧拉路

```

1 int n, deg[N<<1], cur[N<<1];
2 bool vis[N];
3 vector<int> ans;
4 vector<pair<int,int>> G[N<<1];
5

```

```

6 void dfs(int u)
7 {
8     for (int &j = cur[u]; j < G[u].size(); ++j) {
9         auto [v, i] = G[u][j];
10        if (!vis[i]) {
11            vis[i] = true;
12            dfs(v);
13            ans.push_back(i);
14        }
15    }
16 }
17
18 int main()
19 {
20     int s = 1;
21     while (s <= 2*n && deg[s] % 2 == 0) { ++s; }
22     if (s > 2 * n) {
23         s = 1;
24         while (!deg[s]) { ++s; }
25     }
26
27     int cnt = 0;
28     for (int i = 1; i <= 2*n; ++i) {
29         cnt += deg[i] % 2;
30     }
31     if (cnt > 2) {
32         cout << "NO\n"; return;
33     }
34
35     dfs(s);
36
37     if (ans.size() != n) {
38         cout << "NO\n";
39     } else {
40         cout << "YES\n";
41         for (auto x: ans) {
42             cout << x << " \n"[x == ans.back()];
43         }
44     }
45 }

```

3.10 DSU on tree

```

1 const int N=2e5+5;
2 int n, c[N], siz[N], hvs[N];
3 int L[N], R[N], dfn[N], totdfn;
4 vector<int> G[N];
5
6 void dfs_init(int u, int fa)
7 { // 预处理 dfn 和重儿子
8     siz[u] = 1;
9     L[u] = ++totdfn;
10    dfn[totdfn] = u;
11    for (int v: G[u]) {
12        if (v != fa) {
13            dfs_init(v, u);
14            siz[u] += siz[v];
15            if (hvs[u] == 0 || siz[v] > siz[hvs[u]]) {
16                hvs[u] = v;
17            }
18        }
19    }
20    R[u] = totdfn;
21 }
22
23 void add(int u, int dt)
24 {
25     --ccnt[cnt[c[u]]];
26     cnt[c[u]] += dt;
27     ++ccnt[cnt[c[u]]];
28 }
29
30 void dfs_solve(int u, int fa, bool keep)
31 {
32     for (auto v: G[u]) { // 先做轻儿子, 不保留
33         if (v != fa && v != hvs[u]) {
34             dfs_solve(v, u, false);
35         }
36     }
37     if (hvs[u]) { // 再做重儿子, 保留
38         dfs_solve(hvs[u], u, true);
39     }
40     for (auto v: G[u]) { // 再把轻儿子的信息加上

```

```

41     if (v != fa && v != hvs[u]) {
42         for (int i = L[v]; i <= R[v]; ++i) {
43             add(dfn[i], 1);
44         }
45     }
46 }
47 add(u, 1);
48 ans += (cnt[c[u]] * ccnt[cnt[c[u]]] == siz[u]);
49 if (keep) { return; }
50 for (int i = L[u]; i <= R[u]; ++i) {
51     add(dfn[i], -1);
52 }
53 }

```

3.11 tarjan

```

1 struct SCC {
2     int n, totdfn=0, colcnt=0;
3     stack<int> stk;
4     vector<int> dfn, low, col;
5     vector<vector<int>> G;
6     SCC(int n): n(n) {
7         G.assign(n+1, {});
8         dfn.assign(n+1, 0);
9         low.assign(n+1, 0);
10        col.assign(n+1, 0);
11    }
12    void add_edge(int u, int v) {
13        G[u].push_back(v);
14    }
15    void dfs(int u) {
16        stk.push(u);
17        dfn[u] = low[u] = ++totdfn;
18        for (auto v: G[u]) {
19            if (!dfn[v]) {
20                dfs(v);
21                low[u] = min(low[u], low[v]);
22            } else if (!col[v]) {
23                low[u] = min(low[u], dfn[v]);
24            }
25        }
26        if (low[u] == dfn[u]) {
27            col[u] = ++colcnt;
28            int v;
29            do {
30                v = stk.top(); stk.pop();
31                col[v] = colcnt;
32            } while (v != u);
33        }
34    }
35    vector<int> work() {
36        for (int i = 1; i <= n; ++i) {
37            if (!dfn[i]) { dfs(i); }
38        }
39        return col;
40    }
41 };

```

3.12 树哈希

```

1 using ull = unsigned long long;
2 const ull msk = chrono::steady_clock::now().time_since_epoch().count();
3
4 // ull h(ull x) { return x * x * x * x * 1237123 + 19260817; }
5 // ull f(ull x) { return h(x & ((1 << 31) - 1)) + h(x >> 31); }
6 ull shift(ull x)
7 {
8     x ^= msk;
9     x ^= x << 13;
10    x ^= x >> 7;
11    x ^= x << 17;
12    x ^= msk;
13    return x;
14 }
15
16 ull has[N];
17 void getHash(int u, int fa)
18 {
19     has[u] = 1;
20     for (auto v: G[u]) {

```

```

21         if (v == fa) { continue; }
22         getHash(v, u);
23         has[u] += shift(has[v]);
24     }
25 }

```

3.13 Bellman-Ford / SPFA (判负环)

```

1 //Bellman-Ford + SPFA优化, O(n*m), 可用于判负环
2 #include<bits/stdc++.h>
3 using namespace std;
4 const int maxn=2010,maxm=3010,inf=1e8;
5 int head[maxn<<1],n,m,cnt,upfill[maxn],dis[maxn];
6 queue<int>q; bool enter[maxn];
7 struct graph{ int next,to,w; }g[maxn<<1];
8 void add(int u,int v,int w){ g[cnt].next=head[u]; g[cnt].to=v;
9     g[cnt].w=w; head[u]=cnt++; }
10 void build(int n){ cnt=1; while(!q.empty())q.pop();
11     for(int i=1;i<=n;i++){ head[i]=0; upfill[i]=0; dis[i]=inf;
12         enter[i]=0; } }
13 bool spfa(){ dis[1]=0; upfill[1]++; q.push(1); enter[1]=true;
14     while(!q.empty()){ int x=q.front(); q.pop(); enter[x]=
15         false;
16         for(int ei=head[x];ei;ei=g[ei].next){ int v=g[ei].to,w
17             =g[ei].w;
18             if(dis[x]+w<dis[v]){ dis[v]=dis[x]+w;
19                 if(!enter[v]){ if(upfill[v]++==n)return true;
20                     enter[v]=true; q.push(v); } } } }
21     return false; }

```

3.14 Floyd 全源最短路

```

1 //全源最短路 O(n^3), 允许负边但不能有负环
2 const int maxn=1001,inf=1e8; int dis[maxn][maxn];
3 void floyd(){ for(int b=1;b<=n;b++){ for(int i=1;i<=n;i++){ for(
4     int j=1;j<=n;j++){
5         if(dis[i][b]!=inf&&dis[b][j]!=inf&&dis[i][j]>dis[i][b]+dis
6             [b][j])
7             dis[i][j]=dis[i][b]+dis[b][j]; } } } }

```

3.15 树上倍增

```

1 //快速查询从节点往上走k步的祖先, 预处理O(n log n), 查询O(log n)
2 const int maxn=5e5+10; int st[maxn][32],deep[maxn],power;
3 void dfs(int i,int f){ deep[i]=(i==0?1:deep[f]+1); st[i][0]=f;
4     for(int p=1;(1<<p)<=deep[i];p++) st[i][p]=st[st[i][p-1]][p-1];
5     for(int ei=head[i];ei;ei=tr[ei].n) dfs(tr[ei].t,i); }
6 int getkthAncestor(int i,int k){ if(deep[i]<=k)return -1; int
7     s=deep[i]-k;
8     for(int p=power;p>=0;p--) if(deep[st[i][p]]>=s) i=st[i][p];
9     return i; }

```

3.16 树的直径

```

1 //两次DFS求树的直径
2 int maxd=0,far=1;
3 void dfs(int u,int fa,int d){ if(d>maxd){maxd=d;far=u;} for(
4     int v:g[u])if(v!=fa)dfs(v,u,d+1); }
5 // dfs(1,0,0); dfs(far,0,0); maxd即为直径

```

3.17 树的重心

```

1 void dfs(int u, int fa, int &centroid) {
2     sz[u] = 1;
3     bool ok = true;
4     for (int v : g[u]) {
5         if (v == fa) continue;
6         dfs(v, u, centroid);
7         sz[u] += sz[v];
8         if (sz[v] > n / 2) ok = false;
9     }
10    if (n - sz[u] > n / 2) ok = false;
11    if (ok) centroid = u;
12 }

```

3.18 最大流

```

1  const ll INF = 9e18;
2  struct Flow {
3      vector<tuple<int,ll,int>> G[N]; // 终点, 流量, 反边
4      int dis[N], now[N]; // now[u] 为当前弧
5      void add_edge(int u, int v, ll w) {
6          G[u].push_back( { v, w, G[v].size() } );
7          G[v].push_back( { u, 0, G[u].size()-1 } );
8      }
9      void bfs(int st) { // 求层次图
10         queue<int> q;
11         memset(dis, 0, sizeof(dis));
12         q.push(st); dis[st] = 1;
13         while (!q.empty()) {
14             int u = q.front(); q.pop();
15             for (auto& [v, w, inv]: G[u]) {
16                 if (w != 0 && dis[v] == 0) {
17                     dis[v] = dis[u] + 1;
18                     q.push(v);
19                 }
20             }
21         }
22     }
23     ll dfs(int u, int t, ll flow) { // 求阻塞流
24         if (u == t) { return flow; }
25         for (int &i=now[u]; i < (int)G[u].size(); ++i) {
26             auto& [v, w, inv] = G[u][i];
27             if (w != 0 && dis[v] > dis[u]) {
28                 ll d = dfs(v, t, min(flow, w));
29                 if (d > 0) {
30                     w -= d;
31                     get<1>(G[v][inv]) += d;
32                     return d;
33                 }
34             }
35         }
36         return 0;
37     }
38     ll dicnic(int st, int ed) {
39         for (ll flow=0, res;;) {
40             bfs(st);
41             if (dis[ed] == 0) { return flow; }
42             memset(now, 0, sizeof(now));
43             while ((res=dfs(st,ed,INF)) > 0) {
44                 flow += res;
45             }
46         }
47     }
48 };

```

3.19 最近公共祖先 (LCA)

```

1  //利用树链剖分的方法
2  #include<bits/stdc++.h>
3  using namespace std;
4  const int N=5e5+5;
5  struct node{
6      int next,to;
7  }e[N<<1];
8  int tot,head[N];
9  int dep[N],siz[N],hson[N],fa[N];
10 int top[N];
11 int n,m,s;
12 void add(int x,int y){
13     e[++tot].to=y;
14     e[tot].next=head[x];
15     head[x]=tot;
16 }
17 void dfs1(int x,int f){
18     siz[x]=1;
19     fa[x]=f;
20     dep[x]=dep[f]+1;
21     for(int i=head[x];i;i=e[i].next){
22         int y=e[i].to;
23         if(y!=f){
24             dfs1(y,x);
25             siz[x]+=siz[y];
26             if(siz[hson[x]]<siz[y] || !hson[x]){
27                 hson[x]=y;
28             }
29         }
30     }
31 }

```

```

32 void dfs2(int x,int t){
33     top[x]=t;
34     if(!hson[x])return ;
35     dfs2(hson[x],t);
36     for(int i=head[x];i;i=e[i].next){
37         int y=e[i].to;
38         if(y!=fa[x] && y!=hson[x]){
39             dfs2(y,y);
40         }
41     }
42 }
43 int LCA(int x,int y){
44     while(top[x]!=top[y]){
45         if(dep[top[x]]<dep[top[y]]){
46             swap(x,y);
47         }
48         x=fa[top[x]];
49     }
50     return dep[x]<dep[y]?x:y;
51 }
52 int main(){
53     ios::sync_with_stdio(false);
54     cin.tie(NULL);
55     cout.tie(NULL);
56     cin>>n>>m>>s;
57     for(int i=1;i<n;i++){
58         int x,y;
59         cin>>x>>y;
60         add(x,y);
61         add(y,x);
62     }
63     dfs1(s,0);
64     dfs2(s,s);
65     for(int i=1;i<=m;i++){
66         int a,b;
67         cin>>a>>b;
68         cout<<LCA(a,b)<<'\\n';
69     }
70     return 0;
71 }

```

3.20 树链剖分

```

1  #include<algorithm>
2  #include<iostream>
3  #include<cstdlib>
4  #include<cstring>
5  #include<cstdio>
6  #define Rint register int
7  #define mem(a,b) memset(a,(b),sizeof(a))
8  #define Temp template<typename T>
9  using namespace std;
10 typedef long long LL;
11 Temp inline void read(T &x){
12     x=0;T w=1,ch=getchar();
13     while(!isdigit(ch)&&ch!='-')ch=getchar();
14     if(ch=='-')w=-1,ch=getchar();
15     while(isdigit(ch))x=(x<<3)+(x<<1)+(ch^'0'),ch=getchar();
16     x=x*w;
17 }
18
19 #define mid ((l+r)>>1)
20 #define lson rt<<1,l,mid
21 #define rson rt<<1|1,mid+1,r
22 #define len (r-l+1)
23
24 const int maxn=200000+10;
25 int n,m,r,mod;
26 //见题意
27 int e,beg[maxn],nex[maxn],to[maxn],w[maxn],wt[maxn];
28 //链式前向星数组, w[], wt[]初始点权数组
29 int a[maxn<<2],laz[maxn<<2];
30 //线段树数组、Lazy操作
31 int son[maxn],id[maxn],fa[maxn],cnt,dep[maxn],siz[maxn],top[
    maxn];
32 //son[]重儿子编号,id[]新编号,fa[]父亲节点,cnt dfs_clock/dfs序,
    dep[]深度,siz[]子树大小,top[]当前链顶端节点
33 int res=0;
34 //查询答案
35
36 inline void add(int x,int y){//链式前向星加边
37     to[++e]=y;
38     nex[e]=beg[x];

```

```

39     beg[x]=e;
40 }
41 //----- 以下为线段树
42 inline void pushdown(int rt,int lenn){
43     laz[rt<<1]+=laz[rt];
44     laz[rt<<1|1]+=laz[rt];
45     a[rt<<1]+=laz[rt]*(lenn-(lenn>>1));
46     a[rt<<1|1]+=laz[rt]*(lenn>>1);
47     a[rt<<1]%mod;
48     a[rt<<1|1]%mod;
49     laz[rt]=0;
50 }
51
52 inline void build(int rt,int l,int r){
53     if(l==r){
54         a[rt]=wt[l];
55         if(a[rt]>mod)a[rt]%mod;
56         return;
57     }
58     build(lson);
59     build(rson);
60     a[rt]=(a[rt<<1]+a[rt<<1|1])%mod;
61 }
62
63 inline void query(int rt,int l,int r,int L,int R){
64     if(L<=l&&r<=R){res+=a[rt];res%=mod;return;}
65     else{
66         if(laz[rt])pushdown(rt,len);
67         if(L<=mid)query(lson,L,R);
68         if(R>mid)query(rson,L,R);
69     }
70 }
71
72 inline void update(int rt,int l,int r,int L,int R,int k){
73     if(L<=l&&r<=R){
74         laz[rt]+=k;
75         a[rt]+=k*len;
76     }
77     else{
78         if(laz[rt])pushdown(rt,len);
79         if(L<=mid)update(lson,L,R,k);
80         if(R>mid)update(rson,L,R,k);
81         a[rt]=(a[rt<<1]+a[rt<<1|1])%mod;
82     }
83 }
84 //----- 以上为线段树
85 inline int qRange(int x,int y){
86     int ans=0;
87     while(top[x]!=top[y]){//当两个点不在同一条链上
88         if(dep[top[x]]<dep[top[y]])swap(x,y);//把x点改为所在链顶端
89         //的深度更深的那个点
90         res=0;
91         query(1,1,n,id[top[x]],id[x]);//ans加上x点到x所在链顶端 这
92         //一段区间的点权和
93         ans+=res;
94         ans%=mod;//按题意取模
95         x=fa[top[x]];//把x跳到x所在链顶端的那个点的上面一个点
96     }
97     //直到两个点处于一条链上
98     if(dep[x]>dep[y])swap(x,y);//把x点深度更深的那个点
99     res=0;
100     query(1,1,n,id[x],id[y]);//这时再加上此时两个点的区间和即可
101     ans+=res;
102     return ans%mod;
103 }
104
105 inline void updRange(int x,int y,int k){//同上
106     k%=mod;
107     while(top[x]!=top[y]){
108         if(dep[top[x]]<dep[top[y]])swap(x,y);
109         update(1,1,n,id[top[x]],id[x],k);
110         x=fa[top[x]];
111     }
112     if(dep[x]>dep[y])swap(x,y);
113     update(1,1,n,id[x],id[y],k);
114 }
115
116 inline int qSon(int x){
117     res=0;
118     query(1,1,n,id[x],id[x]+siz[x]-1);//子树区间右端点为id[x]+
119     //siz[x]-1
120     return res;
121 }

```

```

120 inline void updSon(int x,int k){//同上
121     update(1,1,n,id[x],id[x]+siz[x]-1,k);
122 }
123
124 inline void dfs1(int x,int f,int deep){//x当前节点, f父亲,
125     //deep深度
126     dep[x]=deep;//标记每个点的深度
127     fa[x]=f;//标记每个点的父亲
128     siz[x]=1;//标记每个非叶子节点的子树大小
129     int maxson=-1;//记录重儿子的儿子数
130     for(Rint i=beg[x];i=nex[i]){
131         int y=to[i];
132         if(y==f)continue;//若为父亲则continue
133         dfs1(y,x,deep+1);//dfs其儿子
134         siz[x]+=siz[y];//把它的儿子数加到它身上
135         if(siz[y]>maxson)son[x]=y,maxson=siz[y];//标记每个非叶子节
136         //点的重儿子编号
137     }
138
139     inline void dfs2(int x,int topf){//x当前节点, topf当前链的最顶
140     //端的节点
141     id[x]++;cnt;//标记每个点的新编号
142     wt[cnt]=w[x];//把每个点的初始值赋到新编号上来
143     top[x]=topf;//这个点所在链的顶端
144     if(!son[x])return;//如果没有儿子则返回
145     dfs2(son[x],topf);//按先处理重儿子, 再处理轻儿子的顺序递归处
146     //理
147     for(Rint i=beg[x];i=nex[i]){
148         int y=to[i];
149         if(y==fa[x]||y==son[x])continue;
150         dfs2(y,y);//对于每一个轻儿子都有一条从它自己开始的链
151     }
152 }
153
154 int main(){
155     read(n);read(m);read(r);read(mod);
156     for(Rint i=1;i<=n;i++)read(w[i]);
157     for(Rint i=1;i<=n;i++){
158         int a,b;
159         read(a);read(b);
160         add(a,b);add(b,a);
161     }
162     dfs1(r,0,1);
163     dfs2(r,r);
164     build(1,1,n);
165     while(m--){
166         int k,x,y,z;
167         read(k);
168         if(k==1){
169             read(x);read(y);read(z);
170             updRange(x,y,z);
171         }
172         else if(k==2){
173             read(x);read(y);
174             printf("%d\n",qRange(x,y));
175         }
176         else if(k==3){
177             read(x);read(y);
178             updSon(x,y);
179         }
180         else{
181             read(x);
182             printf("%d\n",qSon(x));
183         }
184     }
185 }

```

3.21 最大路匹配

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 const int N=510;
4 vector<int>g[N];
5 bool vis[N];
6 int match[N];
7 bool dfs(int u){
8     for(int v:g[u]){
9         if(!vis[v]){
10             vis[v]=1;
11             if(match[v]==0||dfs(match[v])){
12                 match[v]=u;
13                 return true;

```

```

14     }
15     }
16 }
17 return false;
18 }
19 int main(){
20     int n,m,ans=0;
21     cin>>n>>m;
22     for(int i=1;i<=n;i++){
23         int x;
24         cin>>x;
25         for(int j=1;j<=x;j++){
26             int y;
27             cin>>y;
28             g[i].push_back(y);
29         }
30     }
31     for(int i=1;i<=n;i++){
32         memset(vis,0,sizeof(vis));
33         if(dfs(i)) ans++;
34     }
35     cout<<ans;
36 }

```

3.22 Dijkstra 算法

```

1 //小根堆实现较为简单，这里采用以点为复杂度的反向索引堆的优化，
  一般不会用
2 #include<bits/stdc++.h>
3 using namespace std;
4 const int maxn=100010;
5 const int inf=1e8;
6 struct graph{
7     int next,to,w;
8 }g[maxn*2];
9 int head[maxn],cnt,n,m;
10 int heap[maxn];//反向索引堆
11 int where[maxn];//表示点在堆上位置，初始化为-1
12 int heapSize;
13 int dis[maxn];
14 void build(int n){
15     cnt=1;
16     heapSize=0;
17     for(int i=1;i<=n;i++){
18         heap[i]=0;
19         where[i]=-1;
20         head[i]=0;
21         dis[i]=inf;
22     }
23 }
24 void add(int u,int v,int w){
25     g[cnt].next=head[u];
26     g[cnt].to=v;
27     g[cnt].w=w;
28     head[u]=cnt++;
29 }
30 void Swap(int i,int j){
31     swap(heap[i],heap[j]);
32     where[heap[i]]=i;
33     where[heap[j]]=j;
34 }
35 void heapInsert(int i){
36     while(dis[heap[i]]<dis[heap[(i-1)/2]]){
37         Swap(i,(i-1)/2);
38         i=(i-1)/2;
39     }
40 }
41 void addorupdateoringnore(int v,int c){
42     if(where[v]==-1){
43         heap[heapSize]=v;
44         where[v]=heapSize++;
45         dis[v]=c;
46         heapInsert(where[v]);
47     }else if(where[v]>=0){
48         dis[v]=min(dis[v],c);
49         heapInsert(where[v]);
50     }
51 }
52 void heapify(int i){
53     int l=1;
54     while(l<heapSize){
55         int best=l+1<heapSize&&dis[heap[l+1]]<dis[heap[l]]?l

```

```

56         best=dis[heap[best]]<dis[heap[i]]?best:i;
57         if(best==i) break;
58         Swap(best,i);
59         i=best;
60         l=i*2+1;
61     }
62 }
63 bool isEmpty(){
64     return heapSize==0;
65 }
66 int pop(){
67     int ans=heap[0];
68     swap(heap[0],heap[--heapSize]);
69     heapify(0);
70     where[ans]=-2;
71     return ans;
72 }
73 int main(){
74     cin>>n>>m;
75     int s;
76     cin>>s;
77     build(n);
78     for(int i=1;i<=m;i++){
79         int u,v,w;
80         cin>>u>>v>>w;
81         add(u,v,w);
82     }
83     addorupdateoringnore(s,0);
84     while(!isEmpty()){
85         int u=pop();
86         for(int ei=head[u];ei>0;ei=g[ei].next){
87             addorupdateoringnore(g[ei].to,dis[u]+g[ei].w);
88         }
89     }
90     int ans=-inf;
91     for(int i=1;i<=n;i++){
92         /*if(dis[i]==inf){
93             cout<<-1<<endl;
94         }
95         ans=max(ans,dis[i]);*/
96         cout<<dis[i]<<' ';
97     }
98     //cout<<ans<<endl;
99 }

```

3.23 分层图最短路

```

1 //无论是在bfs上分层还是dijkstra上，本质其实就是把原图上的点与
  其的不同状态整合出更多的点
2 //难点在于如何将点上的状态给储存下来，以及如何转移
3 #include<bits/stdc++.h>
4 using namespace std;
5 const int maxn=50010;
6 const int inf=1e8;
7 struct dot{
8     int x,cost,k;
9 };
10 struct graph{
11     int next,to,w;
12 }g[maxn<<1];
13 struct compare{
14     bool operator()(dot x,dot y){
15         return x.cost>y.cost;
16         // 小根堆
17     }
18 };
19 int head[maxn<<1],cnt=1,n,m,k,dis[maxn][20];
20 bool vis[maxn][20];
21 priority_queue<dot,vector<dot>,compare>q;
22 void add(int u,int v,int w){
23     g[cnt].next=head[u];
24     g[cnt].to=v;
25     g[cnt].w=w;
26     head[u]=cnt++;
27 }
28 int main(){
29     cin>>n>>m>>k;
30     int s,t;
31     cin>>s>>t;
32     for(int i=1;i<=m;i++){
33         int u,v,w;
34         cin>>u>>v>>w;
35         add(u,v,w);

```

```

36     add(v,u,w);
37 }
38 for(int i=0;i<=n;i++){
39     for(int j=0;j<=k;j++){
40         dis[i][j]=inf;
41     }
42 }
43 dis[s][0]=0;
44 q.push({s,0,0});
45 while(!q.empty()){
46     int xn=q.top().x;
47     int cost=q.top().cost;
48     int kn=q.top().k;
49     q.pop();
50     if(vis[xn][kn] continue;
51     vis[xn][kn]=1;
52     if(xn==t){
53         cout<<cost<<endl;
54         return 0;
55     }
56     for(int i=head[xn];i>0;i=g[i].next){
57         int v=g[i].to;
58         if(kn<k&&cost<dis[v][kn+1]){
59             dis[v][kn+1]=cost;
60             q.push({v,cost,kn+1});
61         }
62         if(cost+g[i].w<dis[v][kn]){
63             dis[v][kn]=cost+g[i].w;
64             q.push({v,cost+g[i].w,kn});
65         }
66     }
67 }
68 }
69 /*
70 struct Compare {
71     bool operator()(const State& a, const State& b) {
72         return a.cost > b.cost; // 小根堆
73     }
74 };
75 priority_queue<State, vector<State>, Compare> q;
76 这样子定义也可以
77 */

```

3.24 双向广搜

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4 ll n,m;
5 ll an[45];
6 ll l[1<<20+1],r[1<<20+1];
7 ll f(int i,int e,ll s,ll w,ll a[],ll j){
8     if(s>w) return j;
9     if(i==e) a[j++]=s;
10    else {
11        j=f(i+1,e,s,w,a,j);
12        j=f(i+1,e,s+an[i],w,a,j);
13    }
14    return j;
15 }
16 ll compute(){
17     ll lsize=f(0,n>>1,0,m,1,0);
18     ll rsize=f(n>>1,n,0,m,r,0);
19     sort(l,l+lsize);
20     sort(r,r+rsize);
21     ll ans=0;
22     for(int i=lsize-1,j=0;i>0;i--){
23         while(j<rsize&&l[i]+r[j]<=m){
24             j++;
25         }
26         ans+=j;
27     }
28     return ans;
29 }
30 int main(){
31     cin>>n>>m;
32     for(int i=0;i<n;i++) cin>>an[i];
33     sort(an,an+n);
34     cout<<compute();
35 }

```

3.25 tarjan(强连通分量)

```

1 //强连通分量求解
2 int dfn[N], low[N], dfncnt, s[N], in_stack[N], tp;
3 int scc[N], sc; // 结点 i 所在 SCC 的编号
4 int sz[N]; // 强连通 i 的大小
5
6 void tarjan(int u) {
7     low[u] = dfn[u] = ++dfncnt, s[++tp] = u, in_stack[u] = 1;
8     for (int i = h[u]; i; i = e[i].nex) {
9         const int &v = e[i].t;
10        if (!dfn[v]) {
11            tarjan(v);
12            low[u] = min(low[u], low[v]);
13        } else if (in_stack[v]) {
14            low[u] = min(low[u], dfn[v]);
15        }
16    }
17    if (dfn[u] == low[u]) {
18        ++sc;
19        do {
20            scc[s[tp]] = sc;
21            sz[sc]++;
22            in_stack[s[tp]] = 0;
23        } while (s[tp--] != u);
24    }
25 }

```

3.26 边双连通分量

```

1 #include <algorithm>
2 #include <iostream>
3 #include <stack>
4 #include <vector>
5
6 using namespace std;
7 constexpr int N = 5e5 + 5, M = 2e6 + 5;
8 int n, m;
9
10 struct edge {
11     int to, nt;
12 } e[M << 2];
13
14 int hd[N << 1], tot = 1;
15
16 void add(int u, int v) { e[++tot] = edge{v, hd[u]}, hd[u] = tot; }
17
18 void uadd(int u, int v) { add(u, v), add(v, u); }
19
20 int bfnct, sum;
21 int dfn[N], low[N], bcc[N], bcc_cnt;
22 bool vis[N];
23 vector<vector<int>> ans;
24 stack<int> st;
25 void tarjan(int u, int in) {
26     low[u] = dfn[u] = ++bfnct;
27     st.push(u), vis[u] = true;
28     for (int i = hd[u]; i; i = e[i].nt) {
29         int v = e[i].to;
30         if (i == (in ^ 1)) continue;
31         if (!dfn[v]) {
32             tarjan(v, i), low[u] = min(low[u], low[v]);
33         } else if (vis[v]) {
34             low[u] = min(low[u], dfn[v]);
35         }
36     }
37     if (dfn[u] == low[u]) {
38         bcc_cnt++;
39         bcc[u] = bcc_cnt;
40         vector<int> t;
41         t.push_back(u);
42         while (st.top() != u) {
43             t.push_back(st.top()), bcc[st.top()] = bcc_cnt, vis[st.top()] = false, st.pop();
44         }
45         st.pop(), ans.push_back(t);
46     }
47 }

```

3.27 割边

```

1 #include<bits/stdc++.h>
2 using namespace std;

```

```

3  const int maxn=1e5+10;
4  int n,m;
5  int dfn[maxn],low[maxn],dfncnt,cutedge[maxn<<1],tot=1,head[
    maxn<<1],ans1;
6  struct graph{
7      int t,nex;
8  }e[maxn<<1];
9  void add(int u,int v){
10     e[++tot].t=v;
11     e[tot].nex=head[u];
12     head[u]=tot;
13 }
14 void tarjan(int u,int pre) {
15     low[u] = dfn[u] = ++dfncnt;
16     for (int ei = head[u]; ei; ei = e[ei].nex) {
17         if((ei^1)==pre) continue;
18         const int &v = e[ei].t;
19         if (!dfn[v]) {
20             tarjan(v,ei);
21
22             low[u] = min(low[u], low[v]);
23             if(low[v]>dfn[u]){
24                 cutedge[ei]>>1; // 无向图割边
25                 ans1++;
26             }
27         } else {
28             low[u] = min(low[u], dfn[v]);
29         }
30     }
31 }
32 int main(){
33     cin>>n>>m;
34     for(int i=1;i<=m;i++){
35         int u,v;
36         cin>>u>>v;
37         add(u,v);
38         add(v,u);
39     }
40     for(int i=1;i<=n;i++){
41         if(!dfn[i]) tarjan(i,0);
42     }
43     cout<<ans1<<endl;
44     for(int i=1;i<=m;i++){
45         if(cutedge[i]) cout<<i<<' ';
46     }
47 }

```

3.28 点双连通分量

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  constexpr int N = 5e5 + 5, M = 2e6 + 5;
5  int n, m;
6
7  struct edge {
8      int to, nt;
9  } e[M << 1];
10
11 int hd[N], tot = 1;
12
13 void add(int u, int v) { e[++tot] = edge{v, hd[u]}, hd[u] = tot; }
14
15 void uadd(int u, int v) { add(u, v), add(v, u); }
16
17 int ans;
18 int dfn[N], low[N], bcc_cnt;
19 int sta[N], top, cnt;
20 bool cut[N];
21 vector<int> dcc[N];
22 int root;
23
24 void tarjan(int u) {
25     dfn[u] = low[u] = ++bcc_cnt, sta[++top] = u;
26     if (u == root && hd[u] == 0) {
27         dcc[++cnt].push_back(u);
28         return;
29     }
30     int f = 0;
31     for (int i = hd[u]; i; i = e[i].nt) {
32         int v = e[i].to;
33         if (!dfn[v]) {

```

```

34         tarjan(v);
35         low[u] = min(low[u], low[v]);
36         if (low[v] >= dfn[u]) {
37             if (++f > 1 || u != root) cut[u] = true;
38             cnt++;
39             do dcc[cnt].push_back(sta[top--]);
40             while (sta[top + 1] != v);
41             dcc[cnt].push_back(u);
42         }
43     } else
44         low[u] = min(low[u], dfn[v]);
45 }
46 }
47
48 int main() {
49     cin.tie(nullptr)->sync_with_stdio(false);
50     cin >> n >> m;
51     int u, v;
52     for (int i = 1; i <= m; i++) {
53         cin >> u >> v;
54         if (u != v) uadd(u, v);
55     }
56     for (int i = 1; i <= n; i++)
57         if (!dfn[i]) root = i, tarjan(i);
58     cout << cnt << '\n';
59     for (int i = 1; i <= cnt; i++) {
60         cout << dcc[i].size() << ' ';
61         for (int j = 0; j < dcc[i].size(); j++) cout << dcc[i][j]
62             << ' ';
63         cout << '\n';
64     }
65     return 0;
66 }

```

3.29 割点

```

1  /*
2  洛谷 P3388 【模板】割点（割顶）
3  */
4  #include <iostream>
5  #include <vector>
6  using namespace std;
7  int n, m; // n: 点数 m: 边数
8  int dfn[100001], low[100001], idx, res;
9  // dfn: 记录每个点的时间戳
10 // low: 能经过父亲到达最小的编号, idx: 时间戳, res: 答案数量
11 bool vis[100001], flag[100001]; // flag: 答案 vis: 标记是否重
    复
12 vector<int> edge[100001]; // 存图用的
13
14 void Tarjan(int u, int fa) { // u 当前点的编号, fa 自己爸爸的
    编号
15     vis[u] = true; // 标记
16     low[u] = dfn[u] = ++idx; // 打上时间戳
17     int child = 0; // 每一个点儿子数量
18     for (const auto &v : edge[u]) { // 访问这个点的所有邻居 (C
        ++11)
19         if (!vis[v]) {
20             child++; // 多了一个儿子
21             Tarjan(v, u); // 继续
22             low[u] = min(low[u], low[v]); // 更新能到的最小节点编号
23             if (fa != u && low[v] >= dfn[u] && !flag[u]) { // 主要
                代码
24                 // 如果不是自己, 且不通过父亲返回的最小点符合割点的要
                求, 并且没有被标记过
25                 // 要求即为: 删了父亲连不上去了, 即为最多连到父亲
26                 flag[u] = true;
27                 res++; // 记录答案
28             }
29         } else if (v != fa) {
30             // 如果这个点不是自己的父亲, 更新能到的最小节点编号
31             low[u] = min(low[u], dfn[v]);
32         }
33     }
34     // 主要代码, 自己的话需要 2 个儿子才可以
35     if (fa == u && child >= 2 && !flag[u]) {
36         flag[u] = true;
37         res++; // 记录答案
38     }
39 }
40
41 int main() {
42     cin >> n >> m; // 读入数据

```

```

43 for (int i = 1; i <= m; i++) { // 注意点是从 1 开始的
44     int x, y;
45     cin >> x >> y;
46     edge[x].push_back(y);
47     edge[y].push_back(x);
48 } // 使用 vector 存图
49 for (int i = 1; i <= n; i++) // 因为 Tarjan 图不一定连通
50     if (!vis[i]) {
51         idx = 0; // 时间戳初始为 0
52         Tarjan(i, i); // 从第 i 个点开始, 父亲为自己
53     }
54 cout << res << endl;
55 for (int i = 1; i <= n; i++)
56     if (flag[i]) cout << i << " "; // 输出结果
57 return 0;
58 }

```

4 数学

4.1 线性筛

```

1 vector<int> sieve(int n)
2 {
3     vector<bool> vis(n+1);
4     vector<int> mP(n+1,1);
5     vector<int> prime;
6     vis[1] = 1;
7     for (int i = 2; i <= n; ++i) {
8         if (!vis[i]) {
9             prime.push_back(i);
10            mP[i] = i;
11        }
12        for (auto p: prime) {
13            if (i*p > n) { break; }
14            vis[i*p] = 1;
15            mP[i*p] = p;
16            if (i%p == 0) { break; } // 线性筛关键优化
17        }
18    }
19    return prime;
20 }
21
22 // 分解质因数
23 map<int,int> cnt;
24 for (int y = x; y > 1; y /= mP[y]) {
25     ++cnt[mP[y]];
26 }

```

4.2 因数

```

1 // const int X=1e5;
2 // vector<int> fac[X+5];
3 // for (int i = 1; i <= X; ++i) {
4 //     for (int j = i; j <= X; j += i) {
5 //         fac[j].push_back(i);
6 //     }
7 // }
8 vector<int> getFactors(int x){
9     vector<pair<int,int>> fac;
10
11     for(int p=2;p<=x;p++){
12         if(x%p!=0) continue;
13
14         int cnt=0;
15         while(x%p==0){
16             x/=p;
17             cnt++;
18         }
19         fac.push_back({p,cnt});
20     }
21
22     if(x>1)
23         fac.push_back({x,1});
24
25     vector<int> factors={1};
26
27     for(auto [p,cnt]:fac){
28         int sz=factors.size();
29         int mul=1;
30
31         for(int i=1;i<=cnt;i++){

```

```

33         mul*=p;
34
35         for(int j=0;j<sz;j++){
36             factors.push_back(factors[j]*mul);
37         }
38     }
39
40     return factors;
41 }

```

4.3 拉格朗日插值

```

1 #include <iostream>
2 #include <vector>
3 constexpr int MOD = 998244353;
4 using LL = long long;
5
6 int inv(int k) {
7     int res = 1;
8     for (int e = MOD - 2; e; e /= 2) {
9         if (e & 1) res = (LL)res * k % MOD;
10        k = (LL)k * k % MOD;
11    }
12    return res;
13 }
14
15 // 返回 f 满足 f(x_i) = y_i
16 // 不考虑乘法逆元的时间, 显然为 O(n^2)
17 std::vector<int> lagrange_interpolation(const std::vector<int>
18     &x,
19     const std::vector<int>
20     &y) {
21     const int n = x.size();
22     std::vector<int> M(n + 1), px(n, 1), f(n);
23     M[0] = 1;
24     // 求出 M(x) = prod_{i=0..n-1}(x - x_i)
25     for (int i = 0; i < n; ++i) {
26         for (int j = i; j >= 0; --j) {
27             M[j + 1] = (M[j] + M[j + 1]) % MOD;
28             M[j] = (LL)M[j] * (MOD - x[i]) % MOD;
29         }
30     }
31     // 求出 px_i = prod_{j=0..n-1, j!=i}(x_i - x_j)
32     for (int i = 0; i < n; ++i) {
33         for (int j = 0; j < n; ++j)
34             if (i != j) {
35                 px[i] = (LL)px[i] * (x[i] - x[j] + MOD) % MOD;
36             }
37     }
38     // 组合出 f(x) = sum_{i=0..n-1}(y_i / px_i)(M(x) / (x - x_i))
39     for (int i = 0; i < n; ++i) {
40         LL t = (LL)y[i] * inv(px[i]) % MOD, k = M[n];
41         for (int j = n - 1; j >= 0; --j) {
42             f[j] = (f[j] + k * t) % MOD;
43             k = (M[j] + k * x[i]) % MOD;
44         }
45     }
46     return f;
47 }
48
49 int main() {
50     std::ios::sync_with_stdio(false);
51     std::cin.tie(nullptr);
52     int n, k;
53     std::cin >> n >> k;
54     // 求 f(k)
55     std::vector<int> x(n), y(n);
56     for (int i = 0; i < n; ++i) std::cin >> x[i] >> y[i];
57     const auto f = lagrange_interpolation(x, y);
58     int v = 0;
59     for (int i = n - 1; i >= 0; --i) v = ((LL)v * k + f[i]) % MOD;
60     std::cout << v << '\n';
61     return 0;
62 }

```

4.4 快速幂与乘法逆元

```

1 const int MOD=998244353;
2 ll qpow(ll a, ll b=MOD-2)
3 { // 快速幂函数, x 的乘法逆元为 qpow(x)

```

```

4  ll res=1;
5  for (; b; b>>=1) {
6      if (b & 1) { res=(res*a)%MOD; }
7      a = a*a%MOD; // 不要少 MOD
8  }
9  return res;
10 }

```

4.5 排列组合数

```

1  const int N=1e5;
2  ll fac[N+5], Inv[N+5];
3  ll C(ll n, ll k) { return n==0? 1: fac[n] * Inv[k] % MOD * Inv
4      [n-k] % MOD; }
5  ll P(ll n, ll k) { return n==0? 1: fac[n] * Inv[n-k] % MOD; }
6  // 预处理阶乘和阶乘的逆元
7  for (int i = 1; i <= N; ++i) {
8      fac[i] = fac[i-1] * i % MOD;
9      Inv[i] = qpow(fac[i]);
10 }

```

4.6 Lucas

```

1  const int MOD = 1000003;
2  ll C(ll a, ll b) {
3      if (a < b) { return 0; }
4      ll down = 1, up = 1;
5      for (int i = a, j = 1; j <= b; --i, ++j) {
6          up = up * i % MOD;
7          down = down * j % MOD;
8      }
9      return up * qpow(down) % MOD;
10 }
11 ll lucas(ll a, ll b) {
12     if (a < p && b < p) { return C(a, b); }
13     return C(a%MOD, b%MOD) * lucas(a/MOD, b/MOD) % MOD;
14 }

```

4.7 卡特兰数

```

1  ll Cat(ll n) { return C(2*n, n) * qpow(n+1) % MOD; }

```

4.8 数论分块

```

1  ll l=1, r=0;
2  while (l <= n) {
3      r = n / (n/l);
4      // 统计 ans
5      l = r + 1;
6  }

```

4.9 高斯消元

```

1  int Guess(vector<vector<db>> &a)
2  {
3      int n = a.size();
4      int c, r;
5      for (c = 0, r = 0; c < n; ++c) {
6          // 选主元
7          int t = r;
8          for (int i = r + 1; i < n; ++i) {
9              if (fabs(a[i][c]) > fabs(a[t][c])) { t = i; }
10             }
11             if (sgn(a[t][c]) == 0) { continue; }
12
13             // 行交换
14             for (int i = c; i <= n; ++i) { swap(a[t][i], a[r][i]); }
15
16             // 归一化
17             for (int i = n; i >= c; --i) { a[r][i] /= a[r][c]; }
18
19             // 前向消元
20             for (int i = r + 1; i < n; ++i) {
21                 if (sgn(a[i][c]) != 0) {
22                     for (int j = n; j >= c; --j) {
23                         a[i][j] -= a[r][j] * a[i][c];
24                     }
25                 }
26             }
27         }
28     }
29 }

```

```

26     }
27     ++r;
28 }
29
30 if (r < n) {
31     for (int i = r; i < n; ++i) {
32         if (sgn(a[i][n]) != 0) { return 2; } // 无解
33     }
34     return 1; // 无穷多解
35 }
36
37 // 回代
38 for (int i = n - 1; i >= 0; --i) {
39     for (int j = i + 1; j < n; ++j) {
40         a[i][n] -= a[i][j] * a[j][n];
41     }
42 }
43 return 0;
44 }

```

4.10 矩阵快速幂

```

1  using matrix = vector<vector<ll>>;
2  matrix IM(int n) {
3      matrix I(n, vector<ll>(n));
4      for (int i = 0; i < n; ++i) { I[i][i] = 1; }
5      return I;
6  }
7  matrix operator* (matrix A, matrix B) {
8      assert(A.front().size() == B.size());
9      int n = A.size(), m = B.size(), k = B.front().size();
10     matrix C(n, vector<ll>(m));
11     for (int i = 0; i < n; ++i) {
12         for (int j = 0; j < k; ++j) {
13             for (int x = 0; x < m; ++x) {
14                 C[i][j] = add(C[i][j], mul(A[i][x], B[x][j]));
15             }
16         }
17     }
18     return C;
19 }
20 matrix qpow(matrix a, ll b) {
21     matrix res = IM(a.size());
22     for (; b; b>>=1, a=a*a) {
23         if (b & 1) { res = res * a; }
24     }
25     return res;
26 }

```

4.11 自适应辛普森法

```

1  double simpson(double l, double r) {
2      double mid = (l + r) / 2;
3      return (r - l) * (f(l) + 4 * f(mid) + f(r)) / 6; // 辛普
4      森公式
5  }
6
7  double asr(double l, double r, double eps, double ans,
8      int step) { // step是递归的下限
9      double mid = (l + r) / 2;
10     double fl = simpson(l, mid), fr = simpson(mid, r);
11     if (abs(fl + fr - ans) <= 15 * eps && step < 0)
12         return fl + fr + (fl + fr - ans) / 15; // 足够相似的话就直接返回
13     return asr(l, mid, eps / 2, fl, step - 1) +
14         asr(mid, r, eps / 2, fr, step - 1); // 否则分割成
15         两段递归求解
16 }
17
18 double calc(double l, double r, double eps) {
19     return asr(l, r, eps, simpson(l, r), 12);
20 }

```

4.12 多项式

```

1  struct cp {
2      db x, y;
3      cp(db real=0, db imag=0): x(real), y(imag) { };
4      cp operator+ (cp b) const { return {x+b.x, y+b.y}; }
5      cp operator- (cp b) const { return {x-b.x, y-b.y}; }
6      cp operator* (cp b) const { return {x*b.x - y*b.y, x*b.y +

```

```

        y*b.x); }
};
using vcp = vector<cp>;
using Poly = vector<int>;
namespace FFT {
    const db PI = acos(-1.0);
    vcp Omega(int L) {
        vcp w(L);
        w[1] = 1;
        for (int i = 2; i < L; i <= 1) {
            auto w0 = w.begin() + i / 2;
            auto w1 = w.begin() + i;
            cp wn(cos(PI/i), sin(PI/i));
            for (int j = 0; j < i; j += 2) {
                w1[j] = w0[j]>>1;
                w1[j+1] = w1[j] * wn;
            }
        }
        return w;
    }
    auto W = Omega(1 << 21); // NOLINT
    void DIF(cp *a, int n) {
        cp x, y;
        for (int k = n>>1; k; k >>= 1) {
            for (int i = 0; i < n; i += k<<1) {
                for (int j = 0; j < k; ++j) {
                    x = a[i+j];
                    y = a[i+j+k];
                    a[i+j+k] = (a[i+j]-y) * W[k+j];
                    a[i+j] = x+y;
                }
            }
        }
    }
    void IDIT(cp *a, int n) {
        cp x, y;
        for (int k = 1; k < n; k <= 1) {
            for (int i = 0; i < n; i += k << 1) {
                for (int j = 0; j < k; ++j) {
                    x = a[i+j];
                    y = a[i+j+k] * W[k+j];
                    a[i+j+k] = x-y;
                    a[i+j] = x+y;
                }
            }
        }
        const db Inv = 1.0 / n;
        for (int i = 0; i < n; ++i) {
            a[i].x *= Inv;
            a[i].y *= Inv;
        }
        reverse(a+1, a+n);
    }
}

namespace Polynomial {
    void DFT(vcp &a) { FFT::DIF(a.data(), a.size()); }
    void IDFT(vcp &a) { FFT::IDIT(a.data(), a.size()); }
    int norm(int n) { return 1 << (lg(n-1) + 1); }
    vcp &dot(vcp &a, vcp &b) {
        for (int i = 0; i < int(a.size()); ++i) {
            a[i] = a[i] * b[i];
        }
        return a;
    }
    Poly operator+ (Poly a, Poly b) {
        int maxlen = max(a.size(), b.size());
        Poly ans(maxlen + 1);
        a.resize(maxlen + 1);
        b.resize(maxlen + 1);
        for (int i = 0; i < maxlen; ++i) { ans[i] = a[i] + b[i]; }
        return ans;
    }
    Poly operator* (ll k, Poly a) {
        Poly ans;
        for (auto i: a) { ans.push_back(k*i); }
        return ans;
    }
    Poly operator* (Poly a, Poly b) {
        int n = a.size() + b.size() - 1;
        vcp c(norm(n));
        for (int i = 0; i < int(a.size()); ++i) { c[i].x = a[i]

```

```

        for (int i = 0; i < int(b.size()); ++i) { c[i].y = b[i]
            ]; }
        DFT(c);
        dot(c, c);
        IDFT(c);
        a.resize(n);
        for (int i = 0; i < n; ++i) {
            a[i] = int(c[i].y * 0.5 + 0.5);
        }
        return a;
    }
}
using namespace Polynomial;

```

4.13 快速幂

```

int qmi(int m, int k, int p)
{
    int res = 1 % p, t = m;
    while (k)
    {
        if (k&1) res = res * t % p;
        t = t * t % p;
        k >>= 1;
    }
    return res;
}

```

4.14 乘法逆元

```

#include<bits/stdc++.h>
using namespace std;
typedef long long ll;

ll fpm(ll x, ll power, ll mod) {
    x %= mod;
    ll ans = 1;
    for (; power; power >>= 1, (x *= x) %= mod)
        if(power & 1) (ans *= x) %= mod;
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, p;
    cin >> n >> p;
    for(int a=1; a<=n; a++){
        cout << fpm(a, p - 2, p) << endl;
    } //x为a在mod p意义下的逆元
}

```

4.15 组合数求法

```

#include <iostream>
using namespace std;

typedef long long LL;

LL qmi(LL a, int k, int p)
{
    LL res = 1;
    while(k)
    {
        if (k & 1) res = res * a % p;
        k >>= 1;
        a = a * a % p;
    }
    return res;
}

int C(int a, int b, int p)
{
    if(a < b) return 0;

    LL res = 1;
    for(int i = 1, j = a; i <= b; i ++, j -- )
    {
        res = res * j % p;
        res = res * qmi(i, p - 2, p) % p;
    }
}

```

```

29     return res;
30 }
31
32 LL lucas(LL a, LL b, int p)
33 {
34     if(a < p && b < p) return C(a, b, p);
35     return C(a % p, b % p, p) * lucas(a / p, b / p, p) % p;
36 }
37
38 int main()
39 {
40     int n;
41     cin >> n;
42     while(n --)
43     {
44         LL a,b;
45         int p;
46         cin >> a >> b >> p;
47         cout << lucas(a, b, p) << endl;
48     }
49     return 0;
50 }

```

4.16 线性基

```

1 //普通消元法
2 void insert(long long x)
3 {
4     for(int i=63;~i;i--) if((x>>i)&1ll)
5     {
6         if(!place[i]) {place[i]=x;return;}
7         else x^=place[i];
8     }
9 }
10 void insert(int *num)
11 {
12     for(int j=63;~j;j--)
13     {
14         int p=0;
15         for(int i=1;(!p)&&i<=n;i++)
16             if((!v[i])&&((num[i]>>j)&1ll)) p=i;
17         if(!p) {place[j]=0;continue;}
18         v[p]=true,place[j]=num[p];
19         for(int i=1;i<=n;i++) if(i!=p&&((num[i]>>j)&1ll))
20             num[i]^=num[p];
21     }
22 }
23 bool check(long long x)
24 {
25     for(int i=63;~i;i--)
26         if((x>>i)&1ll) x^=place[i];
27     return x==0;
28 }
29 struct Base
30 {
31     long long place[N];
32     void insert(long long x)
33     {
34         for(int i=63;~i;i--) if((x>>i)&1ll)
35         {
36             if(!place[i]) {place[i]=x;return;}
37             else x^=place[i];
38         }
39     }
40 };
41 void merge(Base A,Base &B)
42 {
43     for(int i=63;~i;i--) if(A.place[i])
44         B.insert(A.place[i]);
45 }
46 long long mx()
47 {
48     long long ans=0;
49     for(int i=63;~i;i--)
50         if((ans^place[i])>ans)
51             ans^=place[i];
52     return ans;
53 }
54 long long mn()
55 {
56     for(int i=0;i<=63;i++)
57         if(place[i]) return place[i];
58     return -1;

```

```

59 }

```

4.17 矩阵快速幂及高斯消元

```

1 // 1.矩阵快速幂
2 #include <bits/stdc++.h>
3 using namespace std;
4 #define int long long
5 using ll = long long;
6 const int MOD = 1e9 + 7;
7 // 矩阵结构体
8 struct Matrix {
9     int n, m; // n: 行数, m: 列数
10     vector<vector<ll>>> a;
11     Matrix(int _n, int _m) : n(_n), m(_m), a(_n + 1, vector<ll>
12         (>(_m + 1, 0)) {}
13
14     static Matrix identity(int n) {
15         Matrix I(n, n);
16         for (int i = 1; i <= n; ++i) {
17             I.a[i][i] = 1;
18         }
19         return I;
20     };
21 // 矩阵乘法: C = A * B
22 // 时间复杂度: O(n^3) 或 O(A.n * A.m * B.m)
23 Matrix operator*(const Matrix& A, const Matrix& B) {
24     Matrix C(A.n, B.m);
25     for (int i = 1; i <= A.n; ++i) {
26         for (int j = 1; j <= B.m; ++j) {
27             for (int k = 1; k <= A.m; ++k) {
28                 C.a[i][j] = (C.a[i][j] + A.a[i][k] * B.a[k][j]
29                     ) % MOD;
30             }
31         }
32     }
33     return C;
34 }
35 // 矩阵快速幂: A^p, 时间复杂度: O(n^3 * Log p), A必须是方阵
36 Matrix power(Matrix A, ll p) {
37     Matrix res = Matrix::identity(A.n);
38     while (p) {
39         if (p & 1) res = res * A;
40         A = A * A;
41         p >>= 1;
42     }
43     return res;
44 }
45 signed main(){
46     ios::sync_with_stdio(false);
47     cin.tie(0), cout.tie(0);
48     int n, k; cin >> n >> k;
49     Matrix A(n, n);
50     for(int i = 1; i <= n; i++){
51         for(int j = 1; j <= n; j++){
52             cin >> A.a[i][j];
53         }
54     }
55     Matrix res = power(A, k);
56     for(int i = 1; i <= n; i++){
57         for(int j = 1; j <= n; j++){
58             cout << res.a[i][j] << " ";
59         }
60         cout << endl;
61     }
62 }

```

4.18 线性筛求质因子个数 (Omega 函数)

```

1 const int MAXN = 1e7 + 10;
2 vector<int> primes;
3 vector<int> omega(MAXN, 0);
4 vector<bool> is_comp(MAXN, false);
5 void sieve_omega(int n) {
6     omega[1] = 0;
7     for (int i = 2; i <= n; ++i) {
8         if (!is_comp[i]) { primes.push_back(i); omega[i] = 1;
9             }
10         for (int p : primes) {
11             long long nxt = 1LL * i * p;

```

```

11         if (nxt > n) break;
12         is_comp[nxt] = true;
13         omega[nxt] = omega[i] + 1;
14         if (i % p == 0) break;
15     }
16 }
17 }

```

4.19 线性筛 (O(n))

```

1 int n = 100;
2 vector<int> minp(n + 1, 0), prime;
3 //minp存的是该数的最小质因数
4 for(int i = 2; i <= n; i++){
5     if(minp[i] == 0){
6         minp[i] = i;
7         prime.push_back(i);
8     }
9     for(auto p : prime){
10         if(i * p > n) break;
11         minp[i * p] = p;
12         if(minp[i] == p) break;
13     }
14 }

```

4.20 位运算的部分性质

```

1 1. 基本代数性质
2 交换律
3 c
4 a & b == b & a
5 a | b == b | a
6 a ^ b == b ^ a
7 结合律
8 c
9 (a & b) & c == a & (b & c)
10 (a | b) | c == a | (b | c)
11 (a ^ b) ^ c == a ^ (b ^ c)
12 幂等律
13 c
14 a & a == a
15 a | a == a
16 a ^ a == 0
17 注意：异或不是幂等的。
18
19 单位元、零元
20 运算 单位元 零元
21 & M 全 1 0
22 | 0 M 全 1
23 ^ 0 无零元，每个数的逆元是自身
24 例如：
25
26 c
27 a & M == a
28 a | 0 == a
29 a ^ 0 == a
30 a ^ a == 0
31 互补律
32 c
33 a & ~a == 0
34 a | ~a == M
35 a ^ ~a == M
36 双重非
37 c
38 ~~a == a
39 吸收律
40 c
41 a & (a | b) == a
42 a | (a & b) == a
43 分配律
44 c
45 a & (b | c) == (a & b) | (a & c)
46 a | (b & c) == (a | b) & (a | c)
47 a & (b ^ c) == (a & b) ^ (a & c)
48 注意：^ 对 & 一般没有分配律，即：
49
50 c
51 a ^ (b & c) != (a ^ b) & (a ^ c)
52 德摩根定律
53 c
54 ~(a & b) == ~a | ~b
55 ~(a | b) == ~a & ~b

```

异或取反也有常用转换：

```

56
57 c
58 ~(a ^ b) == ~a ^ b == a ^ ~b
59 ~a ^ ~b == a ^ b
60
61 2. 与、或、非、异或之间的转换
62 与和或互相转换
63 c
64 a | b == ~(~a & ~b)
65 a & b == ~(~a | ~b)
66 异或用与、或、非表示
67 c
68 a ^ b == (a | b) & ~(a & b)
69 a ^ b == (a & ~b) | (~a & b)
70 a ^ b == (a | b) ^ (a & b)
71 a ^ b == (a | b) - (a & b)
72 或、与用异或表示
73 c
74 a | b == a ^ b ^ (a & b)
75 a & b == a ^ b ^ (a | b)
76 非与加减法的关系
77 c
78 ~a == -a - 1
79 -a == ~a + 1
80 a + ~a == M
81 a ^ M == ~a
82
83 3. 与算术运算的关系
84 加法
85 c
86 a + b == (a ^ b) + 2 * (a & b)
87 a + b == (a | b) + (a & b)
88 其中：
89 a ^ b 是不考虑进位的和；
90 a & b 是进位。
91
92 减法
93 c
94 a - b == a + ~b + 1
95 -a == ~a + 1
96 与、或、异或的算术关系
97 c
98 a | b == a + b - (a & b)
99 a & b == a + b - (a | b)
100 a ^ b == a + b - 2 * (a & b)
101
102 a & b == (a + b - (a ^ b)) / 2
103 a | b == (a + b + (a ^ b)) / 2
104 不重叠位的加法
105 如果：
106 c
107 a & b == 0
108 那么：
109
110 c
111 a + b == a | b == a ^ b
112
113 4. 移位与乘除的关系
114 左移
115 c
116 a << n == a * 2^n
117 在固定位宽中相当于对 2^w 取模。
118
119 右移
120 对于无符号数或非负数：
121
122 c
123 a >> n == a / 2^n // 向下取整
124 对于有符号补码数，算术右移等价于向下取整：
125
126 c
127 a >> n == floor(a / 2^n)
128 例如 -3 >> 1 == -2，而不是 C/C++ 中 -3 / 2 == -1，因为 C 语言
129 除法向零取整。
130
131 取模 2^n
132 c
133 a % 2^n == a & ((1 << n) - 1)
134 清零低 n 位
135 c
136 a & ~(1 << n) - 1)
137 循环移位
138 以位宽 w 为例：

```

```

139
140 c
141 // 循环左移 n 位
142 (a << n) | (a >> (w - n))
143
144 // 循环右移 n 位
145 (a >> n) | (a << (w - n))
146
147 5. 位运算与集合表示
148 一个整数的二进制位可以表示一个集合：第 i 位为 1 表示元素 i 在
    集合中。
149
150 c
151 交集： a & b
152 并集： a | b
153 对称差： a ^ b
154 补集： ~a & M
155 差集： a & ~b
156 判断子集：
157
158 c
159 a 是 b 的子集： (a & b) == a
160             或： (a | b) == b
161             或： (a & ~b) == 0
162
163 集合操作：
164
165 c
166 判断元素 i： (a >> i) & 1
167 添加元素 i： a | (1 << i)
168 删除元素 i： a & ~(1 << i)
169 翻转元素 i： a ^ (1 << i)
170 枚举一个集合的所有子集：
171
172 c
173 for (int s = a; s; s = (s - 1) & a) {
174     // s 是 a 的一个非空子集
175 }
176 // 最后还要处理空集 0
177
178 6. 常用位操作恒等式
179 目标 表达式
180 判断奇偶 x & 1
181 乘 2 x << 1
182 除 2 x >> 1
183 判断是否为 2 的幂 x > 0 && (x & (x - 1)) == 0
184 获取第 k 位 (x >> k) & 1
185 设置第 k 位为 1 x | (1 << k)
186 清除第 k 位 x & ~(1 << k)
187 翻转第 k 位 x ^ (1 << k)
188 取最低位 1 x & -x
189 清除最低位 1 x & (x - 1)
190 取最低位 0 ~x & (x + 1)
191 将最低位 0 变为 1 x | (x + 1)
192 将末尾连续 1 清零 x & (x + 1)
193 将末尾连续 0 置 1 x | (x - 1)
194 得到最低位 1 及以下全 1 掩码 x ^ (x - 1)
195 判断相等 (a ^ b) == 0
196 交换 a、b a ^= b; b ^= a; a ^= b;
197 平均值防溢出 (a & b) + ((a ^ b) >> 1)
198 相反数 -x == ~x + 1
199 绝对值, w 位有符号数 mask = x >> (w - 1); abs = (x ^ mask) -
    mask;
200
201 统计 1 的个数 while (x) { x &= x - 1; count++; }
202
203 7. 注意事项
204 有符号与无符号
205 右移分算术右移和逻辑右移。算术右移补符号位，逻辑右移补 0。
206
207 移位位数
208 C/C++ 中移位位数超过等于位宽是未定义行为；Java 等语言会取模。
209
210 优先级
211 位运算优先级容易出错，建议始终加括号，例如：
212
213 c
214 if ((x & 1) == 0)
215 补码性质
216 补码下：
217
218 -x == ~x + 1
219 ~x == -x - 1
220 逻辑运算与位运算不同
221 &&、||、! 是逻辑运算，结果只有 0 或 1，且会短路；
222 &、|、^、~ 是按位运算。
    如果 x,y 只取 0/1，则：
    x && y == x & y

```

```

221 x || y == x | y
222 !x == x ^ 1 == 1 - x

```

4.21 builtin

函数	功能	竞赛用途	典型代码
__builtin_popcount(x)	统计 32 位整数中 1 的个数	状压 DP、枚举子集、判断 2 的幂	<code>int c = __builtin_popcount(mask);</code>
__builtin_popcountll(x)	统计 64 位整数中 1 的个数	64 位掩码统计	<code>int c = __builtin_popcountll(mask);</code>
__builtin_clz(x)	统计 32 位前导零个数	求 $\log_2(x)$	<code>int lg = 31 - __builtin_clz(x);</code>
__builtin_clzll(x)	统计 64 位前导零个数	求 64 位 $\log_2(x)$	<code>int lg = 63 - __builtin_clzll(x);</code>
__builtin_ctz(x)	统计 32 位末尾零个数	求 lowbit、遍历掩码中的 1	<code>int i = __builtin_ctz(mask);</code>
__builtin_ctzll(x)	统计 64 位末尾零个数	64 位 lowbit	<code>int i = __builtin_ctzll(mask);</code>
__builtin_ffs(x)	返回最低位 1 的位置，从 1 开始	找最低位 1 的下标	<code>int pos = __builtin_ffs(mask);</code>
__builtin_ffsll(x)	64 位版 ffs	64 位找最低位 1	<code>int pos = __builtin_ffsll(mask);</code>
__builtin_parity(x)	判断 1 的个数奇偶性	奇偶校验	<code>int p = __builtin_parity(mask);</code>
__builtin_parityll(x)	64 位版 parity	64 位奇偶校验	<code>int p = __builtin_parityll(mask);</code>

5 字符串

5.1 前缀函数与 KMP

```

1 vector<int> prefixFunction(string s)
2 { // fail : 1-index
3     int n = s.length();
4     vector<int> fail(n + 1);
5     for (int i = 1, j = 0; i < n; ++i) {
6         while (j && s[i] != s[j]) { j = fail[j]; }
7         j += (s[i] == s[j]);
8         fail[i + 1] = j;
9     }
10    return fail;
11 }
12
13 void kmp()
14 {
15     string s, t;
16     cin >> s >> t;
17     int n = s.length();
18     int m = t.length();
19     auto fail = prefixFunction(t + "#" + s);
20     for (int i = m + 2; i <= n + m + 1; ++i) {
21         if (fail[i] == m) {
22             cout << i - 2 * m << "\n";
23         }
24     }
25     for (int i = 1; i <= m; ++i) { cout << fail[i] << " \n"[i == m]; }
26 }

```

5.2 Manacher

```

1 vector<int> manacher(string s)
2 {
3     vector<int> d(s.length());
4     d[0] = 1;
5     int r=0, mid=0;
6     for (int i = 1; i < s.length()-1; ++i) { // 最后一个字符
7         // 防越界使用，无需访问
8         d[i] = 1;
9         if (r >= i) { d[i] = min(d[2*mid-i], r-i+1); }
10        while (s[i-d[i]] == s[i+d[i]]) { ++d[i]; } // 需要保
11        // 证不会越界
12        if (i + d[i] > r) {
13            r = i + d[i] - 1;
14            mid = i;
15        }
16    }
17    return d;
18 }
19
20 void solve()
21 {
22     string s, t;

```

```

20     cin >> s;
21     for (auto c: s) {
22         t.push_back('#');
23         t.push_back(c);
24     }
25     t = "." + t + "#!"; // 首位添加两个不同的字符保证不会越界
26     vector<int> d = manacher(t);
27     cout << (*max_element(d.begin(), d.end())) - 1 << endl;
28 }

```

5.3 字符串哈希

```

1  typedef pair<ll, ll> hs;
2  const ll MOD1 = 1e9 + 7, MOD2 = 1e9 + 9;
3  const hs p = { 117, 131 };
4  hs operator+ (hs a, hs b) {
5      return hs{ (a.first + b.first) % MOD1, (a.second + b.second) %
6          MOD2 };
7  }
8  hs operator- (hs a, hs b) {
9      return hs{ (a.first - b.first + MOD1) % MOD1, (a.second - b.second
10         + MOD2) % MOD2 };
11 }
12 hs operator* (hs a, hs b) {
13     return hs{ a.first * b.first % MOD1, a.second * b.second % MOD2 };
14 }
15 struct Hash {
16     int n;
17     vector<hs> hs1, hs2, Pow;
18     Hash(string s) { // 0-index
19         n = s.length();
20         hs1.resize(n + 2);
21         hs2.resize(n + 2);
22         Pow.resize(n + 2);
23         Pow[0] = { 1LL, 1LL };
24         for (int i = 1; i <= n; ++i) {
25             Pow[i] = Pow[i - 1] * p;
26         }
27         for (int i = 1; i <= n; ++i) {
28             hs1[i] = hs1[i - 1] * p + hs{s[i - 1] - 'a' + 1, s[i - 1] - 'a'
29                 + 1};
30         }
31         for (int i = n; i; --i) {
32             hs2[i] = hs2[i + 1] * p + hs{s[i - 1] - 'a' + 1, s[i - 1] - 'a'
33                 + 1};
34         }
35     }
36     hs get1(int l, int r) {
37         return hs1[r] - hs1[l - 1] * Pow[r - l + 1];
38     }
39     hs get2(int l, int r) {
40         return hs2[l] - hs2[r + 1] * Pow[r - l + 1];
41     }
42 };

```

5.4 AC 自动机

```

1  struct AhoCorasick {
2      static constexpr int ALPHABET = 26;
3      struct Node {
4          int len;
5          int link;
6          array<int, ALPHABET> next;
7          Node(): len{0}, link{0}, next{} { }
8      };
9
10     vector<Node> t;
11
12     AhoCorasick() {
13         init();
14     }
15
16     void init() {
17         t.assign(2, Node());
18         t[0].next.fill(1);
19         t[0].len = -1;
20     }
21
22     int newNode() {
23         t.emplace_back();
24         return t.size() - 1;
25     }

```

```

26     int add(const string &a) {
27         int p = 1;
28         for (auto c: a) {
29             int x = c - 'a';
30             if (t[p].next[x] == 0) {
31                 t[p].next[x] = newNode();
32                 t[t[p].next[x]].len = t[p].len + 1;
33             }
34             p = t[p].next[x];
35         }
36         return p;
37     }
38
39     void work() {
40         queue<int> q;
41         q.push(1);
42
43         while (!q.empty()) {
44             int x = q.front();
45             q.pop();
46
47             for (int i = 0; i < ALPHABET; ++i) {
48                 if (t[x].next[i] == 0) {
49                     t[x].next[i] = t[t[x].link].next[i];
50                 } else {
51                     t[t[x].next[i]].link = t[t[x].link].next[i];
52                     q.push(t[x].next[i]);
53                 }
54             }
55         }
56     }
57
58     int next(int p, int x) {
59         return t[p].next[x];
60     }
61
62     int link(int p) {
63         return t[p].link;
64     }
65
66     int len(int p) {
67         return t[p].len;
68     }
69
70     int size() {
71         return t.size();
72     }
73 };
74
75 constexpr int N = 2e6 + 5;
76 vector<int> G[N];
77 int cnt[N];
78
79 void dfs(int u)
80 {
81     for (auto v: G[u]) {
82         dfs(v);
83         cnt[u] += cnt[v];
84     }
85 }
86
87 void solve()
88 {
89     int n;
90     cin >> n;
91
92     AhoCorasick AC;
93     vector<int> pos(n + 1);
94     for (int i = 1; i <= n; ++i) {
95         string s;
96         cin >> s;
97         pos[i] = AC.add(s);
98     }
99     AC.work();
100
101     string T;
102     cin >> T;
103     int p = 1;
104     for (int i = 0; i < int(T.size()); ++i) {
105         int x = T[i] - 'a';
106         p = AC.next(p, x);
107         ++cnt[p];
108     }

```

```

109     }
110
111     for (int i = 1; i < AC.size(); ++i) {
112         G[AC.link(i)].push_back(i);
113     }
114     dfs(1);
115
116     for (int i = 1; i <= n; ++i) {
117         cout << cnt[pos[i]] << "\n";
118     }
119 }

```

5.5 后缀自动机 SAM

```

1 //edited by piaoyun from some other's code
2 //必须#define int long long
3
4 struct SAM {
5     static const int N=1e6+5, S=28;
6     int tot=1, last=1, link[N<<1], ch[N<<1][S], len[N<<1],
7         endpos[N<<1];
8     // 总点数 tot, 点的 index 属于 [1-tot], 空串/根为 1
9     // last 为上一次插入的点
10    // link 为点的 parent 树父节点 / 最长 出现位置与自己不同 的
11    // 后缀
12    // ch[n][s] 指节点 n 末尾加字符 s 所转移到的点
13    // len 指该节点的串的最长长度, 注意到 最短长度 等于 len[
14    // link[n]] + 1 即父节点最长 + 1
15    // endpos[n] 参考 get_endpos() 的注释
16    void clear() {
17        for (int i = 0; i <= tot; ++i) {
18            link[i] = len[i] = endpos[i] = 0;
19            for (int k = 0; k < S; ++k) { ch[i][k] = 0; }
20        }
21        tot = 1;
22        last = 1;
23    }
24
25    // 延长一个字符, 通常为 [1-26]
26    void extend(int w) {
27        int p = ++tot, x = last, r, q;
28        endpos[p] = 1;
29        for (len[last=p]=len[x]+1; x&&!ch[x][w]; x=link[x]) {
30            ch[x][w]=p; }
31        if (!x) { link[p] = 1; }
32        else if (len[x] + 1 == len[q=ch[x][w]]) { link[p]=q; }
33        else {
34            link[r=++tot] = link[q];
35            memcpy(ch[r], ch[q], sizeof(ch[r]));
36            len[r] = len[x] + 1;
37            link[p] = link[q] = r;
38            for (; x && ch[x][w] == q; x = link[x]) { ch[x][w]
39                = r; }
40        }
41    }
42
43    // 注意 vector 占用的空间
44    vector<int> p[N<<1]; //建立 parent 树, 以便从上到下 dfs
45    void dfs(int u) {
46        int v;
47        for (int i = 0; i < int(p[u].size()); ++i) {
48            v = p[u][i];
49            dfs(v);
50            endpos[u] += endpos[v];
51        }
52    }
53
54    //注意! 在使用该方法前, endpos[] 代表每个点作为“终结点”
55    //的次数
56    //使用该方法后, endpos[] 指在串中出现总次数, 即原数组的子树
57    //求和
58    void get_endpos() {
59        for (int i = 1; i <= tot; ++i) { p[i].clear(); }
60        for (int i = 2; i <= tot; ++i) {
61            p[link[i]].push_back(i);
62        }
63        dfs(1);
64        for (int i = 1; i <= tot; ++i) { p[i].clear(); }
65    }
66
67    // 在您不确定是否有抄写错误时再使用该方法
68    // 必须在输入任何数据前自检, 此前的数据会被清空
69    static const int STC = 998244353;

```

```

63 void self_test() {
64     clear();
65     for (int i = 1; i <= 1000; ++i) { extend(i * i % 26 +
66         1); }
67     ll tmp = 107 * last + 301 * tot;
68     for (int i = 1; i <= tot; ++i) {
69         tmp = (tmp * 33 + link[i] * 101 + len[i] * 97) %
70             STC;
71         for (int k = 1; k < S; ++k) { tmp = (tmp + k * ch[
72             i][k]) % STC; }
73     }
74     assert("stage 1" && tmp == 393281314); // stage1 :
75         检查建树是否正确
76     tmp = 0;
77     get_endpos();
78     for (int i = 1; i <= tot; ++i) { tmp = (tmp * 33 +
79         endpos[i]) % STC; }
80     assert("stage 2" && tmp == 178417668); // stage2 :
81         检查endpos计算是否正确, 如果您修改了endpos[]的含
82         义则会报错
83     cout << "Self Test Passed. Remember to delete this
84         function's use. \n";
85     clear();
86 }
87
88 // 调试时可调用
89 void debug_print() {
90     for (int i = 1; i <= tot; ++i) {
91         cerr << "node : " << i << " father : " << link[i]
92             << " endpos : " << endpos[i] << " len : " <<
93             len[i] << "\n";
94     }
95 }
96
97 ll solve() {
98     // 在这里输入你自己的解题逻辑
99     ll ans = 0;
100    get_endpos();
101    for (int i = 1; i <= tot; ++i) {
102        if (endpos[i] >= 2) { ans = max(ans, (ll)endpos[i]
103            *len[i]); }
104    }
105    return ans;
106 }
107
108 void solve()
109 {
110     // sam.self_test();
111     string s; cin >> s;
112     for (auto c: s) { sam.extend(c-'a'+1); }
113     cout << sam.solve() << "\n";
114 }

```

5.6 KMP 算法

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 void get_nxt(string s, int nxt[]){
4     int j = 0;
5     nxt[0] = 0;
6     for (int i = 1; i < s.length(); i++)
7     {
8         while(j>0&&s[i]!=s[j]) j=nxt[j-1];
9         if(s[i]==s[j])j++;
10        nxt[i]=j;
11    }
12 }
13 void solve(){
14     int cnt=0;
15     string s, t;
16     cin >> s >> t;
17     int* nxt = new int[t.length()];
18     get_nxt(t, nxt);
19     int j=0;
20     for (int i = 0; i < s.length(); i++) //这里的s代表主串
21     {
22         while(j > 0 && s[i] != t[j]) j = nxt[j-1];
23         if(s[i]==t[j]) j++;
24         if(j==t.length()){
25             cout<<i-t.length()+2<<endl;
26         }
27     }

```

```

28     for (int i = 0; i < t.length(); i++){
29         cout<<nxt[i]<<' ';
30     }
31 }
32 int main(){
33     solve();
34 }

```

5.7 Trie 树

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 int tree[3000005][70],pass[3000005],endp[3000005],cnt=1;
4 int getidx(char ch){
5     // 映射函数
6     if(ch>='a'&&ch<='z') return ch-'a';
7     if(ch>='A'&&ch<='Z') return ch-'A'+26;
8     return ch-'0'+52;
9 }
10 void insert(string s){//插入操作
11     int cur=1;
12     pass[cur]++;
13     for(int i=0;i<s.length();i++){
14         int idx=getidx(s[i]);
15         if(tree[cur][idx]==0) {
16             tree[cur][idx]=++cnt;
17         }
18         cur=tree[cur][idx];
19         pass[cur]++;
20     }
21     endp[cur]++;
22 }
23 int search_whole_s(string s){
24     //查询整个字符串出现次数
25     int cur=1;
26     for(int i=0;i<s.length();i++){
27         int idx=getidx(s[i]);
28         if(tree[cur][idx]==0) return 0;
29         cur=tree[cur][idx];
30     }
31     return endp[cur];
32 }
33 int search_prefix_s(string s){
34     //查询s作为前缀字符串出现次数
35     int cur=1;
36     for(int i=0;i<s.length();i++){
37         int idx=getidx(s[i]);
38         if(tree[cur][idx]==0) return 0;
39         cur=tree[cur][idx];
40     }
41     return pass[cur];
42 }
43 void delete1(string s)
44 {
45     if(search_whole_s(s)>0)
46     {
47         int cur=1;
48         for(int i=0;i<s.length();i++)
49         {
50             int idx=getidx(s[i]);
51             if(--pass[tree[cur][idx]]==0)
52             {
53                 tree[cur][idx]=0;
54                 return;
55             }
56             cur=tree[cur][idx];
57         }
58         endp[cur]--;
59     }
60 }
61 void clear()
62 {
63     for(int i=1;i<=cnt;i++)
64     {
65         for(int j=0;j<=69;j++) tree[i][j]=0;
66         pass[i]=0;
67         endp[i]=0;
68     }
69     cnt=1;
70 }

```

5.8 string 常用函数

```

1 三、基本属性
2 s.size();        // 长度（常用）
3 s.length();      // 同 size()
4 s.empty();       // 是否为空
5 四、访问与修改
6 s[i];            // 访问（不检查越界）
7 s.at(i);         // 访问（会检查越界）
8
9 s.front();       // 第一个字符
10 s.back();        // 最后一个字符
11 五、拼接与修改
12 1. 拼接
13 s += "abc";
14 s = s + "abc";
15 2. 插入
16 s.insert(2, "abc"); // 在下标2插入
17 3. 删除
18 s.erase(2, 3);      // 从下标2开始删除3个字符
19 4. 替换
20 s.replace(2, 3, "abc"); // 替换区间
21 六、子串操作
22 string t = s.substr(pos, len); // 从pos开始，长度len
23
24 示例：
25
26 string s = "abcdef";
27 s.substr(2, 3);      // "cde"
28 七、查找操作（重点）
29 1. 查找子串
30 s.find("abc");       // 返回第一次出现的位置
31
32 找不到返回：
33
34 string::npos
35
36 示例：
37
38 if (s.find("abc") != string::npos) {
39     // 找到了
40 }
41 2. 反向查找
42 s.rfind("abc");      // 从后往前找
43 3. 查找字符
44 s.find('a');
45 八、比较
46 if (s == t) {}
47 if (s < t) {} // 字典序比较
48
49 底层是字典序（lexicographical order）
50
51 九、排序
52 #include <algorithm>
53
54 sort(s.begin(), s.end());
55
56 示例：
57
58 string s = "dcba";
59 sort(s.begin(), s.end()); // "abcd"
60 十、反转
61 reverse(s.begin(), s.end());
62 十一、大小写转换
63 #include <cctype>
64
65 for (char &c : s) c = toupper(c);
66 for (char &c : s) c = tolower(c);
67 十二、统计字符
68 int cnt = count(s.begin(), s.end(), 'a');

```

6 动态规划

6.1 数位 DP

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 using db = long double;
5 using i128 = __int128;
6
7 const int L=18;
8 int a[L+5];

```

```

9 ll f[L+5][15];
10 ll dfs(int p, bool limit, bool lead0, int last)
11 {
12     if (!p) { return 1; }
13     auto& now = f[p][last];
14     if (!limit && !lead0 && ~now) {
15         return now;
16     }
17     int up = limit ? a[p] : 9;
18     ll res = 0;
19     for (int i = 0; i <= up; ++i) {
20         if (!lead0 && abs(last-i) < 2) { continue; }
21         res += dfs(p-1, limit&&(i==up), lead0&&(i==0), i);
22     }
23     if (!limit && !lead0) {
24         now = res;
25     }
26     return res;
27 }
28 ll query(ll x)
29 {
30     int len = 0;
31     for (; x; x /= 10) {
32         a[++len] = x % 10;
33     }
34     return dfs(len, true, true, 0);
35 }
36
37 void solve()
38 {
39     ll a, b;
40     cin >> a >> b;
41     cout << query(b) - query(a-1) << "\n";
42 }
43
44 int main()
45 {
46     ios::sync_with_stdio(0); cin.tie(0); cout.tie(0);
47     memset(f, -1, sizeof(f)); // 不要忘记设为 -1
48     int t = 1;
49     // cin >> t;
50     while (t--) { solve(); }
51 }

```

6.2 单调队列优化 (滑动窗口)

```

1 auto work = [&](auto&& cmp) -> void {
2     deque<pair<int,int>> q;
3
4     auto update = [&](int id, int x) -> void {
5         while (!q.empty() && cmp(x, q.back().second)) {
6             q.pop_back();
7         }
8         q.emplace_back(id, x);
9     };
10
11     for (int i = 1; i <= k; ++i) {
12         update(i, a[i]);
13     }
14     for (int i = 1, j = i+k-1; j <= n; ++i, ++j) {
15         while (!q.empty() && q.front().first < i) { q.pop_front(); }
16         update(j, a[j]);
17         cout << q.front().second << " \n"[j == n];
18     }
19 };

```

6.3 无向图上回路计数

```

1 // CF11D
2 for (int i = 1; i <= n; ++i) {
3     f[1<<(i-1)][i] = 1;
4 }
5 for (int s = 0; s < (1<<n); ++s) {
6     for (int i = 1; i <= n; ++i) {
7         if (!f[s][i]) { continue; }
8         for (int j = 1; j <= n; ++j) {
9             if (!G[i][j]) { continue; }
10            if ((s&s) > 1<<(j-1)) { continue; } // 起点不能改
11            if (s>>(j-1)&1) {
12                if ((s&s) == 1<<(j-1)) { // 走回起点
13                    ans += f[s][i];

```

```

14        }
15        } else {
16            f[s|(1<<(j-1))][j] += f[s][i];
17        }
18    }
19 }
20 }
21 cout << (ans-m)/2 << "\n"; // 去除无向边, 环正走反走算一个

```

6.4 SOSDP

```

1 for(int k=0;k<=19;k++)
2     for(int i=0;i<=maxn;i++)
3         if((i>>k)&1) g[i] += g[i^(1<<k)]; // i 是超集,
// 有第 k 位
4 for(int k=0;k<=19;k++)
5     for(int i=0;i<=maxn;i++)
6         if(!(i>>k)&1) g[i] += g[i^(1<<k)]; // i 是子
// 集, 无第 k 位

```

6.5 树形 DP

```

1 #include<bits/stdc++.h>
2 //define int long long
3 #define in(a) a = read_int()
4 using namespace std;
5 const int N = 2e5 + 5;
6 const int inf = INT_MAX;
7 inline int read_int() {
8     int x = 0;
9     char ch = getchar();
10    bool f = 0;
11    while('9' < ch || ch < '0') f |= ch == '-', ch = getchar();
12    while('0' <= ch && ch <= '9') x = (x << 3) + (x << 1) + ch - '0', ch = getchar();
13    return f ? -x : x;
14 }
15 struct Edge {
16     int n, v, w;
17 } edge[N << 1];
18 int head[N], eid;
19 void eadd(int u, int v, int w) {
20     edge[++eid].n = head[u], edge[eid].v = v, edge[head[u]] = eid].w = w;
21 }
22 int n;
23 int l[N], r[N], ans[N];
24 void dfs(int u, int fa) { // 树形 DP 部分
25     l[u] = 1; // 初始化左边界
26     for(int i = head[u]; i; i = edge[i].n) {
27         int v = edge[i].v, w = edge[i].w;
28         if(v != fa) {
29             r[v] = w - 1; // 初始化右边界
30             dfs(v, u);
31             r[u] = min(r[u], w - l[v]); // 转移过程
32             l[u] = max(l[u], w - r[v]);
33         }
34     }
35 }
36 queue<int>q;
37 signed main() {
38     in(n);
39     for(int i = 1; i < n; i++) {
40         int u, v, w;
41         in(u), in(v), in(w);
42         eadd(u, v, w), eadd(v, u, w);
43     }
44     r[1] = 1e9; // 初始化根节点的右边界
45     dfs(1, 0); // 求每个点权的取值范围
46     for(int i = 1; i <= eid; i += 2) { // 判无解部分
47         int u = edge[i].v, v = edge[i+1].v, w = edge[i].w;
48         if(r[u] + r[v] < w || l[u] > r[u] || l[v] > r[v]) {
49             cout<<"NO";
50             return 0;
51         }
52     }
53     ans[1] = l[1]; // 若有解, 根节点点权取左边界
54     q.push(1);
55     while(!q.empty()) { // 广搜部分
56         int u = q.front();
57         q.pop();
58         for(int i = head[u]; i; i = edge[i].n) {

```

```

59     int v = edge[i].v , w = edge[i].w;
60     if(ans[v]) continue;
61     ans[v] = w - ans[u]; // 递推求解
62 }
63 }
64 cout<<"YES\n"; // 打印答案
65 for(int i = 1 ; i <= n ; i ++ ) cout<<ans[i]<<' ';
66 return 0;
67
68 }
69 /*
70 #include<bits/stdc++.h>
71 using namespace std;
72 const int maxn=10000;
73 int cnt=1,n,head[maxn],yes[maxn],no[maxn],num[maxn],b[maxn];
74 struct graph{
75     int next,to;
76 }g[maxn];
77 void add(int u,int v){
78     g[cnt].next=head[u];
79     g[cnt].to=v;
80     head[u]=cnt++;
81 }
82 void dfs(int i){
83     no[i]=0;
84     yes[i]=num[i];
85     for(int ei=head[i],v;ei;ei=g[ei].next){
86         v=g[ei].to;
87         dfs(v);
88         no[i]+=max(yes[v],no[v]);
89         yes[i]+=no[v];
90     }
91 }
92 int main(){
93     cin>>n;
94     for(int i=1;i<=n;i++) cin>>num[i];
95     for(int i=1;i<=n;i++){
96         int u,v;
97         cin>>u>>v;
98         b[u]=1;
99         add(v,u);
100     }
101     for(int i=1;i<=n;i++){
102         if(!b[i]){
103             dfs(i);
104             cout<<max(yes[i],no[i])<<endl;
105             break;
106         }
107     }
108 }
109 */

```

6.6 最长不上降子序列

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 const int N = 1e5+9;
5 int n,k,m,a[N],b[N],dp[N],t[N<2];
6
7 void updata(int i,int l,int r,int n,int val){
8     t[i]=max(val,t[i]);
9     if(l==r) return;
10    int mid=(l+r)>>1;
11    if(n<=mid) updata(i<<1,l,mid,n,val);
12    else updata(i<<1|1,mid+1,r,n,val);
13    return;
14 }
15 int getmax(int i,int l,int r,int nl,int nr){
16     if(l==nl&&r==nr){
17         return t[i];
18     }
19     int mid=(l+r)>>1;
20     if(mid>=nr) return getmax(i<<1,l,mid,nl,nr);
21     else if(mid<nl) return getmax(i<<1|1,mid+1,r,nl,nr);
22     else return max(getmax(i<<1,l,mid,nl,mid),getmax(i<<1|1,mid
23     +1,r,mid+1,nr));
24 }
25 void solve(){
26     cin>>n>>k;
27     for(int i=1;i<=n;i++)cin>>a[i],b[i]=a[i];
28     // 离散化

```

```

28     sort(b+1,b+n+1);
29     m=unique(b+1,b+n+1)-b-1;
30     for(int i=1;i<=n;i++)a[i]=lower_bound(b+1,b+m+1,a[i])-b;
31     for(int i=1;i<=n;i++){
32         dp[i]=getmax(1,1,m,1,a[i])+1;
33         updata(1,1,m,a[i],dp[i]);
34     }
35     //到这里dp 中存储的是以a[i]结尾的最长不上降子序列
36     int ans=0;
37     memset(t,0,sizeof t);
38     for(int i=n;i>k;i--){
39         ans=max(ans,dp[i-k]+k-1+getmax(1,1,m,a[i-k],m)+1); //
40         // 通过再次利用线段树找到往后最长 (题目要求)
41         int tem=getmax(1,1,m,a[i],m)+1;
42         updata(1,1,m,a[i],tem);
43     }
44     ans=max(ans,k+getmax(1,1,m,a[k+1],m));
45     cout<<ans<<'\n';
46 }
47 // 1 2 3 ... i-k i-k+1 i-k+2 i-k+3 ... i ...
48
49 int main(){
50     ios::sync_with_stdio(false),cin.tie(nullptr),cout.tie(
51     nullptr);
52     int q=1;
53     //cin>>q;
54     while(q--){
55         solve();
56     }
57     return 0;
58 }

```

7 计算几何

7.1 define

```

1 using ll = long long;
2 using db = double;
3 using T = ll; // 或者 double
4
5 // 如果 T 是整数
6 int sgn(ll x) {
7     return (x > 0) - (x < 0);
8 }
9
10 // 如果 T 是浮点数, 用这个版本
11 /*
12 const double eps = 1e-9;
13 int sgn(double x) {
14     if (fabs(x) < eps) return 0;
15     return x < 0 ? -1 : 1;
16 }
17 */

```

7.2 向量

```

1 using T = ll;
2 struct Point {
3     T x, y;
4     Point(T x=0, T y=0): x(x), y(y) {}
5     friend istream& operator>> (istream& is, Point& p) { return
6     is>>p.x>>p.y; }
7     friend ostream& operator<< (ostream& os, Point& p) { return
8     os<<p.x<<" "<<p.y; }
9     db ang() { return atan2(y, x); }
10    bool operator== (const Point &p) const { return sgn(x-p.x)
11    ==0 && sgn(y-p.y)==0; }
12    bool operator< (const Point &p) const { return x<p.x || (x==
13    p.x && y<p.y); }
14    Point operator+ (const Point &p) const { return Point(x+p.x,
15    y+p.y); }
16    Point operator- (const Point &p) const { return Point(x-p.
17    x, y-p.y); }
18    Point operator* (const T &k) const { return Point(k*x, k*y);
19    }
20    Point operator/ (const T &k) const { return Point(x/k, y/k);
21    }
22    T operator* (const Point &p) const { return x*p.x + y*p.y; }
23    T operator^ (const Point &p) const { return x*p.y - y*p.x; }
24    // 叉乘时打括号
25    T len2() { return (*this)*(*this); }
26 }

```

```

17 db len() { return sqrtl(len2()); } // 等价于 hypotl(x, y)
18 Point rot(db ang) { return Point(x*cos(ang)-y*sin(ang), x*
19     sin(ang)+y*cos(ang)); }
20 Point trunc(db l) { return (*this) * (l/len()); }
21 };
22 using Vector = Point;
23 // 判断 c 是否在 ab 的逆时针方向
24 int toLeft(Point a, Point b, Point c) { return sgn((b-a)^(c-a)
25     ); }
26 // 二维向量夹角
27 db getAngle(Vector a, Vector b) { return fabs(atan2(fabs(a^b),
28     a*b)); }

```

7.3 点线操作

```

1 struct Line {
2     Point a, b;
3     Vector v;
4     Line(Point a, Point b): a(a), b(b) { v = b - a; }
5     int toLeft(Point p) { return sgn(v^(p-a)); }
6     bool onSegment(Point p) { return (toLeft(p)==0) && ((p-a)*(p-
7         b)<=0); }
8     db distToLine(Point p) { return fabs((v^(p-a))/v.len()); }
9     Point proj(Point p) { return a+v*((v*(p-a))/(v*v)); }
10    // 用整数操作判断直线与点 C 的距离是否小于 (等于) 某个值 r
11    // 常见于判断直线与圆的位置关系
12    // 注意又乘再相乘可能会爆 LL, 所以这里用 __int128
13    bool disLess(Point p, int128 r) {
14        int128 xmult = (v^(p-a));
15        return xmult*xmult <= r*r*v.len2();
16    }
17 };
18 // 判断两条直线是否平行
19 bool isParallel(Line l1, Line l2)
20 {
21     return sgn(l1.v^l2.v)==0 && sgn(l1.v^(l1.a-l2.a))!=0;
22 }
23 // 判断两条线段是否相交
24 // 先做两次跨立实验判断是否规范相交, 再特判至少三点共线的情况
25 bool hasIntersection(Line l1, Line l2)
26 {
27     // 特判三点共线的情况
28     if (l1.onSegment(l2.a)) { return true; }
29     if (l1.onSegment(l2.b)) { return true; }
30     if (l2.onSegment(l1.a)) { return true; }
31     if (l2.onSegment(l1.b)) { return true; }
32     // 跨立实验判断两直线是否规范相交
33     // 线段端点分布在直线两侧, “线段与线段/线段与直线/直线与
34     // 直线”是否相交对应修改即可
35     function<bool(Line,Point,Point)> crossStanding = [](Line L,
36         Point a, Point b){
37         return L.toLeft(a) * L.toLeft(b) == -1;
38     };
39     return crossStanding(l1, l2.a, l2.b) && crossStanding(l2,
40         l1.a, l1.b);
41 }
42 // 求两直线交点
43 // 求之前应该先判断有没有交点, 避免精度误差, 同时还要特判共线
44 // 的情况
45 Point getIntersection(Line l1, Line l2)
46 {
47     Point P1=l1.a, P2=l2.a;
48     Point v1=l1.v, v2=l2.v;
49     return P1 + v1 * ( (v2^(P1-P2)) / (v1^v2) );
50 }
51 // 点P到线段AB的距离公式
52 double distToSeg(Point p, Point a, Point b) {
53     if(a == b) return (p-a).len();
54     Point v1=b-a, v2=p-a, v3=p-b;
55     if(sgn(v1*v2) < 0) return v2.len();
56     if(sgn(v1*v3) > 0) return v3.len();
57     return Line(a,b).distToLine(p);
58 }

```

7.4 多边形与凸包

```

1 struct Polygon: vector<Point> {
2     using vector<Point>::vector; // 直接使用 vector 的构造函数
3     size_t nxt(size_t i) { return i+1==size()? 0: i+1; }
4     size_t pre(size_t i) { return i==0? size()-1: i-1; }
5     // 求多边形面积
6     // 为避免精度误差, 可以返回 2*S, 即去掉 "0.5*()"
7     db area() {
8         db res = 0;
9         for (size_t i = 0; i < size(); ++i) {
10             res += 0.5 * ((*this)[i] ^ (*this)[nxt(i)]);
11         }
12         return fabs(res);
13     }
14     // Sunday's algorithm 光线回转法
15     // 在整数范围内判断一个点是否是多边形 (不保证凸包) 内部
16     // 亦可用于求某点相对于该多边形的回转数
17     pair<bool,int> winding(Point p) {
18         int wn = 0, n = this->size();
19         for (int i = 0; i < n; ++i) {
20             if (Line((*this)[i], (*this)[(i+1)%n]).onSegment(p)
21                 ) {
22                 return { true, -inf };
23             }
24             int k = Line((*this)[i], (*this)[(i+1)%n]).toLeft(
25                 p);
26             int d1 = sgn((*this)[i].y - p.y);
27             int d2 = sgn((*this)[(i+1)%n].y - p.y);
28             if (k>0 && d1<=0 && d2>0) { wn++; }
29             if (k<0 && d2<=0 && d1>0) { wn--; }
30         }
31         return { wn!=0, wn };
32     };
33 };

```

7.5 计算几何模板

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using point_t=long double; //全局数据类型, 可修改为 long long
5 // 等
6 constexpr point_t eps=1e-8;
7 constexpr long double PI=3.14159265358979323841;
8
9 // 点与向量
10 template<typename T> struct point
11 {
12     T x,y;
13
14     bool operator==(const point &a) const {return (abs(x-a.x)
15         <=eps && abs(y-a.y)<=eps);}
16     bool operator<(const point &a) const {if (abs(x-a.x)<=eps)
17         return y<a.y-eps; return x<a.x-eps;}
18     bool operator>(const point &a) const {return !(*this<a ||
19         *this==a);}
20     point operator+(const point &a) const {return {x+a.x,y+a.y
21         };}
22     point operator-(const point &a) const {return {x-a.x,y-a.y
23         };}
24     point operator-() const {return {-x,-y};}
25     point operator*(const T k) const {return {k*x,k*y};}
26     point operator/(const T k) const {return {x/k,y/k};}
27     T operator*(const point &a) const {return x*a.x+y*a.y;}
28     // 点积
29     T operator^(const point &a) const {return x*a.y-y*a.x;}
30     // 叉积, 注意优先级
31     int toleft(const point &a) const {const auto t=(*this)^a;
32         return (t>eps)-(t<-eps);} // to-Left 测试
33     T len2() const {return (*this)*(*this);} // 向量长度的平方
34     T dis2(const point &a) const {return (a-(*this)).len2();}
35     // 两点距离的平方
36
37     // 涉及浮点数
38     long double len() const {return sqrtl(len2());} // 向量长度
39     long double dis(const point &a) const {return sqrtl(dis2(a)
40         );} // 两点距离
41     long double ang(const point &a) const {return acosl(max
42         (-1.0l,min(1.0l,((*this)*a)/(len()*a.len()))));} //
43     // 向量夹角
44 }

```

```

32 point rot(const long double rad) const {return {x*cos(rad)
    -y*sin(rad),x*sin(rad)+y*cos(rad)};} // 逆时针旋转
    (给定角度)
33 point rot(const long double cosr,const long double sinr)
    const {return {x*cosr-y*sinr,x*sinr+y*cosr};} // 逆
    时针旋转 (给定角度的正弦与余弦)
34 };
35
36 using Point=point<point_t>;
37
38 // 极角排序
39 struct argcmp
40 {
41     bool operator()(const Point &a,const Point &b) const
42     {
43         const auto quad=[](const Point &a)
44         {
45             if (a.y<-eps) return 1;
46             if (a.y>eps) return 4;
47             if (a.x<-eps) return 5;
48             if (a.x>eps) return 3;
49             return 2;
50         };
51         const int qa=quad(a),qb=quad(b);
52         if (qa!=qb) return qa<qb;
53         const auto t=a^b;
54         return t>eps;
55     }
56 };
57
58 // 直线
59 template<typename T> struct line
60 {
61     point<T> p,v; // p 为直线上一点, v 为方向向量
62
63     bool operator==(const line &a) const {return v.toleft(a.v)
        ==0 && v.toleft(p-a.p)==0;}
64     int toleft(const point<T> &a) const {return v.toleft(a-p)
        ;} // to-left 测试
65     bool operator<(const line &a) const // 半平面交算法定义的
        排序
66     {
67         if (abs(v^a.v)<=eps && v*a.v>=-eps) return toleft(a.p)
            ==-1;
68         return argcmp()(v,a.v);
69     }
70
71     // 涉及浮点数
72     point<T> inter(const line &a) const {return p+v*((a.v^(p-a
        .p))/(v^a.v));} // 直线交点
73     long double dis(const point<T> &a) const {return abs(v^(a-
        p))/v.len();} // 点到直线距离
74     point<T> proj(const point<T> &a) const {return p+v*((v*(a-
        p))/(v*v));} // 点在直线上的投影
75 };
76
77 using Line=line<point_t>;
78
79 // 线段
80 template<typename T> struct segment
81 {
82     point<T> a,b;
83
84     bool operator<(const segment &s) const {return make_pair(a
        ,b)<make_pair(s.a,s.b);}
85
86     // 判定性函数建议在整数域使用
87
88     // 判断点是否在线段上
89     // -1 点在线段端点 / 0 点不在线段上 / 1 点严格在线段上
90     int is_on(const point<T> &p) const
91     {
92         if (p==a || p==b) return -1;
93         return (p-a).toleft(p-b)==0 && (p-a)*(p-b)<-eps;
94     }
95
96     // 判断线段直线是否相交
97     // -1 直线经过线段端点 / 0 线段和直线不相交 / 1 线段和直线
        严格相交
98     int is_inter(const line<T> &l) const
99     {
100         if (l.toleft(a)==0 || l.toleft(b)==0) return -1;
101         return l.toleft(a)!=l.toleft(b);
102     }

```

```

103 // 判断两线段是否相交
104 // -1 在某一线段端点处相交 / 0 两线段不相交 / 1 两线段严格
    相交
105
106 int is_inter(const segment<T> &s) const
107 {
108     if (is_on(s.a) || is_on(s.b) || s.is_on(a) || s.is_on(
        b)) return -1;
109     const line<T> l{a,b-a},ls{s.a,s.b-s.a};
110     return l.toleft(s.a)*l.toleft(s.b)==-1 && ls.toleft(a)
        *ls.toleft(b)==-1;
111 }
112
113 // 点到线段距离
114 long double dis(const point<T> &p) const
115 {
116     if ((p-a)*(b-a)<-eps || (p-b)*(a-b)<-eps) return min(p
        .dis(a),p.dis(b));
117     const line<T> l{a,b-a};
118     return l.dis(p);
119 }
120
121 // 两线段间距离
122 long double dis(const segment<T> &s) const
123 {
124     if (is_inter(s)) return 0;
125     return min({dis(s.a),dis(s.b),s.dis(a),s.dis(b)});
126 }
127 };
128
129 using Segment=segment<point_t>;
130
131 // 多边形
132 template<typename T> struct polygon
133 {
134     vector<point<T>> p; // 以逆时针顺序存储
135
136     size_t nxt(const size_t i) const {return i==p.size()-1?0:i
        +1;}
137     size_t pre(const size_t i) const {return i==0?p.size()-1:i
        -1;}
138
139     // 回转数
140     // 返回值第一项表示点是否在多边形边上
141     // 对于狭义多边形, 回转数为 0 表示点在多边形外, 否则点在多
        边形内
142     pair<bool,int> winding(const point<T> &a) const
143     {
144         int cnt=0;
145         for (size_t i=0;i<p.size();i++)
146         {
147             const point<T> u=p[i],v=p[nxt(i)];
148             if (abs((a-u)^(a-v))<=eps && (a-u)*(a-v)<=eps)
                return {true,0};
149             if (abs(u.y-v.y)<=eps) continue;
150             const Line uv={u,v-u};
151             if (u.y<v.y-eps && uv.toleft(a)<=0) continue;
152             if (u.y>v.y+eps && uv.toleft(a)>=0) continue;
153             if (u.y<a.y-eps && v.y>a.y-eps) cnt++;
154             if (u.y>a.y-eps && v.y<a.y-eps) cnt--;
155         }
156         return {false,cnt};
157     }
158
159     // 多边形面积的两倍
160     // 可用于判断点的存储顺序是顺时针或逆时针
161     T area() const
162     {
163         T sum=0;
164         for (size_t i=0;i<p.size();i++) sum+=p[i]^p[nxt(i)];
165         return sum;
166     }
167
168     // 多边形的周长
169     long double circ() const
170     {
171         long double sum=0;
172         for (size_t i=0;i<p.size();i++) sum+=p[i].dis(p[nxt(i)
        ]);
173         return sum;
174     }
175 };
176
177 using Polygon=polygon<point_t>;

```

```

178 //凸多边形
179 template<typename T> struct convex: polygon<T>
180 {
181     // 闵可夫斯基和
182     convex operator+(const convex &c) const
183     {
184         const auto &p=this->p;
185         vector<Segment> e1(p.size()),e2(c.p.size()),edge(p.size()+c.p.size());
186         vector<point<T>> res; res.reserve(p.size()+c.p.size());
187         ;
188         const auto cmp=[](const Segment &u,const Segment &v) {
189             return argcmp()(u.b-u.a,v.b-v.a);};
190         for (size_t i=0;i<p.size();i++) e1[i]={p[i],p[this->nxt(i)]};
191         for (size_t i=0;i<c.p.size();i++) e2[i]={c.p[i],c.p[c.nxt(i)]};
192         rotate(e1.begin(),min_element(e1.begin(),e1.end(),cmp),e1.end());
193         rotate(e2.begin(),min_element(e2.begin(),e2.end(),cmp),e2.end());
194         merge(e1.begin(),e1.end(),e2.begin(),e2.end(),edge.begin(),cmp);
195         const auto check=[](const vector<point<T>> &res,const point<T> &u)
196         {
197             const auto back1=res.back(),back2=*prev(res.end(),2);
198             return (back1-back2).topleft(u-back1)==0 && (back1-back2)*(u-back1)>=-eps;
199         };
200         auto u=e1[0].a+e2[0].a;
201         for (const auto &v:edge)
202         {
203             while (res.size())>1 && check(res,u)) res.pop_back();
204             res.push_back(u);
205             u=u+v.b-v.a;
206         }
207         if (res.size())>1 && check(res,res[0])) res.pop_back();
208         return {res};
209     }
210     // 旋转卡壳
211     // func 为更新答案的函数，可以根据题目调整位置
212     template<typename F> void rotcaliper(const F &func) const
213     {
214         const auto &p=this->p;
215         const auto area=[](const point<T> &u,const point<T> &v,const point<T> &w){return (w-u)^(w-v);};
216         for (size_t i=0,j=1;i<p.size();i++)
217         {
218             const auto nxti=this->nxt(i);
219             func(p[i],p[nxti],p[j]);
220             while (area(p[this->nxt(j)],p[i],p[nxti])>=area(p[j],p[i],p[nxti]))
221             {
222                 j=this->nxt(j);
223                 func(p[i],p[nxti],p[j]);
224             }
225         }
226     }
227     // 凸多边形的直径的平方
228     T diameter2() const
229     {
230         const auto &p=this->p;
231         if (p.size()==1) return 0;
232         if (p.size()==2) return p[0].dis2(p[1]);
233         T ans=0;
234         auto func=[](const point<T> &u,const point<T> &v,const point<T> &w){ans=max({ans,w.dis2(u),w.dis2(v)});};
235         rotcaliper(func);
236         return ans;
237     }
238     // 判断点是否在凸多边形内
239     // 复杂度 O(Logn)
240     // -1 点在多边形边上 / 0 点在多边形外 / 1 点在多边形内
241     int is_in(const point<T> &a) const
242     {
243         const auto &p=this->p;

```

```

246         if (p.size()==1) return a==p[0]?-1:0;
247         if (p.size()==2) return segment<T>{p[0],p[1]}.is_on(a)?-1:0;
248         if (a==p[0]) return -1;
249         if ((p[1]-p[0]).topleft(a-p[0])==-1 || (p.back()-p[0]).topleft(a-p[0])==1) return 0;
250         const auto cmp=[](const Point &u,const Point &v){return (u-p[0]).topleft(v-p[0])==1;};
251         const size_t i=lower_bound(p.begin()+1,p.end(),a,cmp)-p.begin();
252         if (i==1) return segment<T>{p[0],p[i]}.is_on(a)?-1:0;
253         if (i==p.size()-1 && segment<T>{p[0],p[i]}.is_on(a)) return -1;
254         if (segment<T>{p[i-1],p[i]}.is_on(a)) return -1;
255         return (p[i]-p[i-1]).topleft(a-p[i-1])>0;
256     }
257     // 凸多边形关于某一方向的极点
258     // 复杂度 O(Logn)
259     // 参考资料: https://codeforces.com/blog/entry/48868
260     template<typename F> size_t extreme(const F &dir) const
261     {
262         const auto &p=this->p;
263         const auto check=[](const size_t i){return dir(p[i]).topleft(p[this->nxt(i)]-p[i])>=0;};
264         const auto dir0=dir(p[0]); const auto check0=check(0);
265         if (!check0 && check(p.size()-1)) return 0;
266         const auto cmp=[](const Point &v)
267         {
268             const size_t vi=&v-p.data();
269             if (vi==0) return 1;
270             const auto checkv=check(vi);
271             const auto t=dir0.topleft(v-p[0]);
272             if (vi==1 && checkv==check0 && t==0) return 1;
273             return checkv^(checkv==check0 && t==0);
274         };
275         return partition_point(p.begin(),p.end(),cmp)-p.begin();
276     }
277     // 过凸多边形外一点求凸多边形的切线，返回切点下标
278     // 复杂度 O(Logn)
279     // 必须保证点在多边形外
280     pair<size_t,size_t> tangent(const point<T> &a) const
281     {
282         const size_t i=extreme([&](const point<T> &u){return u-a;});
283         const size_t j=extreme([&](const point<T> &u){return a-u;});
284         return {i,j};
285     }
286     // 求平行于给定直线的凸多边形的切线，返回切点下标
287     // 复杂度 O(Logn)
288     pair<size_t,size_t> tangent(const line<T> &a) const
289     {
290         const size_t i=extreme([&](...){return a.v;});
291         const size_t j=extreme([&](...){return -a.v;});
292         return {i,j};
293     }
294     };
295     using Convex=convex<point_t>;
296     // 圆
297     struct Circle
298     {
299         Point c;
300         long double r;
301         bool operator==(const Circle &a) const {return c==a.c && abs(r-a.r)<=eps;};
302         long double circ() const {return 2*PI*r;}; // 周长
303         long double area() const {return PI*r*r;}; // 面积
304         // 点与圆的关系
305         // -1 圆上 / 0 圆外 / 1 圆内
306         int is_in(const Point &p) const {const long double d=p.dis(c); return abs(d-r)<=eps?-1:d<r-eps;};
307         // 直线与圆关系
308         // 0 相离 / 1 相切 / 2 相交
309         int relation(const Line &l) const
310         {

```

```

319     const long double d=l.dis(c);
320     if (d>r+eps) return 0;
321     if (abs(d-r)<=eps) return 1;
322     return 2;
323 }
324
325 // 圆与圆关系
326 // -1 相同 / 0 相离 / 1 外切 / 2 相交 / 3 内切 / 4 内含
327 int relation(const Circle &a) const
328 {
329     if (*this==a) return -1;
330     const long double d=c.dis(a.c);
331     if (d>r+a.r+eps) return 0;
332     if (abs(d-r-a.r)<=eps) return 1;
333     if (abs(d-abs(r-a.r))<=eps) return 3;
334     if (d<abs(r-a.r)-eps) return 4;
335     return 2;
336 }
337
338 // 直线与圆的交点
339 vector<Point> inter(const Line &l) const
340 {
341     const long double d=l.dis(c);
342     const Point p=l.proj(c);
343     const int t=relation(l);
344     if (t==0) return vector<Point>();
345     if (t==1) return vector<Point>{p};
346     const long double k=sqrt(r*r-d*d);
347     return vector<Point>{p-(l.v/l.v.len())*k,p+(l.v/l.v.
348         len())*k};
349 }
350
351 // 圆与圆交点
352 vector<Point> inter(const Circle &a) const
353 {
354     const long double d=c.dis(a.c);
355     const int t=relation(a);
356     if (t== -1 || t==0 || t==4) return vector<Point>();
357     Point e=a.c-c; e=e/e.len()*r;
358     if (t==1 || t==3)
359     {
360         if (r*r+d*d-a.r*a.r== -eps) return vector<Point>{c+
361             e};
362         return vector<Point>{c-e};
363     }
364     const long double costh=(r*r+d*d-a.r*a.r)/(2*r*d),
365         sinth=sqrt(1-costh*costh);
366     return vector<Point>{c+e.rot(costh, -sinth),c+e.rot(
367         costh,sinth)};
368 }
369
370 // 圆与圆交面积
371 long double inter_area(const Circle &a) const
372 {
373     const long double d=c.dis(a.c);
374     const int t=relation(a);
375     if (t== -1) return area();
376     if (t<2) return 0;
377     if (t>2) return min(area(),a.area());
378     const long double costh1=(r*r+d*d-a.r*a.r)/(2*r*d),
379         costh2=(a.r*a.r+d*d-r*r)/(2*a.r*d);
380     const long double sinth1=sqrt(1-costh1*costh1),sinth2=
381         sqrt(1-costh2*costh2);
382     const long double th1=acos(costh1),th2=acos(costh2);
383     return r*r*(th1-costh1*sinth1)+a.r*a.r*(th2-costh2*
384         sinth2);
385 }
386
387 // 过圆外一点圆的切线
388 vector<Line> tangent(const Point &a) const
389 {
390     const int t=is_in(a);
391     if (t==1) return vector<Line>();
392     if (t== -1)
393     {
394         const Point v={-(a-c).y,(a-c).x};
395         return vector<Line>{{a,v}};
396     }
397     Point e=a-c; e=e/e.len()*r;
398     const long double costh=r/c.dis(a),sinth=sqrt(1-costh*
399         costh);
400     const Point t1=c+e.rot(costh, -sinth),t2=c+e.rot(costh,
401         sinth);
402     return vector<Line>{{a,t1-a},{a,t2-a}};

```

```

403 }
404
405 // 两圆的公切线
406 vector<Line> tangent(const Circle &a) const
407 {
408     const int t=relation(a);
409     vector<Line> lines;
410     if (t== -1 || t==4) return lines;
411     if (t==1 || t==3)
412     {
413         const Point p=inter(a)[0],v={-(a.c-c).y,(a.c-c).x
414             };
415         lines.push_back({p,v});
416     }
417     const long double d=c.dis(a.c);
418     const Point e=(a.c-c)/(a.c-c).len();
419     if (t<2)
420     {
421         const long double costh=(r-a.r)/d,sinth=sqrt(1-
422             costh*costh);
423         const Point d1=e.rot(costh, -sinth),d2=e.rot(costh,
424             sinth);
425         const Point u1=c+d1*r,u2=c+d2*r,v1=a.c+d1*a.r,v2=a
426             .c+d2*a.r;
427         lines.push_back({u1,v1-u1}); lines.push_back({u2,
428             v2-u2});
429     }
430     if (t==0)
431     {
432         const long double costh=(r+a.r)/d,sinth=sqrt(1-
433             costh*costh);
434         const Point d1=e.rot(costh, -sinth),d2=e.rot(costh,
435             sinth);
436         const Point u1=c+d1*r,u2=c+d2*r,v1=a.c-d1*a.r,v2=a
437             .c-d2*a.r;
438         lines.push_back({u1,v1-u1}); lines.push_back({u2,
439             v2-u2});
440     }
441     return lines;
442 }
443
444 // 圆的反演
445 tuple<int,Circle,Line> inverse(const Line &l) const
446 {
447     const Circle null_c={{0.0,0.0},{0.0}};
448     const Line null_l={{0.0,0.0},{0.0,0.0}};
449     if (l.toleft(c)==0) return {2,null_c,l};
450     const Point v=l.toleft(c)==1?Point{l.v.y,-l.v.x}:Point
451         {-l.v.y,l.v.x};
452     const long double d=r*r/l.dis(c);
453     const Point p=c+v/v.len()*d;
454     return {1,((c+p)/2,d/2),null_l};
455 }
456
457 tuple<int,Circle,Line> inverse(const Circle &a) const
458 {
459     const Circle null_c={{0.0,0.0},{0.0}};
460     const Line null_l={{0.0,0.0},{0.0,0.0}};
461     const Point v=a.c-c;
462     if (a.is_in(c)== -1)
463     {
464         const long double d=r*r/(a.r+a.r);
465         const Point p=c+v/v.len()*d;
466         return {2,null_c,{p,{-v.y,v.x}}};
467     }
468     if (c==a.c) return {1,{c,r*r/a.r},null_l};
469     const long double d1=r*r/(c.dis(a.c)-a.r),d2=r*r/(c.
470         dis(a.c)+a.r);
471     const Point p=c+v/v.len()*d1,q=c+v/v.len()*d2;
472     return {1,((p+q)/2,p.dis(q)/2),null_l};
473 }
474 }
475
476 // 圆与多边形面积交
477 long double area_inter(const Circle &circ,const Polygon &poly)
478 {
479     const auto cal=[](const Circle &circ,const Point &a,const
480         Point &b)
481     {
482         if ((a-circ.c).toleft(b-circ.c)==0) return 0.01;
483         const auto ina=circ.is_in(a),inb=circ.is_in(b);
484         const Line ab={a,b-a};
485         if (ina && inb) return ((a-circ.c)^(b-circ.c))/2;
486         if (ina && !inb)

```

```

466 {
467     const auto t=circ.inter(ab);
468     const Point p=t.size()==1?t[0]:t[1];
469     const long double ans=((a-circ.c)^(p-circ.c))/2;
470     const long double th=(p-circ.c).ang(b-circ.c);
471     const long double d=circ.r*circ.r*th/2;
472     if ((a-circ.c).toleft(b-circ.c)==1) return ans+d;
473     return ans-d;
474 }
475 if (!lina && inb)
476 {
477     const Point p=circ.inter(ab)[0];
478     const long double ans=((p-circ.c)^(b-circ.c))/2;
479     const long double th=(a-circ.c).ang(p-circ.c);
480     const long double d=circ.r*circ.r*th/2;
481     if ((a-circ.c).toleft(b-circ.c)==1) return ans+d;
482     return ans-d;
483 }
484 const auto p=circ.inter(ab);
485 if (p.size()==2 && Segment{a,b}.dis(circ.c)<=circ.r+
486     eps)
487 {
488     const long double ans=((p[0]-circ.c)^(p[1]-circ.c)
489         )/2;
490     const long double th1=(a-circ.c).ang(p[0]-circ.c),
491         th2=(b-circ.c).ang(p[1]-circ.c);
492     const long double d1=circ.r*circ.r*th1/2,d2=circ.r
493         *circ.r*th2/2;
494     if ((a-circ.c).toleft(b-circ.c)==1) return ans+d1+
495         d2;
496     return ans-d1-d2;
497 }
498 const long double th=(a-circ.c).ang(b-circ.c);
499 if ((a-circ.c).toleft(b-circ.c)==1) return circ.r*circ
500     .r*th/2;
501 return -circ.r*circ.r*th/2;
502 };
503 long double ans=0;
504 for (size_t i=0;i<poly.p.size();i++)
505 {
506     const Point a=poly.p[i],b=poly.p[poly.nxt(i)];
507     ans+=cal(circ,a,b);
508 }
509 return ans;
510 }
511 // 点集的凸包
512 // Andrew 算法, 复杂度  $O(n\log n)$ 
513 Convex convexhull(vector<Point> p)
514 {
515     vector<Point> st;
516     if (p.empty()) return Convex{st};
517     sort(p.begin(),p.end());
518     const auto check=[](const vector<Point> &st,const Point &u
519         )
520     {
521         const auto back1=st.back(),back2=*prev(st.end(),2);
522         return (back1-back2).toleft(u-back1)<=0;
523     };
524     for (const Point &u:p)
525     {
526         while (st.size()>1 && check(st,u)) st.pop_back();
527         st.push_back(u);
528     }
529     size_t k=st.size();
530     p.pop_back(); reverse(p.begin(),p.end());
531     for (const Point &u:p)
532     {
533         while (st.size()>k && check(st,u)) st.pop_back();
534         st.push_back(u);
535     }
536     st.pop_back();
537     return Convex{st};
538 }
539 // 半平面交
540 // 排序增量法, 复杂度  $O(n\log n)$ 
541 // 输入与返回值都是用直线表示的半平面集合
542 vector<Line> halfinter(vector<Line> l, const point_t lim=1e9)
543 {
544     const auto check=[](const Line &a,const Line &b,const Line
545         &c){return a.toleft(b.inter(c))<0;};
546     l.push_back({{-lim,0},{0,-1}}); l.push_back({{0,-lim

```

```

547     },{1,0}});
548     l.push_back({{lim,0},{0,1}}); l.push_back({{0,lim
549     },{-1,0}});
550     sort(l.begin(),l.end());
551     deque<Line> q;
552     for (size_t i=0;i<l.size();i++)
553     {
554         if (i>0 && l[i-1].v.toleft(l[i].v)==0 && l[i-1].v*[l[i]
555             ].v>eps) continue;
556         while (q.size()>1 && check(l[i],q.back(),q[q.size()
557             -2])) q.pop_back();
558         while (q.size()>1 && check(l[i],q[0],q[1])) q.
559             pop_front();
560         if (!q.empty() && q.back().v.toleft(l[i].v)<=0) return
561             vector<Line>();
562         q.push_back(l[i]);
563     }
564     while (q.size()>1 && check(q[0],q.back(),q[q.size()-2])) q
565         .pop_back();
566     while (q.size()>1 && check(q.back(),q[0],q[1])) q.
567         pop_front();
568     return vector<Line>(q.begin(),q.end());
569 }
570 // 点集形成的最小最大三角形
571 // 极角序扫描线, 复杂度  $O(n^2\log n)$ 
572 // 最大三角形问题可以使用凸包与旋转卡壳做到  $O(n^2)$ 
573 pair<point_t,point_t> minmax_triangle(const vector<Point> &vec
574     )
575 {
576     if (vec.size()<=2) return {0,0};
577     vector<pair<int,int>> evt;
578     evt.reserve(vec.size()*vec.size());
579     point_t maxans=0,minans=numeric_limits<point_t>::max();
580     for (size_t i=0;i<vec.size();i++)
581     {
582         for (size_t j=0;j<vec.size();j++)
583         {
584             if (i==j) continue;
585             if (vec[i]==vec[j]) minans=0;
586             else evt.push_back({i,j});
587         }
588     }
589     sort(evt.begin(),evt.end(),[&](const pair<int,int> &u,
590         const pair<int,int> &v)
591     {
592         const Point du=vec[u.second]-vec[u.first],dv=vec[v.
593             second]-vec[v.first];
594         return argcmp()({du.y,-du.x},{dv.y,-dv.x});
595     });
596     vector<size_t> vx(vec.size(),pos(vec.size()));
597     for (size_t i=0;i<vec.size();i++) vx[i]=i;
598     sort(vx.begin(),vx.end(),[&](int x,int y){return vec[x]<
599         vec[y];});
600     for (size_t i=0;i<vx.size();i++) pos[vx[i]]=i;
601     for (auto [u,v]:evt)
602     {
603         const size_t i=pos[u],j=pos[v];
604         const size_t l=min(i,j),r=max(i,j);
605         const Point vecu=vec[u],vecv=vec[v];
606         if (l>0) minans=min(minans,abs((vec[vx[l-1]]-vecu)^(
607             vec[vx[l-1]]-vecv)));
608         if (r<vx.size()-1) minans=min(minans,abs((vec[vx[r
609             +1]]-vecu)^(vec[vx[r+1]]-vecv)));
610         maxans=max(maxans,abs((vec[vx[0]]-vecu)^(vec[vx[0]]-
611             vecv)),abs((vec[vx.back()-vecu]^(vec[vx.back()-
612             vecv])));
613         if (i<j) swap(vx[i],vx[j]),pos[u]=j,pos[v]=i;
614     }
615     return {minans,maxans};
616 }
617 // 判断多条线段是否有交点
618 // 扫描线, 复杂度  $O(n\log n)$ 
619 bool segs_inter(const vector<Segment> &segs)
620 {
621     if (segs.empty()) return false;
622     using seq_t=tuple<point_t,int,Segment>;
623     const auto seqcmp=[](const seq_t &u, const seq_t &v)
624     {
625         const auto [u0,u1,u2]=u;
626         const auto [v0,v1,v2]=v;
627         if (abs(u0-v0)<=eps) return make_pair(u1,u2)<make_pair
628             (v1,v2);

```

```

609     return u0<v0-eps;
610 };
611 vector<seq_t> seq;
612 for (auto seg:segs)
613 {
614     if (seg.a.x>seg.b.x+eps) swap(seg.a,seg.b);
615     seq.push_back({seg.a.x,0,seg});
616     seq.push_back({seg.b.x,1,seg});
617 }
618 sort(seq.begin(),seq.end(),seqcmp);
619 point_t x_now;
620 auto cmp=[&](const Segment &u, const Segment &v)
621 {
622     if (abs(u.a.x-u.b.x)<=eps || abs(v.a.x-v.b.x)<=eps)
623         return u.a.y<v.a.y-eps;
624     return ((x_now-u.a.x)*(u.b.y-u.a.y)+u.a.y*(u.b.x-u.a.x)
625             )*(v.b.x-v.a.x)<((x_now-v.a.x)*(v.b.y-v.a.y)+v.a
626             .y*(v.b.x-v.a.x))*(u.b.x-u.a.x)-eps;
627 });
628 multiset<Segment,decltype(cmp)> s{cmp};
629 for (const auto [x,o,seg]:seq)
630 {
631     x_now=x;
632     const auto it=s.lower_bound(seg);
633     if (o==0)
634     {
635         if (it!=s.end() && seg.is_inter(*it)) return true;
636         if (it!=s.begin() && seg.is_inter(*prev(it)))
637             return true;
638         s.insert(seg);
639     }
640     else
641     {
642         if (next(it)!=s.end() && it!=s.begin() && (*prev(
643             it)).is_inter(*next(it))) return true;
644         s.erase(it);
645     }
646 }
647 return false;
648 }
649 // 多边形面积并
650 // 轮廓积分, 复杂度  $O(n^2 \log n)$ ,  $n$  为边数
651 //  $ans[i]$  表示被至少覆盖了  $i+1$  次的区域的面积
652 vector<long double> area_union(const vector<Polygon> &polys)
653 {
654     const size_t siz=polys.size();
655     vector<vector<pair<Point,Point>>> segs(siz);
656     const auto check=[&](const Point &u,const Segment &e){
657         return !((u<e.a && u<e.b) || (u>e.a && u>e.b));};
658
659     auto cut_edge=[&](const Segment &e,const size_t i)
660     {
661         const Line le{e.a,e.b-e.a};
662         vector<pair<Point,int>> evt;
663         evt.push_back({e.a,0}); evt.push_back({e.b,0});
664         for (size_t j=0;j<polys.size();j++)
665         {
666             if (i==j) continue;
667             const auto &pj=polys[j];
668             for (size_t k=0;k<pj.p.size();k++)
669             {
670                 const Segment s={pj.p[k],pj.p[pj.nxt(k)]};
671                 if (le.toleft(s.a)==0 && le.toleft(s.b)==0)
672                 {
673                     evt.push_back({s.a,0});
674                     evt.push_back({s.b,0});
675                 }
676                 else if (s.is_inter(le))
677                 {
678                     const Line ls{s.a,s.b-s.a};
679                     const Point u=le.inter(ls);
680                     if (le.toleft(s.a)<0 && le.toleft(s.b)>=0)
681                         evt.push_back({u,-1});
682                     else if (le.toleft(s.a)>=0 && le.toleft(s.
683                         b)<0) evt.push_back({u,1});
684                 }
685             }
686         }
687     }
688     sort(evt.begin(),evt.end());
689     if (e.a>e.b) reverse(evt.begin(),evt.end());
690     int sum=0;
691     for (size_t i=0;i<evt.size();i++)
692     {

```

```

685         sum+=evt[i].second;
686         const Point u=evt[i].first,v=evt[i+1].first;
687         if (! (u==v) && check(u,e) && check(v,e)) segs[sum
688             ].push_back({u,v});
689         if (v==e.b) break;
690     }
691 };
692 for (size_t i=0;i<polys.size();i++)
693 {
694     const auto &pi=polys[i];
695     for (size_t k=0;k<pi.p.size();k++)
696     {
697         const Segment ei={pi.p[k],pi.p[pi.nxt(k)]};
698         cut_edge(ei,i);
699     }
700 }
701 vector<long double> ans(siz);
702 for (size_t i=0;i<siz;i++)
703 {
704     long double sum=0;
705     sort(segs[i].begin(),segs[i].end());
706     int cnt=0;
707     for (size_t j=0;j<segs[i].size();j++)
708     {
709         if (j>0 && segs[i][j]==segs[i][j-1]) segs[i][++cnt
710             ].push_back(segs[i][j]);
711         else cnt=0,sum+=segs[i][j].first^segs[i][j].second
712             ;
713     }
714     ans[i]=sum/2;
715 }
716 // 圆面积并
717 // 轮廓积分, 复杂度  $O(n^2 \log n)$ 
718 //  $ans[i]$  表示被至少覆盖了  $i+1$  次的区域的面积
719 vector<long double> area_union(const vector<Circle> &circs)
720 {
721     const size_t siz=circs.size();
722     using arc_t=tuple<Point,long double,long double,long
723         double>;
724     vector<vector<arc_t>> arcs(siz);
725     const auto eq=[&](const arc_t &u,const arc_t &v)
726     {
727         const auto [u1,u2,u3,u4]=u;
728         const auto [v1,v2,v3,v4]=v;
729         return u1==v1 && abs(u2-v2)<=eps && abs(u3-v3)<=eps &&
730             abs(u4-v4)<=eps;
731     };
732     auto cut_circ=[&](const Circle &ci,const size_t i)
733     {
734         vector<pair<long double,int>> evt;
735         evt.push_back({-PI,0}); evt.push_back({PI,0});
736         int init=0;
737         for (size_t j=0;j<circs.size();j++)
738         {
739             if (i==j) continue;
740             const Circle &cj=circs[j];
741             if (ci.r<cj.r-eps && ci.relation(cj)>=3) init++;
742             const auto inters=ci.inter(cj);
743             if (inters.size()==1) evt.push_back({atan2l((
744                 inters[0]-ci.c).y,(inters[0]-ci.c).x),0});
745             if (inters.size()==2)
746             {
747                 const Point dl=inters[0]-ci.c,dr=inters[1]-ci.
748                     c;
749                 long double argl=atan2l(dl.y,dl.x),argr=atan2l
750                     (dr.y,dr.x);
751                 if (abs(argl+PI)<=eps) argl=PI;
752                 if (abs(argr+PI)<=eps) argr=PI;
753                 if (argl>argr+eps)
754                 {
755                     evt.push_back({argl,1}); evt.push_back({PI
756                         ,-1});
757                     evt.push_back({-PI,1}); evt.push_back({
758                         argr,-1});
759                 }
760             }
761             else
762             {
763                 evt.push_back({argl,1});
764                 evt.push_back({argr,-1});
765             }
766         }
767     }
768 }

```

```

759     }
760     }
761     }
762     sort(evt.begin(), evt.end());
763     int sum=init;
764     for (size_t i=0; i<evt.size(); i++)
765     {
766         sum+=evt[i].second;
767         if (abs(evt[i].first-evt[i+1].first)>eps) arcs[sum].push_back({ci.c, ci.r, evt[i].first, evt[i+1].first});
768         if (abs(evt[i+1].first-PI)<=eps) break;
769     }
770 };
771
772 const auto oint=[](const arc_t &arc)
773 {
774     const auto [cc, cr, l, r]=arc;
775     if (abs(r-l-PI-PI)<=eps) return 2.01*PI*cr*cr;
776     return cr*cr*(r-l)+cc.x*cr*(sin(r)-sin(l))-cc.y*cr*(cos(r)-cos(l));
777 };
778
779 for (size_t i=0; i<circs.size(); i++)
780 {
781     const auto &ci=circs[i];
782     cut_circ(ci, i);
783 }
784 vector<long double> ans(siz);
785 for (size_t i=0; i<siz; i++)
786 {
787     long double sum=0;
788     sort(arcs[i].begin(), arcs[i].end());
789     int cnt=0;
790     for (size_t j=0; j<arcs[i].size(); j++)
791     {
792         if (j>0 && eq(arcs[i][j], arcs[i][j-1])) arcs[i+cnt].push_back(arcs[i][j]);
793         else cnt=0, sum+=oint(arcs[i][j]);
794     }
795     ans[i]=sum/2;
796 }
797 return ans;
798 }

```

8 其它

8.1 二维差分与前缀和

```

1 void cf(int x1, int y1, int x2, int y2){
2     a[x1][y1]++;
3     a[x2+1][y2+1]++;
4     a[x2+1][y1]--;
5     a[x1][y2+1]--;
6 }
7
8 只对四边形四条边差分
9 cf(i, j, i+1, j);
10 cf(i, j+1, i, j+1);
11 cf(i+1, j+1, i+1, j+1);
12 cf(i+1, j+1, i+1-1, j+1);
13
14 前缀和
15 a[i][j] += a[i-1][j] + a[i][j-1] - a[i-1][j-1];

```

8.2 CDQ 分治

```

1 //3810
2
3 #include<bits/stdc++.h>
4 using namespace std;
5 const int maxn=1e5+10;
6 const int maxk=2e5+10;
7 struct node{
8     int idx, a, b, c;
9     bool operator==(const node& other) const {
10         return a == other.a && b == other.b && c == other.c;
11     }
12 }t[maxn];
13 int f[maxn], n, ans[maxk], k;
14 struct BIT{
15     int n;

```

```

16     vector<int>t;
17     BIT(int n):n(n), t(n+1){};
18     int lowbit(int x) {return x&-x;}
19     void modify(int x, int d){
20         for(; x<=n; x+=lowbit(x)) t[x]+=d;
21     }
22     int query(int x){
23         int res=0;
24         for(; x>=lowbit(x)) res+=t[x];
25         return res;
26     }
27 };
28 BIT T(maxk);
29 void prepare(){
30     sort(t+1, t+n+1, [](node a, node b){
31         if(a.a!=b.a) return a.a<b.a;
32         if(a.b!=b.b) return a.b<b.b;
33         return a.c<b.c;
34     });
35     for(int l=1, r=1; l<=n; l+=r){
36         while(r+1<=n&&t[l]==t[r+1]) r++;
37         for(int i=l; i<=r; i++){
38             f[t[i].idx]=r-i;
39         }
40     }
41 }
42 void merge(int l, int mid, int r){
43     int p, q;
44     for(p=l-1, q=mid+1; q<=r; q++){
45         while(p+1<=mid&&t[p+1].b<=t[q].b){
46             p++;
47             T.modify(t[p].c, 1);
48         }
49         f[t[q].idx]+=T.query(t[q].c);
50     }
51     for(int i=l; i<=p; i++){
52         T.modify(t[i].c, -1);
53     }
54     sort(t+1, t+r+1, [](node a, node b){
55         return a.b<b.b;
56     });
57 }
58 void cdq(int l, int r){
59     if(l==r) return;
60     int mid=(l+r)>>1;
61     cdq(l, mid);
62     cdq(mid+1, r);
63     merge(l, mid, r);
64 }
65
66 int main(){
67     cin>>n>>k;
68     for(int i=1; i<=n; i++){
69         cin>>t[i].a>>t[i].b>>t[i].c;
70         t[i].idx=i;
71     }
72     prepare();
73     cdq(1, n);
74     for(int i=1; i<=n; i++){
75         ans[f[i]]++;
76     }
77     for(int i=0; i<=k; i++) cout<<ans[i]<<endl;
78 }

```

8.3 矩阵加速

```

1 //1939
2 // #include <iostream>
3 // #include <cstdio>
4 // #include <cstring>
5
6 // using namespace std;
7
8 // typedef Long Long LL;
9 // const int Mod = 1e9+7;
10 // int T, n;
11 // struct mat{
12 //     LL m[5][5];
13 // }Ans, base;
14 // inline void init() {
15 //     memset(Ans.m, 0, sizeof(Ans.m));
16 //     for(int i=1; i<=3; i++) Ans.m[i][i] = 1;
17 //     memset(base.m, 0, sizeof(base.m));

```

```

18 // base.m[1][1] = base.m[1][3] = base.m[2][1] = base.m[3][2]
    // = 1;
19 // }
20 // inline mat mul(mat a, mat b) {
21 //     mat res;
22 //     memset(res.m, 0, sizeof(res.m));
23 //     for(int i=1; i<=3; i++) {
24 //         for(int j=1; j<=3; j++) {
25 //             for(int k=1; k<=3; k++) {
26 //                 res.m[i][j] += (a.m[i][k] % Mod) * (b.m[k][j] % Mod)
                // ;
27 //                 res.m[i][j] %= Mod;
28 //             }
29 //         }
30 //     }
31 //     return res;
32 // }
33 // inline void Qmat_pow(int p) {
34 //     while (p) {
35 //         if(p & 1) Ans = mul(Ans, base);
36 //         base = mul(base, base);
37 //         p >>= 1;
38 //     }
39 // }
40
41 // int main() {
42 //     scanf("%d", &T);
43 //     while (T--) {
44 //         scanf("%d", &n);
45 //         if(n <= 3) {
46 //             printf("1\n");
47 //             continue;
48 //         }
49 //         init();
50 //         Qmat_pow(n);
51 //         printf("%lld\n", Ans.m[2][1]);
52 //     }
53 // }
54 /*
55 f[i-1]      f[i]
56 f[i-2] ----> f[i-1]
57 f[i-3]      f[i-2]
58 f[i] = f[i-1] * 1 + f[i-2] * 0 + f[i-3] * 1
59 f[i-1] = f[i-1] * 1 + f[i-2] * 0 + f[i-3] * 0
60 f[i-2] = f[i-1] * 0 + f[i-2] * 1 + f[i-3] * 0
61 so
62 1 0 1
63 1 0 0
64 0 1 0
65 */
66 //P6569
67 #include<iostream>
68 #include<cstdio>
69 #include<cstring>
70 #include<algorithm>
71 using namespace std;
72 struct matrix{//用结构体存矩阵敲好使哒~
73     long long qwq[105][105];
74     int x,y;//x,y就是矩阵的行数和列数
75     matrix(){x=0,y=0;memset(qwq,0,sizeof(qwq));};//构造函数，刚
        //听说这玩意，用着玩玩
76 };
77 matrix operator * (const matrix &a,const matrix &b){//重载“异
    //或版矩阵乘法”的运算符
78     matrix c;
79     c.x=a.x,c.y=b.y;
80     for(int i=1;i<=a.x;i++){
81         for(int j=1;j<=b.y;j++){
82             for(int k=1;k<=a.y;k++){
83                 c.qwq[i][j]^=a.qwq[i][k]*b.qwq[k][j];//和矩阵乘法别无二
                    //致，只不过就是改成异或
84             }
85         }
86     }
87     return c;
88 }
89 matrix wyx[40];//存储e矩阵的2^i次方，取这个变量名是因为想借助
    //神仙的力量AC本题
90 long long f[105];//每个城市的初始魔法值（提醒：十年OI一场空
    //.....）
91 int main(){
92     int n,m,q;
93     scanf("%d%d%d",&n,&m,&q);
94     for(int i=1;i<=n;i++)
95         scanf("%lld",&f[i]);

```

```

95     wyx[0].x=n,wyx[0].y=n;//初始化一开始的邻接矩阵（也就是2的0次
        //方）的大小
96     for(int i=1;i<=m;i++){
97         int a,b;
98         scanf("%d%d",&a,&b);
99         wyx[0].qwq[a][b]=wyx[0].qwq[b][a]=1;//给邻接矩阵连上边
100     }
101     for(int i=1;i<32;i++)wyx[i]=wyx[i-1]*wyx[i-1];//利用快速幂的
        //思想，处理出邻接矩阵2的i次方
102     while(q--){
103         long long x;
104         scanf("%lld",&x);
105         matrix ans;
106         for(int i=1;i<=n;i++)ans.qwq[1][i]=f[i];//一开始就是初始f
            //值
107         ans.x=1,ans.y=n;
108         for(int i=0;i<32;i++){//开始进行二进制拆分
109             if((x>>i)&1)//如果这一位是1
110                 ans=ans*wyx[i];//就乘上对应的2的i次方
111         }
112         printf("%lld\n",ans.qwq[1][1]);
113     }
114     return 0;
115 }

```

8.4 cdq 斜率优化

```

1 #include<bits/stdc++.h>
2 using namespace std;
3
4 using ll=long long;
5 using i128=__int128_t;
6
7 const int maxn=5e4+10;
8 const i128 inf=((i128)1<<120);
9
10 int n;
11 ll L;
12 ll c[maxn],s[maxn];
13 i128 f[maxn];
14
15 struct node{
16     ll k;
17     i128 b;
18 };
19
20 vector<node> a,hull;
21
22 bool cmp(node x,node y){
23     if(x.k!=y.k)
24         return x.k<y.k;
25     return x.b<y.b;
26 }
27
28 i128 get(node x,ll p){
29     return (i128)x.k*p+x.b;
30 }
31
32 bool check(node a,node b,node c){
33     return (a.b-b.b)*(c.k-b.k)
34         >=(b.b-c.b)*(b.k-a.k);
35 }
36
37 void build(int l,int r){
38     a.clear();
39     hull.clear();
40
41     for(int i=l;i<=r;i++){
42         node x;
43         x.k=-2*s[i];
44         x.b=f[i]+(i128)s[i]*s[i];
45         a.push_back(x);
46     }
47
48     sort(a.begin(),a.end(),cmp);
49
50     for(auto x:a){
51         if(!hull.empty()&&hull.back().k==x.k)
52             continue;
53
54         while(hull.size()>=2&&
55             check(hull[hull.size()-2],hull.back(),x)){
56             hull.pop_back();

```

```

57     }
58     hull.push_back(x);
59 }
60 }
61 }
62
63 i128 query(ll x){
64     int l=0,r=(int)hull.size()-1;
65
66     while(l<r){
67         int mid=(l+r)>>1;
68
69         if(get(hull[mid],x)>=get(hull[mid+1],x))
70             l=mid+1;
71         else
72             r=mid;
73     }
74
75     return get(hull[l],x);
76 }
77
78 void CDQ(int l,int r){
79     if(l==r)
80         return;
81
82     int mid=(l+r)>>1;
83
84     CDQ(l,mid);
85
86     build(l,mid);
87
88     for(int i=mid+1;i<=r;i++){
89         ll x=s[i]-L;
90
91         i128 res=(i128)x*x+query(x);
92
93         f[i]=min(f[i],res);
94     }
95
96     CDQ(mid+1,r);
97 }
98
99 void print(i128 x){
100     if(x==0){
101         cout<<0;
102         return;
103     }
104
105     if(x<0){
106         cout<<"- ";
107         x=-x;
108     }
109
110     string s;
111
112     while(x){
113         s.push_back('0'+x%10);
114         x/=10;
115     }
116
117     reverse(s.begin(),s.end());
118     cout<<s;
119 }
120
121 int main(){
122     ios::sync_with_stdio(false);
123     cin.tie(nullptr);
124
125     cin>>n>>L;
126
127     s[0]=0;
128
129     for(int i=1;i<=n;i++){
130         cin>>c[i];
131         s[i]=s[i-1]+c[i]*i;
132     }
133
134     for(int i=0;i<=n;i++)
135         f[i]=inf;
136
137     f[0]=0;
138
139     CDQ(0,n);
140

```

```

141     print(f[n]);
142     cout<<"\n";
143
144     return 0;
145 }

```

8.5 并查集

```

1 //推导部分和
2 #include <bits/stdc++.h>
3 using namespace std;
4 typedef long long ll;
5 const ll N=1e5+10;
6 struct tree{
7     ll n,w,to;
8 }e[N<<1];
9 ll head[N<<1],tot,fa[N],vis[N],sum[N];
10 void add(int u,int v,ll w){
11     e[tot].to=v;
12     e[tot].n=head[u];
13     e[tot].w=w;
14     head[u]=tot++;
15 }
16 int find(int i){
17     if(fa[i]!=i) return fa[i]=find(fa[i]);
18     return i;
19 }
20 void unite(int x,int y){
21     x=find(x);
22     y=find(y);
23     if(x!=y) fa[x]=y;
24 }
25 void bfs(int x){
26     vis[x]=1;
27     queue<int>q; q.push(x);
28     sum[x]=0;
29     while(!q.empty()){
30         int u=q.front();
31         q.pop();
32         for(int i=head[u];i!=-1;i=e[i].n){
33             int v=e[i].to;
34             if(vis[v]) continue;
35             vis[v]=1;
36             sum[v]=sum[u]+e[i].w;
37             q.push(v);
38         }
39     }
40 }
41 int main()
42 {
43     memset(head,-1,sizeof(head));
44     int n,m,q;
45     cin>>n>>m>>q;
46     for(int i=1;i<=n;i++) fa[i]=i;
47     for(int i=1;i<=m;i++){
48         ll l,r,w;
49         cin>>l>>r>>w;
50         add(l-1,r,w);
51         add(r,l-1,-w);
52         unite(l-1,r);
53     }
54     for(int i=0;i<=n;i++){
55         if(!vis[i]) bfs(i);
56     }
57     for(int i=1;i<=m;i++){
58         int l,r;
59         cin>>l>>r;
60         if(find(l-1)!=find(r)) cout<<"UNKNOWN"<<endl;
61         else cout<<sum[r]-sum[l-1]<<endl;
62     }
63     return 0;
64 }

```

8.6 FFT (快速傅里叶变换)

```

1 // 当vector用就可以了
2 #include <bits/stdc++.h>
3 #define fp(i, a, b) for (int i = (a), i##_ = (b) + 1; i < i##_
4     ; ++i)
5 #define fd(i, a, b) for (int i = (a), i##_ = (b) - 1; i > i##_
6     ; --i)
7 using namespace std;

```

```

6 using ll = int64_t;
7 using db = double;
8 /*
9  */
10 struct cp {
11     db x, y;
12     cp(db real = 0, db imag = 0) : x(real), y(imag) {};
13     cp operator+(cp b) const { return {x + b.x, y + b.y}; }
14     cp operator-(cp b) const { return {x - b.x, y - b.y}; }
15     cp operator*(cp b) const { return {x * b.x - y * b.y, x *
16         b.y + y * b.x}; }
17 };
18 using vcp = vector<cp>;
19 using Poly = vector<int>;
20 namespace FFT {
21     const db pi = acos(-1);
22     vcp Omega(int L) {
23         vcp w(L);
24         w[1] = 1;
25         for (int i = 2; i < L; i <= 1) {
26             auto w0 = w.begin() + i / 2, w1 = w.begin() + i;
27             cp wn(cos(pi / i), sin(pi / i));
28             for (int j = 0; j < i; j += 2)
29                 w1[j] = w0[j >> 1], w1[j + 1] = w1[j] * wn;
30         }
31         return w;
32     }
33     auto W = Omega(1 << 21); // NOLINT
34     void DIF(cp *a, int n) {
35         cp x, y;
36         for (int k = n >> 1; k; k >>= 1)
37             for (int i = 0; i < n; i += k << 1)
38                 for (int j = 0; j < k; ++j)
39                     x = a[i + j], y = a[i + j + k],
40                     a[i + j + k] = (a[i + j] - y) * W[k + j], a[i
41                         + j] = x + y;
42     }
43     void IDIT(cp *a, int n) {
44         cp x, y;
45         for (int k = 1; k < n; k <= 1)
46             for (int i = 0; i < n; i += k << 1)
47                 for (int j = 0; j < k; ++j)
48                     x = a[i + j], y = a[i + j + k] * W[k + j], a[i
49                         + j + k] = x - y,
50                     a[i + j] = x + y;
51         const db Inv = 1. / n;
52         fp(i, 0, n - 1) a[i].x *= Inv, a[i].y *= Inv;
53         reverse(a + 1, a + n);
54     }
55 } // namespace FFT
56 namespace Polynomial {
57     // basic operator
58     void DFT(vcp &a) { FFT::DIF(a.data(), a.size()); }
59     void IDFT(vcp &a) { FFT::IDIT(a.data(), a.size()); }
60     int norm(int n) { return 1 << (lg(n - 1) + 1); }
61     // Poly mul
62     vcp &dot(vcp &a, vcp &b) {
63         fp(i, 0, a.size() - 1) a[i] = a[i] * b[i];
64         return a;
65     }
66     Poly operator+(Poly a, Poly b) {
67         int maxlen = max(a.size(), b.size());
68         Poly ans(maxlen + 1);
69         a.resize(maxlen + 1), b.resize(maxlen + 1);
70         for (int i = 0; i < maxlen; i++) ans[i] = a[i] + b[i];
71         return ans;
72     }
73     Poly operator*(ll k, Poly a) {
74         Poly ans;
75         for (auto i : a) ans.push_back(k * i);
76         return ans;
77     }
78     Poly operator*(Poly a, Poly b) {
79         int n = a.size() + b.size() - 1;
80         vcp c(norm(n));
81         fp(i, 0, a.size() - 1) c[i].x = a[i];
82         fp(i, 0, b.size() - 1) c[i].y = b[i];
83         DFT(c), dot(c, c), IDIT(c), a.resize(n);
84         fp(i, 0, n - 1) a[i] = int(c[i].y * .5 + .5);
85         return a;
86     }
87 }

```

```

85 } // namespace Polynomial
86 /*
87  */
88 using namespace Polynomial;

```

8.7 NTT (数论变换)

```

1 #include <bits/stdc++.h>
2 #define fp(i, a, b) for (int i = (a), i##_ = (b) + 1; i < i##_
3     ; ++i)
4 #define fd(i, a, b) for (int i = (a), i##_ = (b) - 1; i > i##_
5     ; --i)
6 using namespace std;
7 const int maxn = 2e5 + 5, P = 998244353;
8 using ll = int64_t;
9 #define ADD(a, b) (((a) += (b)) >= P ? (a) -= P : 0)
10 #define SUB(a, b) (((a) -= (b)) < 0 ? (a) += P : 0)
11 #define MUL(a, b) (ll(a) * (b) % P)
12 int POW(ll a, int b = P - 2, ll x = 1) {
13     for (; b; b >>= 1, a = a * a % P)
14         if (b & 1) x = x * a % P;
15     return x;
16 }
17 namespace NTT {
18     const int g = 3;
19     vector<int> Omega(int L) {
20         int wn = POW(g, P / L);
21         vector<int> w(L);
22         w[L >> 1] = 1;
23         fp(i, L / 2 + 1, L - 1) w[i] = MUL(w[i - 1], wn);
24         fd(i, L / 2 - 1, 1) w[i] = w[i << 1];
25         return w;
26     }
27     auto W = Omega(1 << 21);
28     void DIF(int *a, int n) {
29         for (int k = n >> 1; k; k >>= 1) {
30             for (int i = 0, y; i < n; i += k << 1)
31                 fp(j, 0, k - 1) y = a[i + j + k],
32                 a[i + j + k] = MUL(a[i + j] - y +
33                     P, W[k + j]),
34                 ADD(a[i + j], y);
35         }
36     }
37     void IDIT(int *a, int n) {
38         for (int k = 1; k < n; k <= 1) {
39             for (int i = 0, x, y; i < n; i += k << 1)
40                 fp(j, 0, k - 1) x = a[i + j], y = MUL(a[i + j + k
41                     ], W[k + j]),
42                 a[i + j + k] = x - y < 0 ? x - y +
43                     P : x - y,
44                 ADD(a[i + j], y);
45         }
46         int Inv = P - (P - 1) / n;
47         fp(i, 0, n - 1) a[i] = MUL(a[i], Inv);
48         reverse(a + 1, a + n);
49     }
50 } // namespace NTT
51 namespace Polynomial {
52     using Poly = std::vector<int>;
53     Poly &operator*=(Poly &a, int b) {
54         for (auto &x : a) x = MUL(x, b);
55         return a;
56     }
57     Poly operator*(Poly a, int b) { return a *= b; }
58     Poly operator*(int a, Poly b) { return b * a; }
59     void DFT(Poly &a) { NTT::DIF(a.data(), a.size()); }
60     void IDFT(Poly &a) { NTT::IDIT(a.data(), a.size()); }
61     int norm(int n) { return 1 << (32 - __builtin_clz(n - 1)); }
62     void norm(Poly &a) {
63         if (!a.empty()) a.resize(norm(a.size()), 0);
64     }
65     Poly &dot(Poly &a, Poly &b) {
66         fp(i, 0, a.size() - 1) a[i] = MUL(a[i], b[i]);
67         return a;
68     }
69     Poly operator*(Poly a, Poly b) {
70         int n = a.size() + b.size() - 1, L = norm(n);
71         if (a.size() <= 8 || b.size() <= 8) {
72             Poly c(n);
73             fp(i, 0, a.size() - 1) fp(j, 0, b.size() - 1) c[i + j]
74                 =

```

```

70         (c[i + j] + (ll)a[i] * b[j]) % P;
71         return c;
72     }
73     a.resize(L), b.resize(L);
74     DFT(a), DFT(b), dot(a, b), IDFT(a);
75     return a.resize(n), a;
76 }
77 } // namespace Polynomial
78 using namespace Polynomial;

```

8.8 扫描线算法

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4 ll arr[15010], n, height[15010];
5
6 struct node{
7     ll l, r, h;
8 } a[15010];
9
10 struct compare{
11     bool operator()(pair<ll, ll> a, pair<ll, ll> b){
12         if(a.first != b.first) return a.first < b.first;
13         return a.second > b.second;
14         // 大根堆
15     }
16 };
17 bool cmp(node x, node y){
18     return x.l < y.l;
19 }
20 int main(){
21     ll l, r, h, m = 0;
22     while(scanf("%lld %lld %lld", &l, &h, &r) != EOF){
23         a[++m].l = l;
24         a[m].r = r - 1;
25         a[m].h = h;
26         arr[++n] = l;
27         arr[++n] = r;
28         arr[++n] = r - 1;
29     }
30     sort(arr + 1, arr + n + 1);
31     ll len = unique(arr + 1, arr + n + 1) - arr - 1;
32     for(int j = 1; j <= m; j++){
33         a[j].l = lower_bound(arr + 1, arr + len + 1, a[j].l) - arr;
34         a[j].r = lower_bound(arr + 1, arr + len + 1, a[j].r) - arr;
35     }
36     sort(a + 1, a + m + 1, cmp);
37     priority_queue<pair<ll, ll>, vector<pair<ll, ll>>, compare>
38         heap;
39     for(int i = 1, j = 0; i <= len; i++){
40         for(; j <= m && a[j].l <= i; j++){
41             heap.push(make_pair(a[j].h, a[j].r));
42         }
43         while(!heap.empty() && heap.top().second < i)
44             heap.pop();
45     }
46     if(!heap.empty()){
47         height[i] = heap.top().first;
48     }
49 }
50 }
51 ll ansn = 0;
52 pair<ll, ll> ans[15010];
53 for(ll i = 1, pre = 0; i <= len; i++){
54     if(pre != height[i]){
55         ans[++ansn] = make_pair(arr[i], height[i]);
56     }
57     pre = height[i];
58 }
59 for(int i = 1; i <= ansn; i++){
60     cout << ans[i].first << ' ' << ans[i].second << ' ';
61 }
62 }

```

8.9 最长上升子序列

```

1 #include <cstdio>
2 #include <algorithm>
3 const int MAXN = 200001;
4 int a[MAXN];

```

```

5 int d[MAXN];
6 int main()
7 {
8     int n;
9     scanf("%d", &n);
10    for(int i = 1; i <= n; i++)
11        scanf("%d", &a[i]);
12    d[1] = a[1];
13    int len = 1;
14    for(int i = 2; i <= n; i++)
15    {
16        if(a[i] > d[len])
17            d[++len] = a[i];
18        else
19        {
20            int j = std::lower_bound(d + 1, d + len + 1, a[i]) - d;
21            d[j] = a[i];
22        }
23    }
24    printf("%d\n", len);
25    return 0;
26 }

```

8.10 全排列输出

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4
5 int main() {
6     std::vector<int> arr = {1, 2, 3}; // 示例数组
7
8     // 先排序（必须步骤，确保生成所有排列）
9     std::sort(arr.begin(), arr.end());
10
11    // 输出所有排列组合
12    do {
13        for (int num : arr) {
14            std::cout << num << " ";
15        }
16        std::cout << "\n";
17    } while (std::next_permutation(arr.begin(), arr.end()));
18
19    return 0;
20 }

```

8.11 位运算用法

```

1 /*为了更好的理解状压dp，首先介绍位运算相关的知识。
2
3 '&'符号，x&y，会将两个十进制数在二进制下进行与运算（都1为1，其余为0）然后返回其十进制下的值。例如3(11)&2(10)=2(10)。
4 '/'符号，x/y，会将两个十进制数在二进制下进行或运算（都0为0，其余为1）然后返回其十进制下的值。例如3(11)/2(10)=3(11)。
5 '^'符号，x^y，会将两个十进制数在二进制下进行异或运算（不同为1，其余为0）然后返回其十进制下的值。例如3(11)^2(10)=1(01)。
6 '~'符号，~x，按位取反。例如~101=010。
7 '<<'符号，左移操作，x<<2，将x在二进制下的每一位向左移动两位，最右边用0填充，x<<2相当于让x乘以4。'>>'符号，是右移操作，x>>1相当于给x/2，去掉x二进制下的最右一位
8 1. 判断一个数字x二进制下第i位是不是等于1。（最低第1位）
9
10 方法：if(((1<<(i-1))&x)>0) 将1左移i-1位，相当于制造了一个只有第i位上是1，其他位上都是0的二进制数。然后与x做与运算，如果结果>0，说明x第i位上是1，反之则是0。
11
12 2. 将一个数字x二进制下第i位更改成1。
13
14 方法：x=x|(1<<(i-1)) 证明方法与1类似。
15
16 3. 将一个数字x二进制下第i位更改成0。
17
18 方法：x=x& ~(1<<(i-1))
19
20 4. 把一个数字x二进制下最靠右的第一个1去掉。
21
22 方法：x=x&(x-1)
23 __builtin_popcount() 二进制数中1的个数
24 */
25 #include <iostream>
26 using namespace std;

```

```

27 struct state {
28     int st[1 << 12+1];
29     int num;
30 } a[15];
31 int m, n;
32 int f[15][1 << 12+1];
33 const int MOD = 100000000;
34 void getstate(int i, int t) {
35     a[i].num = 0;
36     for (int j = 0; j < (1 << n); j++) {
37         if ((j & (j << 1)) || (j & t))
38             continue;
39         a[i].st[++a[i].num] = j;
40     }
41 }
42 void dp() {
43     for (int i = 1; i <= m; i++)
44         for (int j = 1; j <= a[i].num; j++)
45             f[i][j] = 0;
46     for (int j = 1; j <= a[1].num; j++)
47         f[1][j] = 1;
48     for (int i = 2; i <= m; i++) {
49         for (int j = 1; j <= a[i].num; j++) {
50             int s1 = a[i].st[j];
51             for (int k = 1; k <= a[i-1].num; k++) {
52                 int s2 = a[i-1].st[k];
53                 if (s1 & s2) continue;
54                 f[i][j] = (f[i][j] + f[i-1][k]) % MOD;
55             }
56         }
57     }
58     int ans = 0;
59     for (int j = 1; j <= a[m].num; j++)
60         ans = (ans + f[m][j]) % MOD;
61     cout << ans << endl;
62 }
63 int main() {
64     cin >> m >> n;
65     for (int i = 1; i <= m; i++) {
66         int t = 0;
67         for (int j = 0; j < n; j++) {
68             int x;
69             cin >> x;
70             t = (t << 1) + (1 - x);
71         }
72         getstate(i, t);
73     }
74     dp();
75     return 0;
76 }

```

8.12 Map 的使用

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 int main(){
4     //什么是map? map是干什么用的
5     map<int,int>mp; //定义了一个名叫mp的map容器
6     mp[10001]=82;mp[100002]=82;
7     //输出10001信息
8     cout<<mp[10001]<<endl; //可能会溢出
9     //安全的方法
10    if(mp.find(10005)!=mp.end()){
11        cout<<mp[10005]<<endl;
12    }
13
14    else cout<<"没有此人信息"<<endl;
15    //储存学生姓名对应成绩
16    map<string,int>stu;
17    stu["xiaoming"]=85;
18    stu["xiaohong"]=77;
19    cout<<stu["xiaohong"]<<endl;
20    //map的遍历
21    //1.
22    for(auto it:stu){ //将容器从头到尾遍历了一遍
23        //map自动按照首位字典序从小到大排序
24        cout<<it.first<<" "<<it.second<<endl;
25    }
26    //2.迭代器类型
27    map<string,int>::iterator it; //定义map<string,int>类型的迭代器it
28    for(it=stu.begin();it!=stu.end();it++){
29        cout<<(*it).first<<" "<<(*it).second<<endl;

```

```

30 }
31 }

```

8.13 优先队列 (堆)

```

1 //自定义类的优先队列
2 struct Student {
3     std::string name;
4     int score;
5     int age;
6
7     // 重载 < 运算符, 用于大根堆 (按score从大到小)
8     bool operator<(const Student& other) const {
9         // 注意: priority_queue默认用 < 比较, 但实际表现是最大堆
10        // 所以这里返回 true 表示优先级更低
11        return score < other.score; // 大根堆
12    }
13 };
14 priority_queue<Student>q1; //大根堆
15 priority_queue<int>q2; //默认大根堆
16 priority_queue<int,vector<int>,greater<int>>q3; //小根堆

```

8.14 扫描线 + 线段树 (矩形面积并)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4 ll arr[15010], n, height[15010];
5
6 struct node{
7     ll l, r, h;
8 }a[15010];
9
10 struct compare{
11     bool operator()(pair<ll,ll> a,pair<ll,ll> b){
12         if(a.first!=b.first) return a.first<b.first;
13         return a.second>b.second;
14         //大根堆
15     }
16 };
17 bool cmp(node x,node y){
18     return x.l<y.l;
19 }
20 int main(){
21     ll l,r,h,m=0;
22     while(scanf("%lld %lld %lld",&l,&h,&r)!=EOF){
23         a[++m].l=l;
24         a[m].r=r-1;
25         a[m].h=h;
26         arr[++n]=1;
27         arr[++n]=r;
28         arr[++n]=r-1;
29     }
30     sort(arr+1,arr+n+1);
31     ll len=unique(arr+1,arr+n+1)-arr-1;
32     for(int j=1;j<=m;j++){
33         a[j].l=lower_bound(arr+1,arr+len+1,a[j].l)-arr;
34         a[j].r=lower_bound(arr+1,arr+len+1,a[j].r)-arr;
35     }
36     sort(a+1,a+m+1,cmp);
37     priority_queue<pair<ll,ll>, vector<pair<ll,ll>>, compare>
38     heap;
39     for(int i=1,j=0;i<=len;i++){
40         for(;j<=m&&a[j].l<=i;j++){
41             heap.push(make_pair(a[j].h,a[j].r));
42         }
43         while(!heap.empty()&&heap.top().second<i)
44             heap.pop();
45     }
46
47     if(!heap.empty()){
48         height[i]=heap.top().first;
49     }
50 }
51
52 ll ansn=0;
53 pair<ll,ll>ans[15010];
54 for(ll i=1,pre=0;i<=len;i++){
55     if(pre!=height[i]){
56         ans[++ansn]=make_pair(arr[i],height[i]);

```

```

57     pre=height[i];
58 }
59 for(int i=1;i<=ansn;i++){
60     cout<<ans[i].first<<' '<<ans[i].second<<' ';
61 }
62 }

```

8.15 建树参考

```

1 #include<bits/stdc++.h>
2 // #define int long long
3 #define in(a) a = read_int()
4 using namespace std;
5 const int N = 2e5 + 5;
6 const int inf = INT_MAX;
7 inline int read_int() {
8     int x = 0;
9     char ch = getchar();
10    bool f = 0;
11    while('0' < ch || ch < '9') f |= ch == '-', ch = getchar();
12    while('0' <= ch && ch <= '9') x = (x << 3) + (x << 1) + ch -
13        '0', ch = getchar();
14    return f ? -x : x;
15 }
16 struct Edge {
17     int n, v, w;
18 } edge[N << 1];
19 int head[N], eid;
20 void eadd(int u, int v, int w) {
21     edge[++eid].n = head[u], edge[eid].v = v, edge[eid].w = w;
22 }
23 int n;
24 int l[N], r[N], ans[N];
25 void dfs(int u, int fa) { // 树形 DP 部分
26     l[u] = 1; // 初始化左边界
27     for(int i = head[u]; i; i = edge[i].n) {
28         int v = edge[i].v, w = edge[i].w;
29         if(v != fa) {
30             r[v] = w - 1; // 初始化右边界
31             dfs(v, u);
32             r[u] = min(r[u], w - l[v]); // 转移过程
33             l[u] = max(l[u], w - r[v]);
34         }
35     }
36 }
37 queue<int> q;
38 signed main() {
39     in(n);
40     for(int i = 1; i < n; i++) {
41         int u, v, w;
42         in(u), in(v), in(w);
43         eadd(u, v, w), eadd(v, u, w);
44     }
45     r[1] = 1e9; // 初始化根节点的右边界
46     dfs(1, 0); // 求每个点权的取值范围
47     for(int i = 1; i <= eid; i += 2) { // 判无解部分
48         int u = edge[i].v, v = edge[i+1].v, w = edge[i].w;
49         if(r[u] + r[v] < w || l[u] > r[v] || l[v] > r[u]) {
50             cout<<"NO";
51             return 0;
52         }
53     }
54     ans[1] = l[1]; // 若有解，根节点点权取左边界
55     q.push(1);
56     while(!q.empty()) { // 广搜部分
57         int u = q.front();
58         q.pop();
59         for(int i = head[u]; i; i = edge[i].n) {
60             int v = edge[i].v, w = edge[i].w;
61             if(ans[v]) continue;
62             ans[v] = w - ans[u]; // 递推求解
63             q.push(v);
64         }
65     }
66     cout<<"YES\n"; // 打印答案
67     for(int i = 1; i <= n; i++) cout<<ans[i]<<' ';
68     return 0;
69 }

```

8.16 Hash 树

```

1 // 树同构
2
3 #include <bits/stdc++.h>
4
5 typedef long long LL;
6 const int N = 60, MO = 998244353;
7
8 struct Edge {
9     int nex, v;
10 } edge[N << 1]; int tp;
11
12 int e[N], n, m, turn, fr[N], p[1000010], top, siz[N], h[N], g[N];
13 std::vector<int> v[N];
14 bool vis[1000010];
15
16 inline void add(int x, int y) {
17     tp++;
18     edge[tp].v = y;
19     edge[tp].nex = e[x];
20     e[x] = tp;
21     return;
22 }
23
24 inline bool equal(int a, int b) {
25     int len = v[a].size();
26     if(len != v[b].size()) return false;
27     for(int i = 0; i < len; i++) {
28         if(v[a][i] != v[b][i]) return false;
29     }
30     return true;
31 }
32
33 inline void getp(int n) {
34     for(int i = 2; i <= n; i++) {
35         if(!vis[i]) {
36             p[++top] = i;
37             for(int j = 1; j <= top && i * p[j] <= n; j++) {
38                 vis[i * p[j]] = 1;
39                 if(i % p[j] == 0) {
40                     break;
41                 }
42             }
43         }
44     }
45     return;
46 }
47
48 void DFS_1(int x, int f) {
49     siz[x] = 1;
50     h[x] = 1;
51     for(int i = e[x]; i; i = edge[i].nex) {
52         int y = edge[i].v;
53         if(y == f) continue;
54         DFS_1(y, x);
55         h[x] = (h[x] + 1ll * h[y] * p[siz[y]] % MO) % MO;
56         siz[x] += siz[y];
57     }
58     return;
59 }
60
61 void DFS_2(int x, int f, int V) {
62     g[x] = (h[x] + 1ll * V * p[n - siz[x]] % MO) % MO;
63     v[turn].push_back(g[x]);
64     V = (1ll * V * p[n - siz[x]] % MO + 1) % MO;
65     for(int i = e[x]; i; i = edge[i].nex) {
66         int y = edge[i].v;
67         if(y == f) {
68             continue;
69         }
70         DFS_2(y, x, ((LL)V + h[x] - 1 - 1ll * h[y] * p[siz[y]] % MO + MO) % MO);
71     }
72     return;
73 }
74
75 int main() {
76     getp(1000009);
77     scanf("%d", &m);
78     for(turn = 1; turn <= m; turn++) {
79         scanf("%d", &n);
80         tp = 0;
81         memset(e + 1, 0, n * sizeof(int));
82         for(int i = 1, x; i <= n; i++) {

```

```
83         scanf("%d", &x);
84         if(x) {
85             add(x, 1);
86             add(i, x);
87         }
88     }
89     DFS_1(1, 0);
90     DFS_2(1, 0, 0);
91     std::sort(v[turn].begin(), v[turn].end());
92 }
93
94 for(int i = 1; i <= m; i++) {
95     fr[i] = i;
96 }
97 for(int i = 2; i <= m; i++) {
98     for(int j = 1; j < i; j++) {
99         if(equal(i, j)) {
100             fr[i] = fr[j];
101             break;
102         }
103     }
104 }
105 for(int i = 1; i <= m; i++) {
106     printf("%d\n", fr[i]);
107 }
108 return 0;
109 }
```